

RANDOMIZED DATA SPLITTING LEADS TO INFLATED PERFORMANCE FOR SEIZURE FORECASTING ALGORITHMS

by
Camden Shultz

A thesis submitted to The Johns Hopkins University in conformity
with the requirements for the degree of Master of Science in Engineering

Baltimore, Maryland
May 2024

© 2024 Camden Shultz
All rights reserved

Abstract

Recent advancements in machine learning (ML) have facilitated personalized treatments across various domains, particularly in healthcare. In this thesis, we present the application of seizure prediction in which we analyze electrographic signals to predict when a seizure is likely to occur. To begin, we outline Toast, a Python framework specifically designed for ML researchers to efficiently develop, train, and assess neural networks using patient-specific datasets. Unique to Toast are features like experiment management, simple network construction, and optimized data handling for time series data, setting it apart from existing libraries. We delve into the challenges faced by seizure forecasting algorithms in managing epilepsy, emphasizing the necessity of timely warnings and proactive interventions. Addressing the historical struggle of algorithms when applied to unseen, real-time data, we propose alternative data-splitting strategies to counter the naivety of traditional ML algorithms, which often assume data points are independent. Specifically, we introduce two alternative strategies—a time-based split and a semi-divided split—which provide more accurate representations of the underlying data compared to randomized sampling strategies and yield performance expected in deployed applications. Furthermore, we discuss model-agnostic approaches for tracking distributional shifts of input features and internal representations using techniques such as multivariate PCA reconstruction error, t-SNE visualizations, and optimal transport-based distance measurements across time and patients. We demonstrate the effectiveness of these measures in correlating with a model’s performance on held-out data, thus highlighting their utility in predicting algorithmic robustness and generalization capabilities. Driven by the innovative use of

Abstract

modular implantable devices aimed at understanding and treating neural pathologies, lightweight, robust, and interpretable ML models customized for seizure forecasting hold promise in bridging two pivotal clinical phases: sensing and intervention within a closed-loop system.

Keywords: Machine Learning, Distribution Shifts, Seizure Prediction

Primary reader and thesis advisor

Dr. Pedro Irazoqui
Professor
Department of Electrical Engineering
Cybernetics Implantable Devices Lab,
Johns Hopkins University, Baltimore, MD

Secondary reader and thesis supervisor

Dr. David Yarowsky
Professor
Department of Computer Science
Center for Language and Speech Processing,
Johns Hopkins University, Baltimore MD

This thesis is dedicated to my parents

Acknowledgement

First and foremost, I would like to express my deepest gratitude to my advisor, Dr. Pedro Irazoqui, for his unwavering support, guidance, and encouragement throughout the entirety of this research endeavor. Pedro and I met over a semester’s worth of office hours. One hour at a time, we chatted about almost everything except the course itself — the future of wearable devices, custom ASICs, and whatever we probably last read on Ars Technica. His expertise and optimism have been a significant driving force behind this thesis and two-and-a-half fulfilling years of research. He allowed me to conduct impact-driven research within the Cybernetics Implantable Devices Lab, provide a platform to develop my research questions, apply for fellowships, and forge collaborations with other institutions.

I’d also like to express my sincerest gratitude to my thesis supervisor and computer science advisor, Dr. David Yarowsky, for his feedback and help throughout my undergraduate and graduate career. David has been a familiar face for many years and I appreciate the time, effort, and advice he has provided.

Lastly, special thanks are due to my friends, colleagues, and co-authors who have provided excellent guidance and insightful discussions, which have greatly contributed to the completion of this thesis. In particular, I’d like to thank Dr. Trevor Meyer whose expertise and willingness to share knowledge have made him an indispensable resource in navigating the intricate landscape of machine learning in medical applications.

Table of Contents

Abstract	ii
Dedication	iv
Acknowledgement	v
List of Tables	ix
List of Figures	x
List of Source Code	xiii
Chapter 1 Introduction and Motivating Applications for Machine Learning In Seizure Prediction	1
1.1 Background and preliminaries	3
1.1.1 Seizure Types	3
1.1.2 Focal seizure generation and propagation	5
1.1.3 Electrical Signatures of Seizures	6
1.1.4 Temporal Non-Stationarity	6
1.1.5 Pre-ictal classification: One size does not fit all	7
1.2 Overview of this Thesis	7
1.3 Publications and code	10
Chapter 2 Toast: A PyTorch package for repeatable deep learning experiments	11
2.1 Introduction and Motivation	11
2.1.1 Requirements	11
2.2 Toast design model	13
2.2.1 Kitchen: Parallelized Training	15
2.2.2 Heat: Deep Neural Networks	19

Table of Contents

2.2.3	Bread: Datasets & Dataloaders	25
2.2.4	Butter: Signal Processing	31
2.2.5	Plate & Critic: Evaluation and Tracking	35
2.2.6	Weights & Biases Integration	38
2.3	Time Scale Network: A shallow neural network for time series data .	42
2.3.1	Toast Experiments with the Time Scale Network	43
2.4	Discussion and future work	47
Chapter 3	Seizure prediction and non-stationary machine learning	48
3.1	Introduction	48
3.2	Background And Related Work	50
3.3	Data Splitting Strategies	56
3.3.1	Selection of EEG recordings	58
3.4	Methodology and Results	62
3.4.1	Impact On Accuracy Deflation	63
3.4.2	Temporal Non-Stationary	68
3.5	Discussion	80
3.6	Conclusion	81
Chapter 4	Conclusion	82
	Bibliographic references	85
Appendix A	Supplementary Material	96
A.1	Datasets and Training	96
A.1.1	EEG Channels	96
A.1.2	Optimization and Training Details	97
A.2	Additional Results	98
A.2.1	Vanderbilt University Medical Center (Some)	98
A.2.2	CHB-MIT (Some)	98

Table of Contents

A.2.3	Vanderbilt University Medical Center (VUMC) Dataset Figures (25 Per)	98
A.2.4	CHB-MIT (25 Per)	99
A.2.5	Vanderbilt University Medical Center (VUMC) Dataset Figures (All)	99
A.2.6	CHB-MIT (all)	99
A.3	Robustness of Optimal Transport Distance to Number of Samples . .	107
A.4	Optimal Transport Drift Detection Examples	107

List of Tables

Table 2.1	Times to import 15 sec of data 1000 times.	26
Table 2.2	Times to import entire data file, replicating a typical machine learning pipeline.	26
Table 2.3	Standardized .wav Format	27
Table 2.4	Toast label format	28
Table 3.1	Number of samples for each subject in the VUMC dataset . . .	60
Table 3.2	Number of samples for each subject in the CHB-MIT dataset .	61
Table 3.3	Average Model Performance across models and splits for CHB-MIT and VUMC.	64
Table 3.4	Full Subject Results VUMC	66
Table 3.5	Full Subject Results CHBMIT	67
Table 3.6	Relative PCA reconstruction error between test and training splits.	73
Table 3.7	Relative PCA reconstruction error between test and training splits for subjects in VUMC	74
Table 3.8	Relative PCA reconstruction error between test and training splits for subjects in CHB-MIT.	75
Table A.1	CHBMIT Subject Info	100

List of Figures

Figure 2.1	Toast Package	13
Figure 2.2	Toast Kitchen submodule	15
Figure 2.3	GPU utilization while training 800 individual networks	17
Figure 2.4	GPU memory allocation while training 800 individual networks.	17
Figure 2.5	System RAM (non-swap) allocation while training 800 individual networks.	18
Figure 2.6	Toast Heat submodule	19
Figure 2.7	Toast Bread submodule	25
Figure 2.8	Toast.Bread: Temporal split: Training, Test, and Validation split for a subject in CHB-MIT.	29
Figure 2.9	Generated Plots for groups 0-3 of 10-Fold cross-validation in a temporal split	30
Figure 2.10	Toast Butter submodule	31
Figure 2.11	Output of example 2.8.	33
Figure 2.12	Automatic weight and activation tracking for during training.	36
Figure 2.13	Critic Plot: Plot subject training, validation, and test data and the resulting class balance for each split.	37
Figure 2.14	W&B Dashboard: 10-Fold cross-validation accuracy across all subjects for each learning rate.	39
Figure 2.15	W&B Dashboard: 10-Fold cross-validation accuracy across all hyperparameters for each subject in CHB-MIT.	39

List of Figures

Figure 2.16	W&B Dashboard: System resource tracking	40
Figure 2.17	W&B Dashboard: A simple sweep across learning rates for each subject in CHB-MIT.	40
Figure 2.18	Time Scale Network Architecture.	42
Figure 3.1	Illustration of random, divided, and temporal splits.	56
Figure 3.2	The in-distribution (Random i.i.d.) vs out-of-distribution (Divided, Temporal) F1 scores	65
Figure 3.3	Distribution Shift in embeddings: t-SNE dimensionality reduction of activations	70
Figure 3.4	Dataset distance ratio vs data split for VUMC and CHBMIT.	78
Figure 3.5	Relative Optimal Transport Distance between reference distribution (P_{train}) and chunks from the test distribution (P_{test})	79
Figure A.1	Vanderbuilt University Medical Center (VUMC) Dataset for subjects that contain both interictal and pre-ictal segments in both training and testing sets.	101
Figure A.2	CHB-MIT Dataset for subjects that contain both interictal and pre-ictal segments in both training and testing sets.	102
Figure A.3	Vanderbuilt University Medical Center (VUMC) Dataset in which both the interictal and baseline data comprise at least 25%.	103
Figure A.4	CHB-MIT Dataset for subjects in which both the interictal and baseline data comprise at least 25%.	104
Figure A.5	Vanderbuilt University Medical Center (VUMC) Dataset for all subjects.	105
Figure A.6	CHB-MIT Dataset for all subjects.	106

List of Figures

Figure A.7	Sample Robustness: Distances between the training and test distributions are computed for varying sized subsets of the compared distributions over 10 repetitions. Example for a randomly selected subject in the VUMC dataset. Patterns are consistent across subjects in CHB-MIT and VUMC.	108
Figure A.8	Optimal Transport Drift Detection: Random Split for subjects in the VUMC dataset. 10,000 reference samples, 500-sampled test chunks.	109
Figure A.8	Optimal Transport Drift Detection: Random Split for subjects in the VUMC dataset. 10,000 reference samples, 500-sampled test chunks.	110
Figure A.9	Optimal Transport Drift Detection: Temporal Split for subjects in the VUMC dataset. 10,000 reference samples, 500-sampled test chunks.	111
Figure A.9	Optimal Transport Drift Detection: Temporal Split for subjects in the VUMC dataset. 10,000 reference samples, 500-sampled test chunks.	112
Figure A.10	Optimal Transport Drift Detection: Random Split for subjects in the CHBMIT dataset. 10,000 reference samples, 500-sampled test chunks.	113
Figure A.10	Optimal Transport Drift Detection: Random Split for subjects in the CHBMIT dataset. 10,000 reference samples, 500-sampled test chunks.	114
Figure A.11	Optimal Transport Drift Detection: Temporal Split for subjects in the CHBMIT dataset. 10,000 reference samples, 500-sampled test chunks.	115
Figure A.11	Optimal Transport Drift Detection: Temporal Split for subjects in the CHBMIT dataset. 10,000 reference samples, 500-sampled test chunks.	116

List of Source Examples

2.1	Kitchen Module: Train many networks in parallel over many hyperparameters and subjects.	16
2.2	Toast’s JSON-like description for defining model architectures	21
2.3	Instantates the network with the correct dimensions and performs a dry-run test.	22
2.4	Output of	22
2.5	Example PyTorch model instance constructed by the model definition.	23
2.6	Dataset Bread: Generate K-Folds for cross-validation.	29
2.7	a Butterworth low-pass filtering function is imported and used to filter out frequencies above 50 Hz.	32
2.8	Moving window functions for tracking dominant and mean frequencies.	32
2.9	Creates decreasing-frequency-spectrum pink noise	34
2.10	Critic Plot: Plot subject training, validation, and test data and the resulting class balance for each split.	37
2.11	Machine Parameters: worker and node restrictions	43
2.12	Network Params: The TiSc Net architecture.	45
2.13	Additional Metrics: Easily choose which metrics to track throughout the training and evaluation stages.	45
2.14	Dataset Params: Selects a processed version of a dataset, applying any extra dataset splits or folds for cross-validation.	46
2.15	Runtime Settings: Notification and checkpointing settings.	46

Chapter 1

Introduction and Motivating Applications for Machine Learning In Seizure Prediction

Epilepsy is a chronic neurological disorder of the brain that affects nearly 50 million people worldwide and roughly 1.2% of the US population [1]. Spontaneous seizure recurrence manifests in various forms and presents a range of challenges that can be severely limiting [2], stigmatizing (e.g. involuntary movement and control over bowel or bladder function) [1], socially disruptive (e.g. loss of awareness or consciousness) [3, 4], and even life-threatening as in cases of Sudden Unexpected Death in Epilepsy [5, 6]. Moreover, those with epilepsy face heightened risks of physical injuries (e.g. vehicle accidents, fractures, and head injuries) related to seizures [1, 7]. These risk factors are further compounded by increased rates of psychiatric conditions such as anxiety and depression, often left untreated [8–10]. Similarly, the risk of premature death in people with epilepsy is up to three times higher than in the general population [1]. In 2016, epilepsy accounted for 0.5% of the total disease burden, a measure of time lost due to premature death or residual disabilities [11, 12]. Many of these deaths, including falls, drowning, and prolonged seizures, are potentially preventable. A 2016 survey by the Epilepsy Foundation highlighted unpredictability as a significant concern, irrespective of seizure frequency or severity [13]. Consequently, there is considerable interest in seizure prediction as a means to enhance epilepsy management and improve the quality of life for individuals with this condition.

Epilepsy treatment primarily relies on medication, with surgery being a secondary

option. Yet, anti-epileptic drugs (AEDs) can exacerbate mental health issues, are not always effective, failing to control seizures in approximately 25% of cases [14], and surgery isn't always feasible [15]. Predicting seizures based on electrical patterns and intervening with methods such as "vagus nerve stimulation" (VNS), could offer non-pharmacological alternatives to reduce seizure frequency in drug-resistant epilepsy (DRE), also known as refractory epilepsy [16–19]. VNS was first approved as an adjunctive therapy by the FDA in 1997 and the first implantable brain-responsive neurostimulator, the RNS[®] System, was later approved in 2013 [16]. These closed-loop systems rely on identifying a pre-ictal state and subsequently applying targeted neurostimulation to avert an impending seizure.

Therefore, there is a pressing need to develop systems capable of identifying pre-ictal states for quality-of-life improvements and responsive neurostimulation efficacy. In the RNS[®] System, detection parameters are set by the physician using trial-and-error approaches with tools such as measuring abrupt changes in specific spectral power bands. Nevertheless, patients typically encounter 600-2000 detections and subsequent stimulations daily, on average [16]. Hence, solutions with better sensitivity and specificity are required. Recently, a bevy of machine learning approaches for seizure prediction have been proposed [20–25]. These algorithms attempt to tease out the relevant neurophysiologic biomarkers for seizure prediction by learning them over large amounts of data, typically EEG signals, though measures based on movement and cardiac functions have also been used [26, 27].

Advancements in machine learning continue to accelerate and improve seizure prediction tasks, however, the application remains comparatively challenging to other

classification tasks for a variety of reasons: 1) As with other healthcare applications, the availability of data including both pre-ictal and ictal states is difficult to obtain due patient privacy, clinical limitations, and the lack of standardized recording formats [28–30]. 2) The electrical patterns associated with seizures tend to differ among patients. Hence, to have any value, machine learning algorithms need long-duration recordings consisting of many seizures for each patient [2, 27, 31–33]. 3) Pre-ictal and ictal patterns have been known to vary within patients over circadian and even slower timescales [34, 35]. 4) EEG data is frequently accompanied by significant sources of noise and artifacts (e.g. motion). 5) The duration of the pre-ictal state is not consistently defined across literature and is likely to differ amongst individuals and species with epilepsy [30]. Rat models, for instance, tend to have seizures much more frequently than humans. Our work aims to take a step towards solving and investigating some of these challenges.

1.1 Background and preliminaries

1.1.1 Seizure Types

Seizures are divided into two major categories depending on where they originate in the brain: generalized, focal, and unknown [3, 36, 37].

Focal seizures Focal seizures are characterized by their origination in a localized brain region within a singular cerebral hemisphere but may spread from there and later generalize [38]. These seizures manifest in either motor or non-motor onset categories, further classified based on levels of consciousness: aware, impaired, or

unknown awareness [37–39].

Motor-onset focal seizures encompass a spectrum of movements including automatism, atonic, clonic, epileptic spasms, hyperkinetic, myoclonic, or tonic activities [38]. *Clonic movements* are typified by recurrent, brief contractions across various muscle groups, akin to twitching or rhythmic jerking, whereas *atonic movements* denote a loss of limb tone. *Myoclonic episodes* present as irregular, non-rhythmic jerks in limbs, contrasting with *tonic movements* characterized by sustained contractions and limb posturing. *Automatisms* are non-purposeful, involuntary functions such as lip smacking, blinking, or grunting. *Epileptic spasms* entail repetitive waist flexion and arm movements [31, 38].

Conversely, non-motor focal seizures primarily exhibit autonomic, psychic, or sensory symptoms [38]. *Autonomic seizures* involve parts of the brain responsible for involuntary functions. These seizures may cause changes in blood pressure, heart rhythm, or bowel and bladder function [40]. *Psychic seizures* can evoke emotions or recollections, eliciting sensations of fear, anxiety, or déjà vu. *Sensory seizures* manifest as alterations in perception, such as auditory disturbances, hallucinations, or olfactory distortions [38].

Focal aware seizure (FAS), previously known as simple focal seizure and often referred to as an aura, occurs without loss of consciousness [39]. In the case of focal impaired awareness seizures (FIAS), formally referred to as complex focal seizures, the patient may stare blankly into space, fully or partially lose awareness, and experience automatisms [41, 42]. These seizures frequently begin with an aura and spread to larger

brain regions or possibly to both sides of the brain forming *focal to bilateral tonic-clonic seizures* [31, 37]. Some individuals may experience such seizures without conscious awareness or memory, posing a significant risk due to involuntary movements [38, 42].

Generalized seizures Generalized seizures begin in bilateral networks or as focal seizures that later generalize. The most common subtypes include absence, generalized tonic-clonic (GTC), myoclonic, and atonic seizures [3]. *Absence* seizures are characterized by sudden unresponsiveness to external stimuli, often without convulsive movements. EEG recordings frequently show generalized epileptiform discharges during this time [37]. In contrast, *GTC* (formally grand mal seizures) seizures involve bilateral, symmetrical convulsions affecting all limbs, accompanied by loss of consciousness [37, 38]. Myoclonic seizures are not associated with any obvious disturbance of consciousness. They are frequently presented as upward jerks of the arm corresponding to generalized spike-and-wave activity (3Hz) [37]. *Atonic seizures* tend to cause a patient to fall forward whereas, generalized tonic-clonic seizures generally cause a patient to fall backward [37].

1.1.2 Focal seizure generation and propagation

The epileptogenic zone (EZ) denotes the theoretical region within the brain responsible for the initiation of focal seizure activity and if removed is presumed to result in seizure freedom [43]. However, instances exist where complete lesion resection fails to suppress future focal seizure activity inpatients [44], suggesting that seizures manifest as networks of neuronal activity [45–47].

Seizure propagation thus occurs when there is sufficient activation to recruit

surrounding neurons [43]. This hypersynchrony may lead to a loss of surrounding inhibition and a spread of seizure activity to nearby areas. Distant areas may also be recruited through long association pathways normally suppressed by inhibitory neurons [48, 49]. This balance between recruitment and inhibition ultimately determines to what extent the seizure may or may not spread [2, 49].

1.1.3 Electrical Signatures of Seizures

EEG waveforms are often analyzed based on their frequency content, commonly grouped into four primary bands: delta (0–3+ Hz), theta (4–7+ Hz), alpha (8–13+ Hz), and beta (>14 Hz) [50]. Seizures typically follow complex, non-random abnormalities such as spikes, sharp waves, and spike-wave complexes, which can be captured by intracranial or scalp EEG recordings. Often, seizure onset is marked by high-frequency bursts of action potentials and the hypersynchronization of neuronal sub-networks [48]. These EEG patterns may manifest across some or all recording channels for focal and generalized seizures, respectively. While certain patterns may co-occur between patients, seizure initiation and propagation tend to vary between individuals, resulting in largely patient-specific electrical patterns [2].

1.1.4 Temporal Non-Stationarity

Many investigations into brain functional connectivity operate under the assumption of temporal stationarity. However, brain networks undergo nearly constant reorganization in response to various stimuli [2, 51–54]. These temporal fluctuations in intra- and inter-network connectivity might correlate with heightened initiation and severity of seizures during specific cycles [55–57]. For instance, Leguia, Andrzejak, Rummel, Fan,

Mirro, Tcheng, Rao & Baud [34], in a retrospective study, observed a high prevalence of circadian and multidien seizure cycles. However, these cycles can vary significantly between patients. Therefore, an optimal patient EEG recording should span several months and encompass segments in which the patient is awake and asleep, as epileptic activity exhibits variation across these time scales [3].

1.1.5 Pre-ictal classification: One size does not fit all

Predicting seizures boils down to distinguishing between pre-ictal and baseline EEG segments, yet challenges outlined in preceding sections complicate this task. An algorithm effective for one individual may not yield reliable results for another, given the numerous personalized factors influencing how indicative features manifest [2].

As a result, the most successful outcomes for a group of subjects have been attained through the use of patient-specific networks trained on an individual basis. However, these solutions demand substantial data, underscoring the critical need for algorithms with faster turnaround times. Improved accessibility to standardized data formats and training/evaluation pipelines will likely help this effort.

1.2 Overview of this Thesis

In **Chapter 2** we present Toast, a novel Python framework designed to streamline the development, training, and rigorous evaluation of neural networks over extensive patient-specific datasets, keeping in line with the need for individualized treatments. Our primary aim is to cater to the needs of the ML researcher, offering a rich set of automated functionalities for experiment management, network construction,

optimized data handling for complex time series data, and comprehensive evaluations.

While there are several existing libraries built upon PyTorch backends, such as Keras and PyTorch-Lightning, Toast is hyperfocused on research and rapid prototyping. Notably, Toast stands out from these toolkits in its unique approach to conducting repeatable experiments, each defined by a concise JSON-like description. This framework facilitates effortless swapping of architectures and hyperparameter adjustments while ensuring consistent comparisons with identical training procedures. In concert with Toast is Kitchen, a sub-module designed for the parallelized training of numerous smaller networks tailored to patient-specific tasks across various hardware accelerators. These features converge to form a cohesive ecosystem, offering experiment summary statistics, nearby signal processing functions to interact with data and robust model checkpointing capabilities. Furthermore, we showcase the development of the Time Scale Network (TiSc Net), conceived in tandem with Toast, exemplifying the efficacy of our framework in practice.

In **Chapter 3**, we further define the seizure prediction task and delve into the utilization of machine learning (ML) as a crucial component in predicting and managing impending seizures, an important use case that we continue to refer back to throughout this thesis.

Advancements in machine learning have accelerated its integration into practical applications across diverse tasks. However, recent work has unveiled a prevalent issue: these models tend to exhibit excessive confidence in their probability estimates, often resulting in erroneous predictions. This phenomenon becomes particularly

problematic when deployed in real-world scenarios where the data distribution may evolve, leading to unexpected model behaviors. Recognizing the limitations of machine learning systems is crucial for managing potential risks, especially in critical domains like healthcare. Moreover, a careful understanding of what ML models are capable of answering and not can lead to more balanced decision-making processes between automated systems and human experts. We delve into the impact of out-of-distribution data on the performance of deployed machine learning systems. Specifically, we focus on EEG data from epilepsy patients, aiming to enhance the safety and efficacy of machine-learning solutions in this field. We begin by challenging the conventional approach of randomized data splitting, highlighting its inflated results that fail to mirror real-world conditions. To address this, we propose two alternative strategies: a time-based split and a semi-divided split. These alternatives better simulate data streaming scenarios, allowing for a more comprehensive evaluation of seizure prediction algorithms against temporal shifts. Furthermore, we explore a range of model-agnostic techniques to monitor, visualize, and quantify distributional shifts in the data. This comprehensive approach aims to provide insights into the robustness and generalization capabilities of machine learning models in dynamic environments.

Lastly, **Chapter 4** concludes this thesis. We summarize and discuss the contributions of the preceding chapters and propose a set of directions for future work.

1.3 Publications and code

The Toast framework is available at

- <https://github.com/CenterforImplantableDevices/toast>

Code for reproducing the Time Scale Network (TiSc) architecture along with code to replicate Seizure Prediction results on the CHB-MIT dataset can be found here:

- <https://github.com/CenterforImplantableDevices/TimeScaleNetwork>
- https://github.com/CenterforImplantableDevices/TiScNet_chbmit

Meyer, T., Shultz, C., Dehak, N., Moro-Velazquez, L. & Irazoqui, P. *Time Scale Network: A Shallow Neural Network For Time Series Data* 2023. arXiv: [2311.06170](https://arxiv.org/abs/2311.06170) [cs]

Chapter 4 is adapted from the following work Shultz, C., Meyer, T. & Irazoqui, P. Randomized Data Splitting Leads to Inflated Performance for Seizure Forecasting Algorithms, 9 (2024)

Chapter 2

Toast: A PyTorch package for repeatable deep learning experiments

Portions of this chapter are reorganized from the original **Toast** documentation. Meyer, T. & Shultz, C. *Toast* Center for Implantable Devices. 2022

Section 2.3 of this chapter is adapted from the following journal article: Meyer, T., Shultz, C., Dehak, N., Moro-Velazquez, L. & Irazoqui, P. *Time Scale Network: A Shallow Neural Network For Time Series Data* 2023. arXiv: [2311.06170](https://arxiv.org/abs/2311.06170) [cs]

The Toast framework is available at <https://github.com/CenterforImplantableDevices/toast>

2.1 Introduction and Motivation

Here we introduce **Toast**, an open-source Python package designed to significantly simplify the process of conducting large, reproducible, and scalable machine learning experiments. **Toast** leverages the power of PyTorch, a widely-used machine learning framework, to provide researchers and practitioners with a comprehensive toolkit for developing and evaluating machine learning models.

2.1.1 Requirements

Experiment Tracking: **Toast** aims to offer robust experiment tracking capabilities. By logging comprehensive details of experiments, including hyperparameters, model configurations, and training metrics, one can ensure reproducibility even when the model architecture or dataset changes. This simplifies result comparison across

different experiments.

Parallelized Implementation: With the increasing complexity of machine learning models and datasets, parallelized implementation accelerates the training process, reducing the time required for experimentation. **Toast** is designed to train many smaller patient-specific workloads over a large set of hyperparameters and datasets.

Researcher First Approach: **Toast** adopts a researcher-first approach, offering modularity and tools that make swapping in and out different model architectures, hyperparameters, or datasets effortless—all while maintaining a consistent training loop for comparability. This approach streamlines experimentation, minimizes boilerplate code, eliminates redundancy, and offers easy endpoints for architecture development.

The remainder of this chapter will focus primarily on the core features and practical applications of **Toast**. We start by briefly outlining each submodule and the new changes introduced in **Toast 1.5**. We then continue with Section 2.3 demonstrating the basic usage through an example with the Timescale Network as groundwork for the next chapter. Finally, we conclude with Section 2.4 and quickly discuss upcoming changes and future work on the **Toast** library.

Toast is hosted and developed at <https://github.com/CenterforImplantableDevices/toast>. Full documentation, descriptions of submodules, and function definitions are available at <https://cid.wse.jhu.edu/toast-docs/>. The version of **Toast** used in this Thesis and currently being developed is **Version 1.5**. Some examples referenced are hosted in additional repositories: https://github.com/CenterforImplantableDevices/TiScNet_mitbih, https://github.com/CenterforImplantableDevices/TiScNet_chbmit,

enabling readers quick access to re-run and experiment with **Toast** on their own machines.

2.2 Toast design model

We now give a high-level overview of the structure and capabilities of the package.

Figure 2.1 visualizes the dependency graph of the **Toast** package and its submodules.

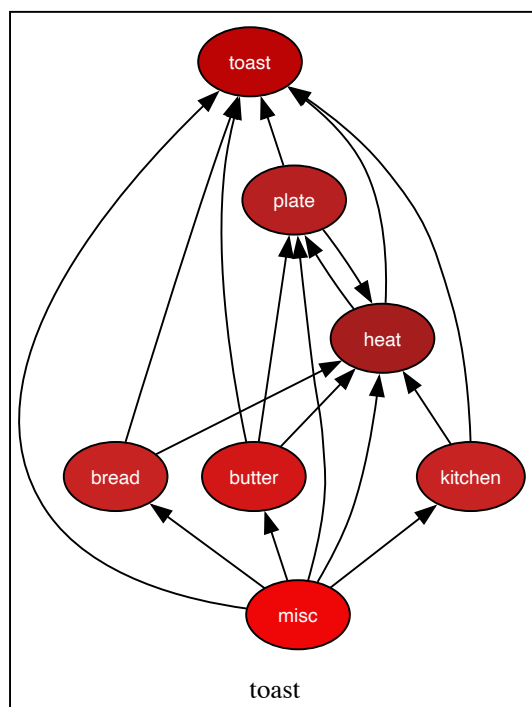


Figure 2.1: Toast Package

- **Kitchen:** Highly-optimized functions for parallelized training across multiple accelerators.
- **Heat:** A submodule for the design, implementation, and tracking of deep neural

networks.

- **Bread**: Functions for managing datasets and applying transformations.
- **Butter**: Signal processing functions for cleaning, exploring, and processing data in the time and frequency domain.
- **Plate**: Utilities for tracking, checkpointing artifacts, and comparing model performance over many runs.

2.2.1 Kitchen: Parallelized Training

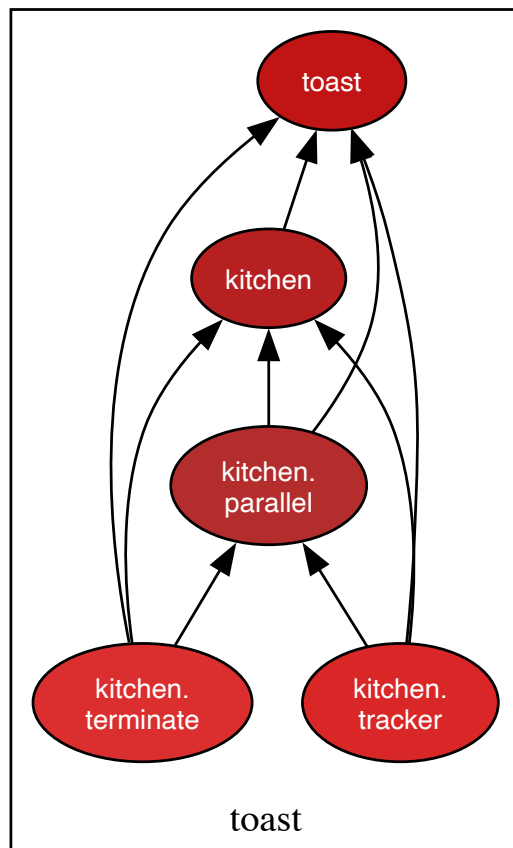


Figure 2.2: Toast Kitchen submodule

Unlike other parallelism techniques such as data parallelism (the model is replicated across each GPU worker and each processes a different set of samples simultaneously), pipeline parallelism (model layers are split across GPUs), or tensor parallelism (single layers are split across GPUs), our workloads often face a different need—training many smaller patient-specific networks in parallel. The Kitchen submodule is at the core of **Toast’s** management of multi-gpu multi-process training.

Scalability: The kitchen submodule enables the training of many small networks concurrently, which is crucial for scaling up machine learning tasks. Kitchen distributes the workload across available resources, dynamically calculating current system limits, to maximize throughput (Figures 2.3, 2.4, 2.5). This scalability is crucial when dealing with large datasets containing many patients.

Performance Optimization: Unlike data parallelism, which often falls short in terms of performance due to the limitations imposed by Python’s Global Interpreter Lock (GIL), kitchen implements multi-process parallelism. This allows **toast** to sidestep GIL contention and access the full computational power of the underlying hardware. Model information is passed to workers only at initialization and once model training is finished to reduce process initialization overhead. As a result, training can be executed with optimal efficiency in parallel.

Summary Statistics: In addition to resource allocation, kitchen further manages data passing between processes and aggregating summary statistics between patient models in only a few lines (Example 2.1).

```
fcn_args, test_mem_args = train_params.get_function_arguments(
    ↪ manipulated_parallel, folds_parallel, network_parameters, dataset,
    ↪ ...)

results = run_parallel( train_params.train_parallel, fcn_args,
    ↪ test_mem_args, **machine_params, ...)
```

Example 2.1: Kitchen Module: Train many networks in parallel over many hyperparameters and subjects.

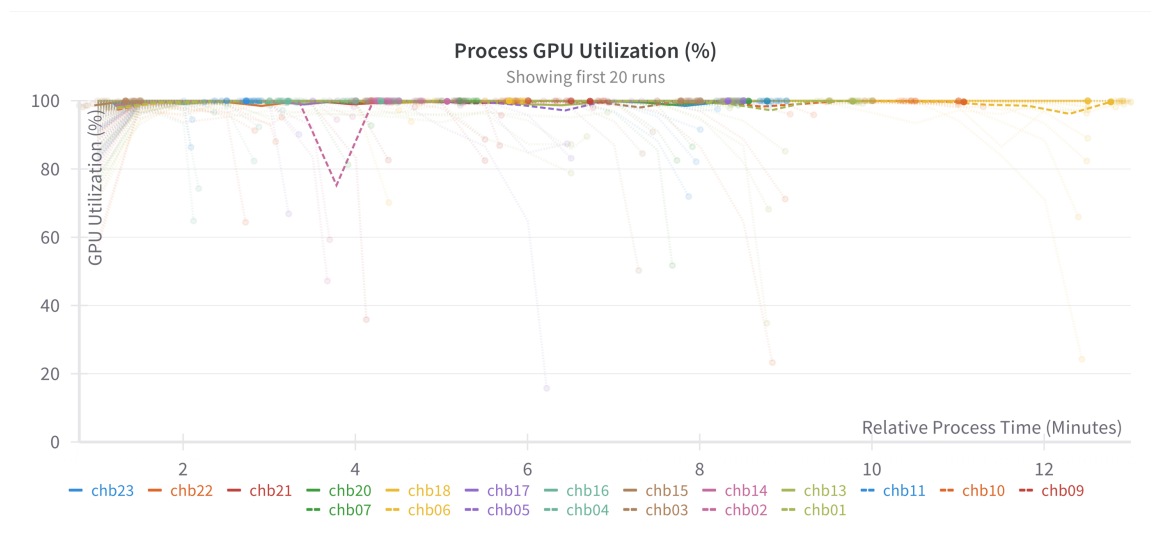


Figure 2.3: GPU utilization while training 800 individual networks: 10-fold cross-validation over 4 sets of hyperparameters from 20 patients in the CHB-MIT dataset.

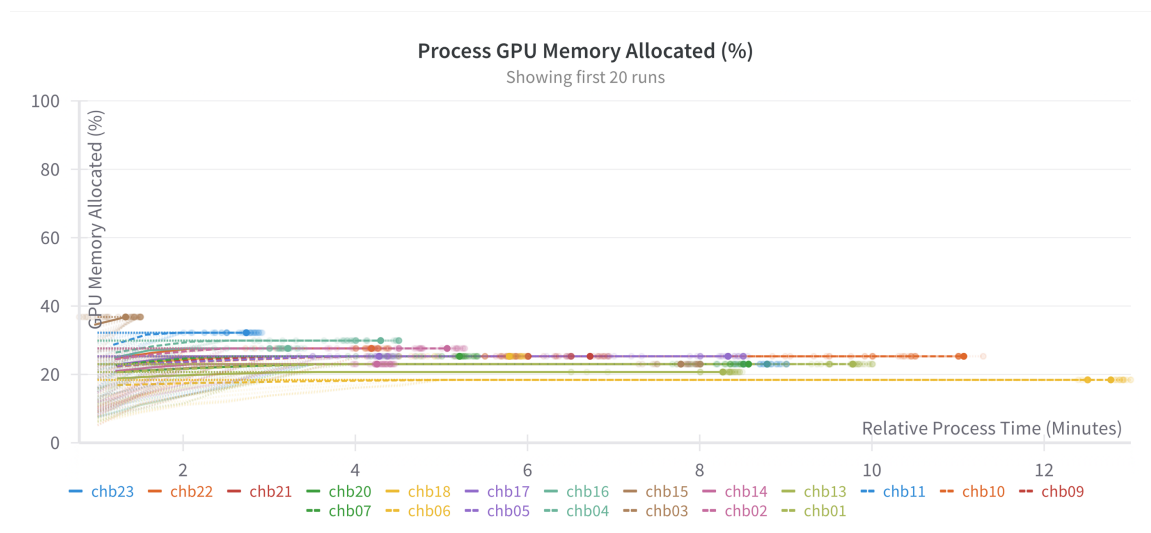


Figure 2.4: GPU memory allocation while training 800 individual networks: 10-Fold cross-validation over 4 sets of hyperparameters from 20 patients in the CHB-MIT dataset.

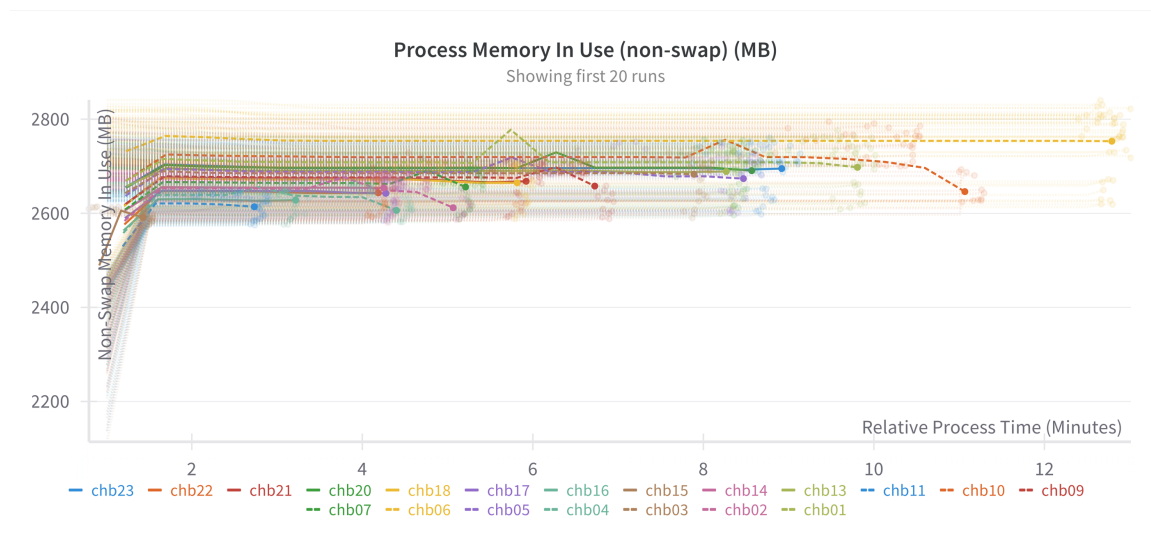


Figure 2.5: System RAM (non-swap) allocation while training 800 individual networks: 10-Fold cross-validation over 4 sets of hyperparameters from 20 patients in the CHB-MIT dataset.

2.2.2 Heat: Deep Neural Networks

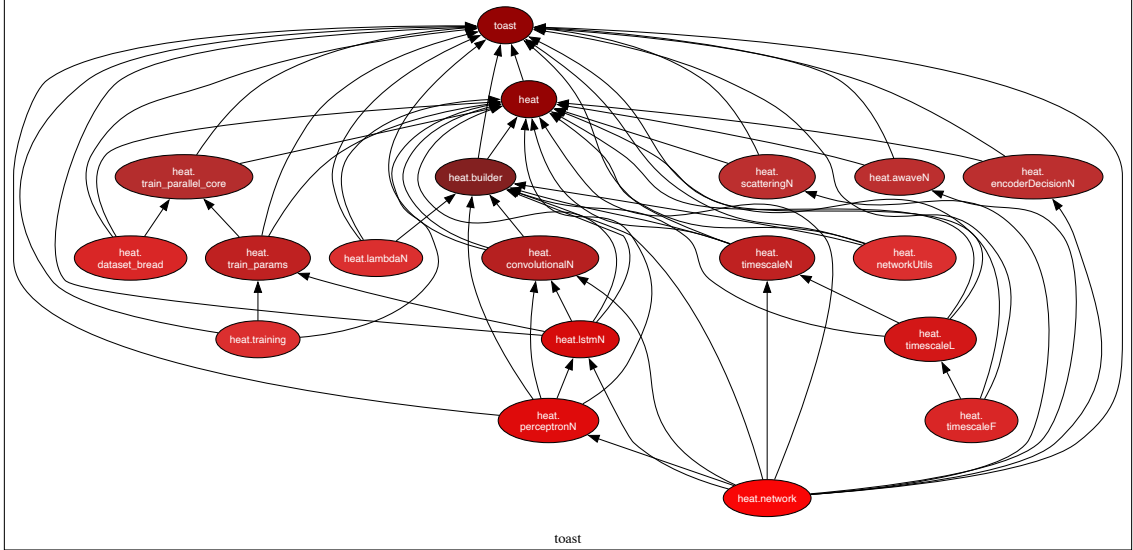


Figure 2.6: Toast Heat submodule

The base class for all neural network layers defined in the **Toast** package is the class `toast.heat.Network`. Objects of this class contain all the functionality of PyTorch’s `nn.Module` class with additional tracking features to easily train and debug deep neural networks. As a result, any native PyTorch model can also be used with the **Toast** framework. Previous versions of the package redefined the most common neural network architectures (e.g. Linear Layers, Long short-term memory (LSTM) networks, recurrent neural network (RNN), and Convolutional Neural Networks (CNNs)) as individual classes extending the **Network** class. These convenience classes helped to easily initialize networks that followed a certain predefined organization. For example, an `LstmN` instance might include some permutation of LSTM (`rnn`) layers as the first layers in a network, typically used as a “feature extraction” section, followed by fully connected layers typically used as a “classifier”

section. This limited the number of possible model configurations that could be defined. The primary benefit of using **Toast**'s convenience classes was that many of the network's layer descriptions and dimensions could be left blank or filled with a placeholder to be calculated on model instantiation, reminiscent of many PyTorch operations such as `torch.reshape(a_3D_batched_tensor, (-1,256,256,3))`. Since then, **Version 1.5** has refactored much of the codebase to organize itself around the `toast.hear.BuilderN` class to support a larger variety of architectures. `BuilderN` groups much of the same functionality that the original convenience classes introduced but now allows for any permutation of layers to be defined. As before, using any of the architectures defined in **Toast**, `BuilderN` can infer the operations needed (e.g. reshaping, flattening, and/or rotations) to interface between them and automatically insert them into the defined model as needed.

For instance, as a simple example, we define a deep CNN followed by a fully connected layer with an output size of 1 in Example 2.2. We've left the input shape as **None** as well as the input dimension of the last linear layer:

```
network_params = {  'input_shape': None, #[1, 23, 1280],
  'layer_desc': [(['cnn', [(['conv', { 'kernel': [2,3], 'stride': 1,
    ↪ 'number_repitions': 32, 'batch_norm': True}),
      ('max', { 'kernel': [2,2], 'stride': 2,
    ↪ 'dropout': 0.0}),
      ('conv', { 'kernel': [2,3], 'stride': 1,
    ↪ 'number_repitions': 1, 'batch_norm':
    ↪ True}),
      ('max', { 'kernel': [2,2], 'stride': 2,
    ↪ 'dropout': 0.0}),
      ('conv', { 'kernel': [2,3], 'stride': 1,
    ↪ 'number_repitions': 1, 'batch_norm':
    ↪ True}),
      ('max', { 'kernel': [2,2], 'stride': 2,
    ↪ 'dropout': 0.0}),
      ('conv', { 'kernel': [1,3], 'stride': 1,
    ↪ 'number_repitions': 1, 'batch_norm':
    ↪ True})])],
    ('mlp', [(['linear', { 'layer_size': [None, 1]})])
  ]
}
```

Example 2.2: Toast’s JSON-like description for defining model architectures. Here a CNN, followed by a fully connected layer to a scalar output is shown. The input shape and dimensions between layers are automatically inferred when the model is instantiated.

With the input shown in 2.2, **Toast** automatically infers the placeholder dimensions and instantiates the model shown in Example 2.5 with only a few lines (Example 2.3). The model is then tested with a dry-run to ensure the forward and backward passes operate correctly, as shown in Example 2.4

```
network, network_parameters = train_params.get_initialized_network(
    ↪ dataset, network_params, path_log_file,
    ↪ pretrain_outputs=pretrain_info)
```

Example 2.3: Instantates the network with the correct dimensions and performs a dry-run test.

```
Network has 20929 parameters.
Test Sample: tensor([[[[ -7., -10., -9., ..., -5., -8., -9.],
                        [ 12., 11., 15., ..., -43., -48., -48.],
                        [ 42., 25., 10., ..., -22., -28., -29.],
                        ...,
                        [ 16., 19., 28., ..., 4., 5., 2.],
                        [-17., -7., 10., ..., -25., -24., -22.],
                        [ -3., -10., -16., ..., -47., -48., -48.]]]]],
                    size: torch.Size([1, 1, 16, 1280])
Test Label:  tensor([[1.]]), size: torch.Size([1, 1])
Sample Type:  <class 'torch.Tensor'>
Label Type:   <class 'torch.Tensor'>
Output: tensor([[0.5850]], grad_fn=<SigmoidBackward0>),
        size: torch.Size([1, 1])
Forward Time: 0.048621177673339844
Output Type:  <class 'torch.Tensor'>
Output Data:  tensor([[0.5850]])
Backward Time: 0.00782322883605957
```

Example 2.4: Output of

The instantiated networks' forward and backward layers are further verified via a quick dry-run as shown in Example 2.4

```

BuilderN(
  (network): Sequential(
    (0): ConvolutionalN(
      (feature_extract_layers): Sequential(
        (0): Conv2d(1, 32, kernel_size=(2, 3), stride=(1, 1))
        (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
          ↪ track_running_stats=True)
        (2): ReLU()
        (3): MaxPool2d(kernel_size=[2, 2], stride=[2, 2], padding=0,
          ↪ dilation=1, ceil_mode=False)
        (4): Conv2d(32, 32, kernel_size=(2, 3), stride=(1, 1))
        (5): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
          ↪ track_running_stats=True)
        (6): ReLU()
        (7): MaxPool2d(kernel_size=[2, 2], stride=[2, 2], padding=0,
          ↪ dilation=1, ceil_mode=False)
        (8): Conv2d(32, 32, kernel_size=(2, 3), stride=(1, 1))
        (9): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
          ↪ track_running_stats=True)
        (10): ReLU()
        (11): MaxPool2d(kernel_size=[2, 2], stride=[2, 2], padding=0,
          ↪ dilation=1, ceil_mode=False)
        (12): Conv2d(32, 32, kernel_size=(1, 3), stride=(1, 1))
        (13): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
          ↪ track_running_stats=True)
        (14): ReLU()
      )
    )
    (1): LambdaN(desc: networkUtils.lambda_flatten)
    (2): PerceptronN(
      (layers): Sequential(
        (0): Sequential(
          (0): Linear(in_features=4992, out_features=1, bias=True)
          (1): Sigmoid()
        )
      )
    )
  )
)

```

Example 2.5: Example PyTorch model instance constructed by the model definition.

To enhance flexibility further, **Version 1.5** also introduced the **LambdaN** class, a lightweight layer facilitating the integration of arbitrary processing functions into the architecture stack. This feature is particularly useful for tasks such as slicing layer activations or reshaping data for novel preceding and succeeding layers.

There are numerous benefits to **Toast's** approach to model definitions that make it worth considering. First, model definitions are simplistic, easy to define, and quick to test. We've abstracted much of the complexity required to ensure the correct dimensions match between layer types. This allows for **Toast** to be accessible to researchers who may not be familiar with the structure of some architectures or are looking to prototype quickly. Second, by supporting placeholder values for the input and output dimensions of models, **Toast** easily supports rapid benchmarking of the same model on multiple data sources that might vary in structure or dimensions. Finally, **Toast's** JSON-like description of complete model architectures serves as a useful abstraction for future AutoML applications. This allows **Toast** to easily instantiate and iterate through a number of machine learning pipelines in parallel that try different algorithms, architectures, and parameters, allowing users to quickly identify the best model for their use case, construct an ensemble of models for improved predictive performance, or access powerful ML solutions without extensive programming knowledge.

Format **Toast** primarily interacts with data in a standardized `.wav` file format to optimize loading speeds and offer flexibility in defining sample labels for large time-series datasets. A variety of file types and structures were tested, with `.wav` performing the fastest in a majority of cases. See Table 2.1 and Table 2.2 for a brief overview.

File Type	Import Time (s)
.h5py	1.5200
.h5pyc	1.5417
.csv	7485.6096
.pkl	1279.2596
.wav	0.4887
.edf	326.6037
.mne	16627.8916

Table 2.1: Times to import 15 sec of data 1000 times.

File Type	Import Time (s)
.h5py	0.6668
.h5pyc	0.4770
.csv	259.1395
.pkl	1.3699
.wav	0.7231
.edf	17.6900
.mne	20.4247

Table 2.2: Times to import entire data file, replicating a typical machine learning pipeline.

In particular, **Toast** organizes raw data into three subfiles: labels, data, and metadata as shown in Table 2.3. Sample information is then stored in .pkl format as

Filename	Channel 0	Channel 1	Channel 2
some_file_identifier_label.wav	label (integer form)	X	X
some_file_identifier_data.wav	data_ch_0	data_ch_1	...
some_file_identifier_metadata.wav	POSIX Timestamp - seconds	POSIX Timestamp - nanoseconds	X

Table 2.3: Standardized .wav Format

a 2D list of dictionaries whose structure is outlined in Table 2.4. List index 0 contains a list of dictionaries with descriptions of data (samples) with a label “0”, list index 1 contains a list of dictionaries that describe data with a label “1”, etc. Isolating sample metadata in pickle files optimizes read speed and disk utilization, allowing **Toast** to rapidly access data shards/partitions and easily update segment descriptions without reformatting/processing entire datasets. This is particularly useful when one is interested in utilizing the same raw features for multiple tasks. For example, the MIT BIH V1.0.0 dataset [60] contains a large number of rhythm annotations (Atrial flutter, Ventricular bigeminy, Supraventricular tachyarrhythmia, etc.) that are not relevant to all prediction tasks. Rather than storing multiple pre-processed copies of the dataset or reprocessing the data each time a model is trained, we only store modified metadata.

Toast provides high-level functionality in `toast.bread.format` to format and save various datasets to the standardized format. `toast.bread.label` provides the functionality to easily identify or modify the sample information. Lastly, `toast.bread.load` offers a variety of methods for loading data from `csv`, `edf`, `hdf5`, `pkl`, and `wav` files.

dict_key	dict_value
filename	The name of the file where this data is from. Omit and "_data.wav" or "_metadata.wav" suffix.
label_int	The label that the data represents (in integer form)
start_index	The index (not time) that marks the start of the data in the file specified in "filename".
stop_index	The index (not time) that marks the end of the data in the file specified in "filename".

Table 2.4: Toast label format: contents of the dictionaries describing the data may vary, but must always contain the above fields. Other possible fields include: start_datetime, end_datetime, etc.

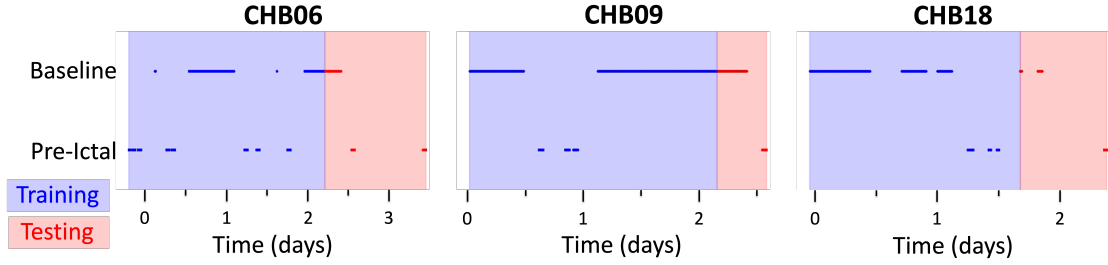


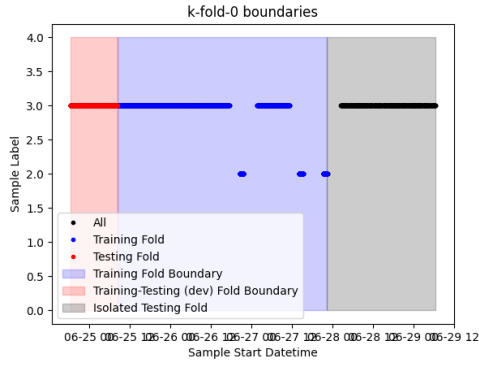
Figure 2.8: Toast.Bread: Temporal split: Training, Test, and Validation split for a subject in CHB-MIT.

2.2.3.2 `toast.heat.Dataset_Bread`

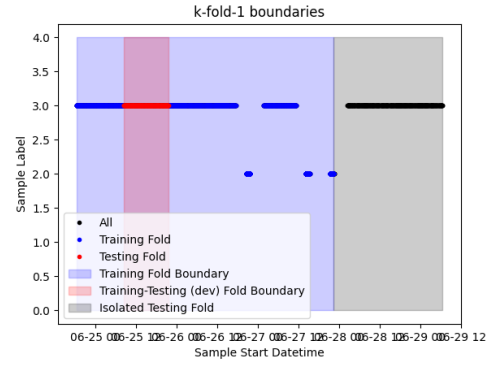
Within **Toast**, `toast.heat.Dataset_Bread` holds essential class definitions that dictate how data and datasets are managed. The `toast.heat.Dataset_Bread.Dataset_Bread` class, an extension of `torch.utils.data.Dataset`, is responsible for applying any specified transformations to the loaded data. Additionally, `toast.heat.Dataset_Bread.Data_Bread` oversees multiple instances of `Dataset_Bread`, serving purposes like training, validation, and testing. **Toast** simplifies dataset management, facilitating tasks such as generating folds for cross-validation (as illustrated in Example 2.6 and Figure 2.9) and organizing data into various splits, including random, divided, or temporal arrangements (as discussed in Chapter 3 and visualized in Figure 2.8).

```
dataset = dataset_bread.Data_Bread( raw_data_path_sub,
    ↪ segments_path_sub, needs_validation=False, **dataset_params)
dataset.generate_training_k_folds( num_folds, groupby=groupby,
    ↪ plot_foldGroup_path=os.path.join(path_output_subject, "summary",
    ↪ output_subfolder))
```

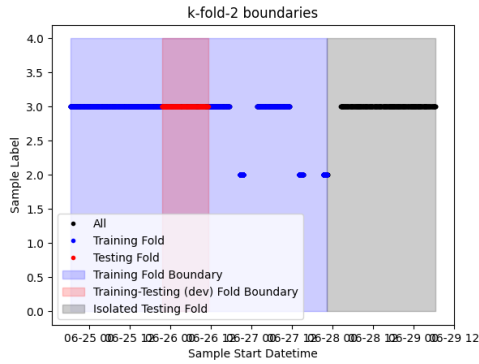
Example 2.6: Dataset Bread: Generate K-Folds for cross-validation.



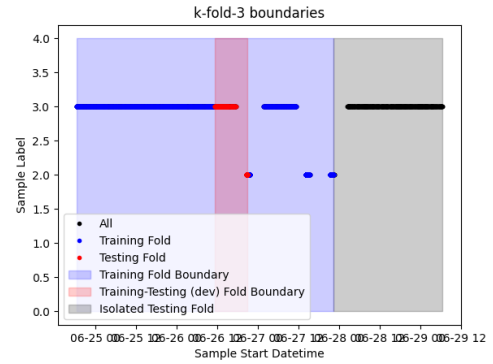
(a) Cross-Validation Group 0



(b) Cross-Validation Group 1



(c) Cross-Validation Group 2



(d) Cross-Validation Group 3

Figure 2.9: Generated Plots for groups 0-3 of 10-Fold cross-validation in a **temporal split**.

Filters: Any functionality that filters or cleans data in some way. This includes textbook filters like Butterworth filters (Example 2.7), averaging filters, or handcrafted filters based on specific phenomena

```
from toast.butter.filters import Butterworth
```

```
filtered_data = Butterworth.lowpass_filter( data_channels, sample_frequency, 50)
```

Example 2.7: a Butterworth low-pass filtering function is imported and used to filter out frequencies above 50 Hz.

Freq: Any functionality or operations that occur in the frequency domain. This includes Fourier-based techniques and operations that rely on Fourier-based techniques such as frequency tracking using STFTs (Example 2.8 and Figure 2.11).

```
import numpy as np
f_1 = 10/60 # Frequency, in cycles per second, or Hertz
f_2 = 100/60
f_3 = 250/60
fs = 1250 # Sampling rate, or number of measurements per second
t = np.arange(0, 60, 1/fs)
sig1 = np.sin(f_1 * 2 * np.pi * t) + np.sin(f_2 * 2 * np.pi * t) +
    ↪ np.sin(f_3 * 2 * np.pi * t)

f_1 = 15/60 # Frequency, in cycles per second, or Hertz
f_2 = 140/60
f_3 = 300/60
fs = 1250 # Sampling rate, or number of measurements per second
t = np.arange(0, 60, 1/fs)
sig2 = np.sin(f_1 * 2 * np.pi * t) + np.sin(f_2 * 2 * np.pi * t) +
    ↪ np.sin(f_3 * 2 * np.pi * t)

sig = np.concatenate((sig1, sig2))
HR_peak = track_peak_frequency( sig, fs, 40/60, 220/60) * 60
HR_mean = track_mean_frequency( sig, fs, 40/60, 220/60) * 60
```

Example 2.8: Moving window functions for tracking dominant and mean frequencies.

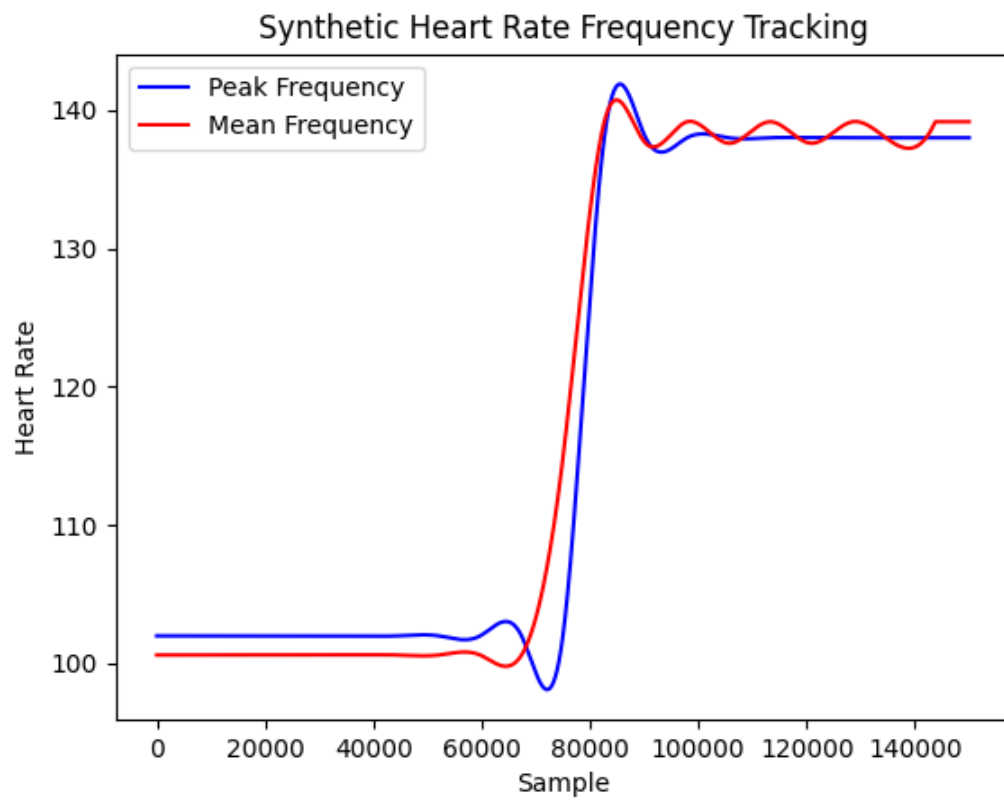


Figure 2.11: Output of example 2.8.

Noise: Functionality for generating spectrum-specified noise (e.g. *pink noise* in Example 2.9) and other transformations.

```
from toast.butter.freq import Fourier
from toast.butter.noise import get_color
```

```
time = np.arange( 10000)
noise = get_color( time.shape, 'pink')
```

Example 2.9: Creates decreasing-frequency-spectrum pink noise

Slice: Utilities for excluding sets of outliers, slicing across given data axes, and various moving window functions for tracking the moving average, standard deviation, variance, median, max, min, and root-mean-squared values along a specified axis.

2.2.5 Plate & Critic: Evaluation and Tracking

Lastly, in addition to standard performance metric tracking, **Toast** offers the ability to easily set hooks on model activations and weights to provide a more in-depth evaluation. This feature allows for a granular examination of model stability or how weight distributions converge during training (Figure 2.12). This level of transparency can be invaluable for understanding why certain predictions are made and for diagnosing architectural issues present in a model. Toast offers a similar level of detail when examining datasets, particularly in how data is split for training and testing, and the resulting class proportions as shown in Example 2.10 and Figure 2.13.

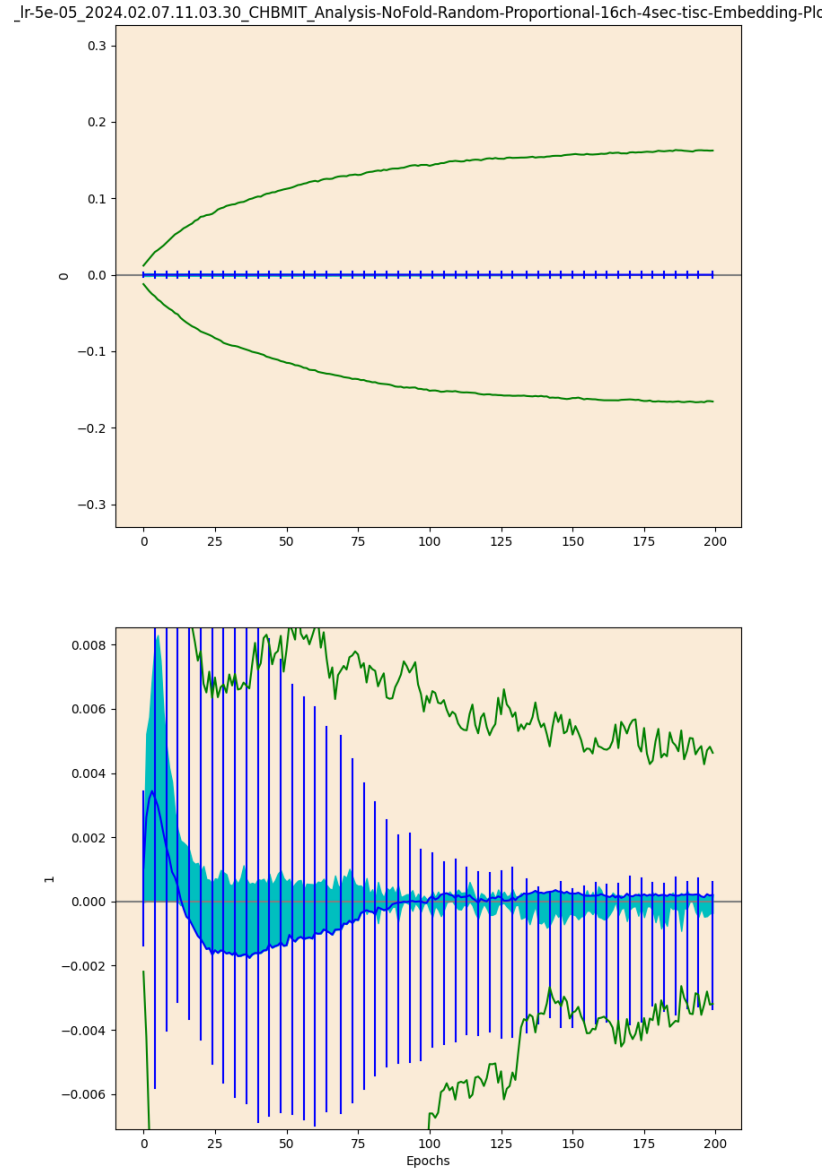
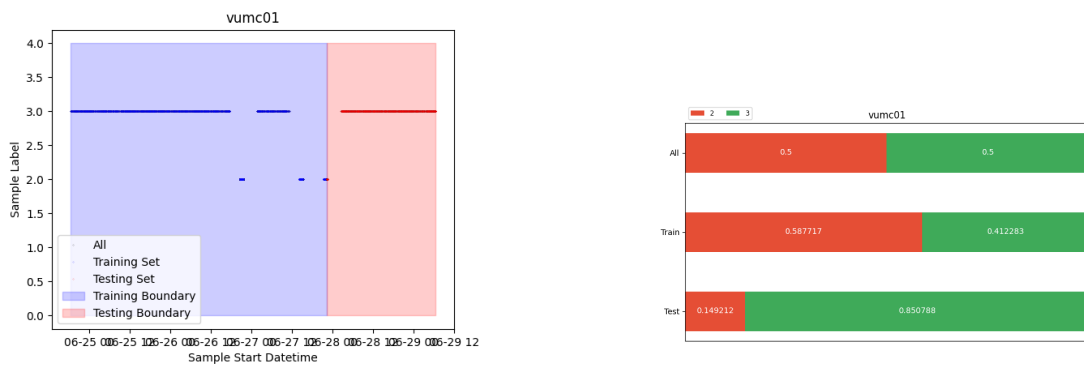


Figure 2.12: Automatic weight and activation tracking for during training.

```
critic_plot.plot_train_dev_test_splits(dataset,
    ↪ os.path.join(path_output_subject, "summary", output_subfolder),
    ↪ "TrainDevTestSplits_" + sub, sub)
critic_plot.plot_class_balance(dataset,
    ↪ output_path=os.path.join(path_output_subject, "summary",
    ↪ output_subfolder), filename="Plot_Class_Balance_" + sub, title=sub)
```

Example 2.10: Critic Plot: Plot subject training, validation, and test data and the resulting class balance for each split.



(a) Training and testing splits: subject vumc01 with a temporal split.

(b) Class Balance: Subject vumc01 with a temporal split.

Figure 2.13: Critic Plot: Plot subject training, validation, and test data and the resulting class balance for each split.

2.2.6 Weights & Biases Integration

In **Version 1.5**, **Toast** added the option to integrate with Weights & Biases (W&B) for tracking, versioning, and visualizing experiments. All of **Toast's** added evaluation functions can optionally be shared with WandB for quick management and experiment bookkeeping. Some of the key W&B features include:

Experiment Tracking: The integration of **Toast** with W&B provides the ability to quickly compare and visualize multiple machine learning experiments from a single web dashboard. Metrics for performance (See Figure 2.14 and Figure 2.15), losses, and weight distributions are presented in real-time during the training process. Furthermore, W&B can utilize **Toast's** experiment config files for easy checkpointing of model weights, hyperparameters, and model history for future reproducibility. In addition to performance metrics, W&B has the option to track live system-level performance for GPU utilization and resource allocation, offering a single point of contact that improves over **Toast's** original email alerts to identify bottlenecks and crashed runs (Figure 2.16).

Launch: W&B offers the ability to launch new jobs and ML workflows instead of through the command line. Various tasks can be initiated directly from the web dashboard. Due to the simple nature of **Toast's** JSON-like experiment descriptions, any job that can be launched from toast can be launched from W&B.

Sweeps: Building off of launch is **Sweeps**. Toast's integration with W&B facilitates hyperparameter optimization with Bayesian optimization, the Hyperband algorithm for efficient parameter tuning, and early stopping. Furthermore, W&B can visualize



Figure 2.14: W&B Dashboard: 10-Fold cross-validation accuracy across all subjects for each learning rate.



Figure 2.15: W&B Dashboard: 10-Fold cross-validation accuracy across all hyperparameters for each subject in CHB-MIT.

the impact factor and correlation of different hyperparameters on model metrics. A simple sweep over learning-rates are visualized in Figure 2.17.



Figure 2.16: W&B Dashboard: System resource tracking

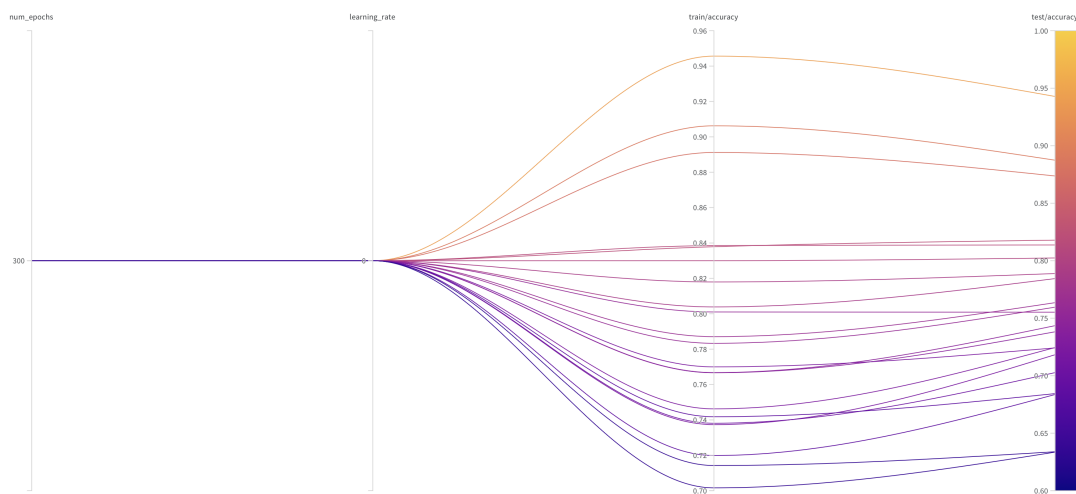


Figure 2.17: W&B Dashboard: A simple sweep across learning rates for each subject in CHB-MIT.

Model Registry: W&B can register and organize ML models, and track their lineage and the pipelines used for training them. This is particularly useful for tracking a hierarchy of pretraining and finetuning strategies for individual subjects.

In summary, the integration of Toast with W&B represents a significant step forward in machine learning experimentation. It provides researchers with a robust toolkit for conducting, tracking, and optimizing experiments while promoting ease of access and reproducibility of the process.

2.3 Time Scale Network: A shallow neural network for time series data

As a concrete example of experimentation with Toast, we briefly outline the Time Scale Network (TiSc Net) [58] that was designed in parallel with **Toast**, highlighting the ability to develop novel architectures within the **Toast** ecosystem.

TiSc Net is designed to extract input features at multiple timescales simultaneously, implementing strided convolution in a structure similar to the discrete wavelet transform (DWT). Unlike traditional wavelet transforms, however, TiSc Net employs learned waveform kernels via back-propagation, freeing them from numerical constraints and enabling independent optimization for each time scale.

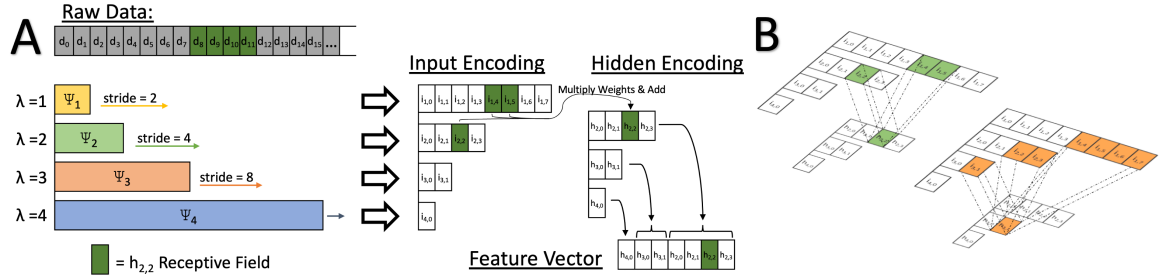


Figure 2.18: Time Scale Network Architecture.

In the input layer of TiSc Net, waveforms of exponentially increasing length are translated and multiplied across the data in non-overlapping windows. Each output generates a new activation, creating receptive fields equivalent to those found in the individual coefficients of a DWT. Unlike wavelet transforms, however, TiSc Net applies each scaled waveform to the input data in parallel rather than sequentially, reducing much of the complexity to a single matrix-vector operation and reshaping data.

This translation-dilation pattern persists within the embedding space, where TiSc hidden layers combine activations based on the same time-scaled receptive fields present in the raw features. The architecture permits the repetition or stacking of multiple hidden layers to create deeper or wider networks, as illustrated in Figure 2.18.

An advantage of TiSc Net lies in its computational complexity, which ranges between $O(n)$ and $O(n \log n)$ operations, depending on the input size. This complexity represents a significant improvement over convolution operations in spectrograms or Convolutional Neural Networks (CNNs), which typically range between $O(n \log n)$ and $O(n^2)$, influenced by input dimensions and the utilization of fast implementations [58].

For a more comprehensive discussion on the benefits and applications of the Time Scale Network, readers are referred to Meyer et al. [58].

2.3.1 Toast Experiments with the Time Scale Network

Here we share how TiSc Net can be configured using **Toast's** configuration settings.

```
machine_params = { 'use_gpu' : True,
    'max_job_gpu' : 16, #16,
    'max_job_sys' : -1,
    'max_mem_gpu' : 24e9,
    'percent_memory' : 1.0
}
```

Example 2.11: Machine Parameters: The maximum number of workers and jobs per GPU and compute node can be determined. Toast will allocate as many resources as possible within these constraints.

Toast uses the **BuilderN** class to instantiate all models once the network parame-

ters are defined. Example 2.12 demonstrates how this can be done for the TiSc Net architecture. As described in previous sections we set the input shape to `None` in Line 2 to allow toast to infer any needed dimensions. `layer_desc` describes the model architecture to be instantiated.

The first element sets the range of window sizes to translate over the input features (line 4): `[min_window, max_window]`. If set to `None` then the largest window size is assumed to be the length of the input. Additional hidden layers, defined as `[min_window, max_window, inclusive]` (line 5) can be appended to deepen the network. Moreover, Multiple TisC channels can be applied, similar to multi-headed attention and CNNs (line 8). Finally, a fully connected layer is added to a 2-dim output with 20% dropout (lines 10-12). Additional training details, including the base nonlinearity, optimizer and optimizer settings, training epochs, and optional tracking details are also defined here.

Desired metrics to track and log to W&B can also easily be set, as in Example 2.13. The desired dataset settings are set in the `dataset_params` (Example 2.14). `segment_file` and `data_files` select which version of the preprocessed dataset to use, including segment length and any transformations or class subsampling applied. `label_keys` can be used to re-assign label names. The number of folds for cross-validation can be set using `num_folds` or left at 1 for a single train, dev, test split. Lastly, training and evaluation dataset splits (described in detail in Chapter 3) can be set using the `data_split` option.

Further runtime settings can be set to control checkpointing and notification

```
1  # Network Info
2  network_params = {'input_shape': None,
3                    'layer_desc': [
4                        ('tisc', ('tisc', {'window_size': [[32, None],
5                                                         [2, None, False]],
6                                                         'length_input': 16384,
7                                                         'dropout': 0.2,
8                                                         'tisc_channels': 1
9                                                         })),
10                       ('mlp', [(('linear', { 'layer_size': [None, 2],
11                                             'dropout': [0.2],
12                                             'bias': [True]})])
13                    ],
14                    'nonlinearity': torch.nn.ReLU,
15                    'num_epochs': 200,
16                    'learning_rate': 5e-5,
17                    'optimizer': 'rmsprop',
18                    'weight_decay': 0.05,
19                    'momentum': 0,
20                    'loss_fn': torch.nn.BCELoss,
21                    'batch_size': 256,
22                    'track_weights': True,
23                    'track_activations': False,
24                    'activation_layers': [], # ['network.1'],
25                    'show_timings': False
26                }
```

Example 2.12: Network Params: The TiSc Net architecture.

```
metrics = {
    "f1": BinaryF1Score,
    "roc_auc": BinaryAUROC,
    "pr_auc": BinaryAUPRC
}
```

Example 2.13: Additional Metrics: Easily choose which metrics to track throughout the training and evaluation stages.

strategies (Example 2.15).

```
dataset_params = {'segment_file'   : ['4sec', 'pred-eq' ], #'pred-eq'
                  'data_files'     : '16ch', #'16ch',
                  'label_key'      : {2: 1, 3: 0},
                  'one_hot_encode' : True,
                  'num_folds'      : 1,
                  'fold_groupby'   : None,
                  'training_perc'  : 0.8,
                  'testing_perc'   : 0.2,
                  'data_split'     : "Random-Proportional"
                  }
```

Example 2.14: Dataset Params: Selects a processed version of a dataset, applying any extra dataset splits or folds for cross-validation.

```
# Runtime settings
runtime_settings = {
    'save_networks'      : True,
    'checkpoint'         : False,
    'save_indiv_net_plots' : 1,
    'skip_already_proc'  : False,
    'cache_results_file' : False,
    'sync_synology'      : False,
    'email_alerts'       : False,
    'email_address'      : "",
    'quiet'              : True,
}
```

Example 2.15: Runtime Settings: Notification and checkpointing settings.

2.4 Discussion and future work

Toast presents a significant advancement in simplifying the process of conducting large-scale machine-learning experiments. For the Cybernetics Implantable Devices Lab, toast has served as a foundational toolset with the necessary abstractions and scalability to efficiently develop, train, and deploy machine learning models. By abstracting away implementation details and offering high-level APIs, Toast accelerates the model development process while ensuring flexibility for customization and experimentation.

Moving forward, future work on the Toast Python library will likely focus on several key themes. First, there will be a continued emphasis on optimizing Toast’s parallelization techniques, with a potential expansion into model parallelism to accommodate larger models more effectively. There is also the opportunity here to integrate with other large distributed training frameworks such as Ray. Second, we plan to add TensorFlow as an additional backend to better leverage its superior options for serving and compressing models, particularly for deployment on low-power edge devices and microcontrollers. Third, we plan to develop additional evaluation tools, particularly those capable of tracking the distributional shift of data and monitoring model performance degradation over time. Finally, we hope to support all common architectures with the `BuilderN` class out of the box, streamlining the model development process and further increasing Toast’s versatility and usability for a variety of application areas.

Chapter 3

Seizure prediction and non-stationary machine learning

*Data is a precious thing and will last longer than the
systems themselves.*

– TIM BERNERS-LEE, INVENTOR OF THE WORLD WIDE
WEB

This chapter is adapted from the following work to be submitted

Shultz, C., Meyer, T. & Irazoqui, P. Randomized Data Splitting Leads to Inflated Performance for Seizure Forecasting Algorithms, 9 (2024).

3.1 Introduction

Seizure forecasting algorithms could play a vital role in managing epilepsy by providing timely warnings and enabling proactive interventions; however, algorithms have historically struggled when translated to unseen, real-time data and deployed implementations — miss-classifying data segments entirely or compensating heavily with many false positives [61].

Out-of-distribution data may be a contributing factor to the poor performance of machine learning systems on the edge. Conventionally, machine learning algorithms assume independence and identical distribution (i.i.d.) of data points, dividing them randomly across time into training, validation, and testing sets. However, this approach overlooks potential distributional shifts over time and across patients, which

are common in real-world applications, leading to significant performance degradation at inference time in practice. To monitor the performance of deployed models, it is customary to assess the model against the current target distribution and retrain it when necessary [62]. In scenarios where labeled data is unavailable or sparse such as in individualized healthcare applications (i.e. seizure prediction), however, quantifying performance degradation with an unsupervised model-agnostic metric becomes imperative.

In this Chapter we explore methods for detecting and characterizing distribution shifts in EEG data from epilepsy patients, aiming to overcome a critical hurdle in the safe and effective deployment of machine learning solutions to the field. First, we highlight that the most common data organization strategy in literature, randomized splitting, leads to significantly inflated results that do not translate to real-world scenarios. To address this, we propose two alternative strategies: (i) a time-based split, and (ii) a semi-divided split, which better mimic real-world data streaming conditions and allow for assessing the generalization and robustness of seizure prediction algorithms against temporal shifts. We then explore various model-agnostic approaches to monitor, visualize, and quantify the incurred distributional shift.

In contrast to synthetic datasets that induce distribution shifts through various transformations, we explore the natural and gradual shifts inherent within EEG data over time [63–67]. A deeper understanding of underlying distributional shifts may shed light on robustness and generalization problems, which are particularly important to seizure forecasting and other healthcare machine-learning applications. Subsequent sections of this Chapter present a systematic evaluation of the impact of different

data-splitting strategies on the performance of seizure forecasting algorithms.

3.2 Background And Related Work

Despite prior acknowledgment in literature [68], the impact of data splitting strategies on the performance of machine learning algorithms warrants further investigation, especially in highly correlated time-series data such as EEG. It is not uncommon to see and consequently expect near-perfect performance from state-of-the-art models. The abundance of machine learning literature reporting inflated performance metrics for healthcare applications with random data splitting strategies is well documented [20, 69, 70].

Distributional Shifts Under the typical independent and identically distributed (i.i.d.) assumption, we assume that the distribution of the test or target data, denoted as P_{test} , aligns with that of the training data, represented as P_{train} . However, a distribution shift occurs when this assumption is violated, leading to $P_{train} \neq P_{test}$. Such shifts can arise from alterations in the distribution of the data’s features, labels, or both [71, 72].

Covariate shift pertains to changes solely in the feature distribution $p(x)$ while maintaining the label distribution. Learning under covariate shifts, with the assumption that $p(x, y) = p(x)p(y|x)$, can be thought of as a form of causal learning, focused on capturing the underlying causal mechanism $p(y|x)$ rather than spurious correlations in $p(x)$ that may differ between domains [71]. Covariate shift encompasses subcategories such as domain shift, where the entire feature distribution undergoes change,

and sub-population shift, where the distribution’s support over features remains constant, but the distribution over subgroups (e.g., prior probabilities) alters [71]. In comparison, label shift assumes changes in the label distribution $p(y)$. Whereas concept shift implies alterations in the mechanism linking features to labels, denoted as $p(y|x)$.

There is ample opportunity for distribution shifts to occur within the health-care sector. One significant and frequent concern revolves around demographic sub-population shifts, encompassing variables like race, gender, or socioeconomic status among patients [73]. Additionally, the diversity observed across hospitals, including differences in operational protocols, ambient noise levels, or equipment standards, is a noteworthy example of domain shift [65] [74, 75]. Moreover, label shift can emerge from seemingly trivial factors such as variations in disease prevalence or transmission rates [76]. Lastly, many distribution shifts stem from concept drifts, such as changes in clinical procedures or modifications to the criteria defining particular diagnoses [65, 71, 75, 77]. In this work, we are primarily concerned with diachronic covariate shifts that affect model performance. Models are trained for each patient individually and clinical procedures have remained constant across our subject cohort.

Measuring Discrepancies Between Distributions Detecting distribution shifts involves quantifying the discrepancy and equality between two probability distributions, a fundamental problem in data analysis. Thus, it’s natural to consider the relationship between distributional shifts, data splitting regimes, and the robustness of trained machine learning models in this context. There are two major approaches for quantifying and detecting distribution shifts.

Integral probability metrics (IPMs) and f -divergences compare the empirical data distributions between two different periods in time. IPMs, such as the Wasserstein distance, total variation distance, and Maximum Mean Discrepancy (MMD), operate on the premise that if two distributions are identical, some critic function should yield the same expectation under both distributions [72] [78] [79]. The choice of critic function ultimately decides the specific distance metric, and in certain cases, can even be parameterized by neural networks, although this does not generally result in a metric [80] [81]. Conversely, f -divergences, such as KL divergence and the Jensen-Shannon divergence, leverage entropy functions to compare distributions pointwise, with identical distributions assigning identical likelihoods to each point. One can therefore quantify a discrepancy between two distributions based on the difference in likelihood ratios. A major challenge in this approach however is computing $D_f(Q||P)$, necessitating knowledge of the densities of distributions Q and P [82]. To address this, much work has been done to estimate density ratios directly, rather than the densities themselves, particularly in high-dimensional spaces where division can significantly increase the estimation error [82]. Furthermore, in empirical settings with finite samples, mismatches in the support of distributions Q and P require additional smoothing for accurate comparison. Notably, while f -divergences assume a constant value under disjoint support, IPMs with a smooth critic function furnish continuous distance measures even in such scenarios [81].

Sequential analysis continuously computes a test statistic between windows with the null hypothesis that the distribution between two periods is the same. The two-sample Kolmogorov-Smirnov (KS) test has gained widespread acceptance due to its

well-established history. While effective for examining univariate data, its applicability to high-dimensional multivariate data remains uncertain [83]. Similarly, kernel-based methods for comparing multivariate samples, such as MMD, which evaluates the difference between two probability distributions by considering the mean of their embeddings in a reproducing kernel Hilbert space, present an enticing approach. However, these methods suffer from scalability issues with large datasets and a significant decrease in statistical power as the dimensionality of the data increases [78]

Optimal Transport and Wasserstein Distances Within the probabilistic framework of distributional disparities between training and deployment data, optimal transport (OT) emerges as a potentially strong alternative for quantifying the discrepancy between two distributions. Originating from problems aiming to determine the most efficient transportation scheme for materials from one location to another without any loss, the foundational principles of optimal transport theory offer unique geometric advantages. Formally, optimal transport considers probability measures $\mu \in P(\Omega_s), \nu \in P(\Omega_t)$, where Ω_s, Ω_t are the source and target spaces, respectively. OT defines a push-forward operator $T : \Omega_s \rightarrow \Omega_t$ such that $T_{\#}\mu = \nu$ minimizes the transportation cost between distributions. The Wasserstein metric is the associated divergence. Similarly, significant changes in the Wasserstein distance between test and reference periods can be used to determine distribution shifts.

Crucially, unlike other divergences such as KL-divergence, which require the distributions to share a common support, Wasserstein distances alleviate this issue through the use of kernels, making them an attractive choice for comparing empirical distributions and shapes [84–86]. Moreover, conventional methods for computing

dissimilarities between distributions often fail to offer qualitative insights into the underlying reasons for observed differences. In contrast, the Wasserstein distance not only provides a measure of discrepancy but also offers a transformation map that aligns one distribution with another. Additionally, the Wasserstein distance has been shown to be resilient to minor perturbations in distributions, thereby affording a more robust empirical measure of distance in the presence of noise [79].

Several recent papers have proposed optimal transport as a tool for comparing datasets and detecting out-of-distribution samples. Alvarez-Melis & Fusi [87] propose a distance metric between datasets by solving an inner and outer OT problem over the features and labels respectively. Lu, Qin, Zhai, Shen, Chen, Wang, Kolouri, Stepputtis, Campbell & Sycara [88] build on OT by formulating a method to predict confidence in the presence of pseudo-label shift through optimal transport thresholding. Wasserstein distances have further been explored for use in hypothesis testing, such as its utility in goodness-of-fit testing and two-sample testing, as discussed in works in [89] and [90] respectively. Numerous other applications of OT are surveyed in [79].

The Monge problem: The original OT problem, considers probability measures $\mu \in P(\Omega_s), \nu \in P(\Omega_t)$, where Ω_s, Ω_t are the source and target spaces, respectively. The cost of moving a unit of mass from ω_s to ω_t is given by a cost function $c(\omega_s, \omega_t)$. There then exists a push-forward operator $T : \Omega_s \rightarrow \Omega_t$ such that $T_{\#}\mu = \nu$. Importantly, this push-forward operator must conserve mass, i.e. $\nu(A) = \mu(T^{-1}(A))$ for all $A \in \Omega_t$. The Monge problem seeks to find a mapping $T : \Omega_s \rightarrow \Omega_t$ that minimizes the cost:

$$\inf_{T_{\#}\mu=\nu} \int_{\Omega_s} c(x, T(x)) \mu(x) dx \quad (3.1)$$

Under this formulation, $T_{\#}\mu = \nu$ is not a convex constraint and there does not necessarily exist a unique solution. Furthermore, one can not map discrete measures to continuous measures which are of interest in the practical setting of comparing empirical distributions, frequently constructed from finite samples $x_1, x_2, \dots, x_n$, and $y_1, y_2, \dots, y_m$ as $\mu = \frac{1}{n} \sum_{i=1}^n \mu_i \delta_{x_i}$ and $\nu = \frac{1}{m} \sum_{j=1}^m \nu_j \delta_{y_j}$, where μ_i, ν_j are probabilities in the simplex.

The Kantorovich Problem: The Kantorovich formulation is a relaxation of the Monge problem. Rather than seek a strict mapping $T : \Omega_s \rightarrow \Omega_t$, the Kantorovich problem seeks a probabilistic coupling $\gamma \in \mathcal{P}(\Omega_s \times \Omega_t)$ (i.e. a joint distribution $P(\Omega_s \times \Omega_t)$) such that the marginals of γ are μ and ν [91].

$$\gamma \in \mathcal{P} = \gamma \geq 0, \int_{\Omega_s} \gamma(x, y) dy = \mu, \int_{\Omega_t} \gamma(x, y) dx = \nu \quad (3.2)$$

This relaxed formulation allows for the splitting of mass from one point to multiple points, and vice versa, and finds a coupling γ that minimizes the cost:

$$\inf_{\gamma \in \mathcal{P}} \int_{\Omega_s \times \Omega_t} c(x, y) \gamma(x, y) dx dy \quad (3.3)$$

If $c(x, y) = D(x, y)^p$, where D is a distance-metric and some $p \geq 1$, then for

measures μ, ν the p-Wasserstein distance is defined as:

$$W_p^p(\mu, \nu) = \left(\inf_{\gamma \in \mathcal{P}} \iint D(x, y)^p \gamma(x, y) dx dy \right) = \mathbb{E}_{(x, y) \sim \gamma} [D(X, Y)^p] \quad (3.4)$$

The case of $p = 1$ s is also known as the Earth Mover's Distance [92].

3.3 Data Splitting Strategies

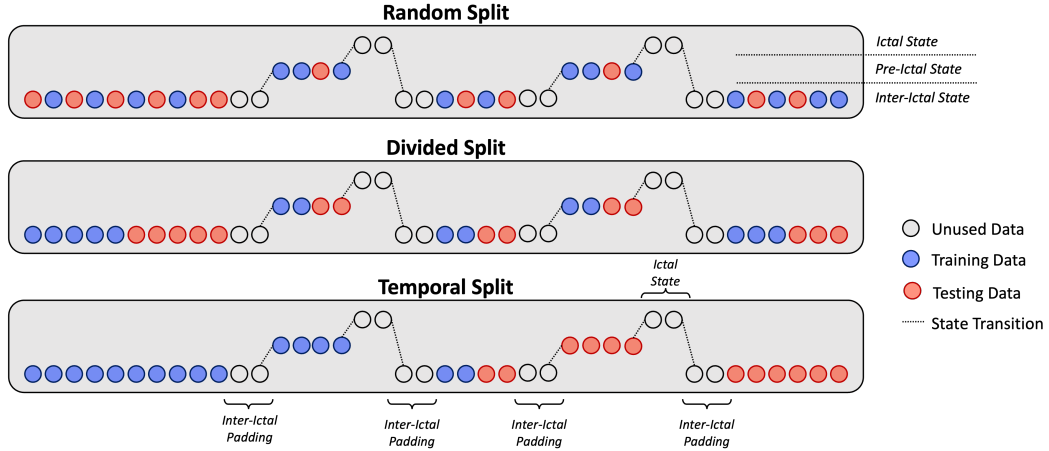


Figure 3.1: Illustration of the proposed data splitting strategies. Random Splitting (top) results in proportional randomly sampled segments for training and testing sets over the same support. Divided Splitting (middle) creates groups by dividing each continuous class event chronologically, whereas Temporal Splitting (bottom) chronologically divides the entire patient recording. Data used for training is shown in blue and testing in red. Data that is used to pad the ictal state is shown in grey. The validation set is not shown for brevity.

We introduce three methodologies aimed at highlighting the predictive efficacy of forecasting algorithms across various temporal data distributions. These approaches, termed Random Split (RS), Divided Split (DS), and Temporal Split (TS), illustrated

in Figure 3.1, are designed to organize training and testing segments in distinct time-aware ways.

In *Random Splits*, segments are sampled i.i.d. from the entire dataset, ensuring that they originate from the same temporal epoch (P_{train} and P_{test} originate from the same period). While prevalent in existing literature, this method disregards temporal structure and inter-segment dependencies for time-series data, reflecting anticipated performance in scenarios devoid of distributional shifts and in-distribution test sets. *Temporal Splitting* involves withholding contiguous data periods for testing, with the remaining data allocated for training. Due to the non-stationary nature of EEG and other time-series data, outcomes using this approach typically underperform compared to Random Splits. However, for signals exhibiting temporal distributional shifts, Temporal Splitting underscores model robustness, generalizability, and performance on out-of-distribution test data. *Divided Splitting* represents a hybridization of the aforementioned strategies. Data is partitioned by segmenting each continuous class instance chronologically, thereby reserving initial segments for training and latter segments for testing. While less severe than Temporal Splitting, this approach prevents the model from fully capturing the data distribution (as a random split would) in the presence of rapidly occurring temporal shifts and from memorizing highly correlated segments.

We anticipate superior model performance on testing splits when bolstered by robust training support. Naturally, Random Splits are expected to yield optimal results on held-out test data, given their representativeness of both training and testing data. We examine this empirically in Section 3.4.

To the best of our knowledge, this is the first endeavor to comprehensively compare the performance of seizure forecasting algorithms across various data-splitting strategies. Our investigation fills a critical gap in the existing literature by systematically evaluating the efficacy of these algorithms in accommodating different temporal data distributions. By rigorously examining the predictive power of forecasting models under diverse splitting methodologies, we aim to provide valuable insights into the robustness and generalizability of deployable seizure prediction algorithms that would inevitably be exposed to distribution shifts in the wild.

3.3.1 Selection of EEG recordings

We work with two surface EEG datasets for seizure detection and prediction: the publicly available CHB-MIT Scalp EEG Database [68] and the Vanderbilt University Medical Center (VUMC) Database curated in collaboration with William P. Nobis [93]. Both datasets consist of long-term surface-EEG recordings from patients with epilepsy. Each processed non-overlapping EEG segment is recorded at 256 Hz, is composed of the same 16 differential channels (See Appendix A; Table A.1), and is typically between 4 and 5 seconds (16,384 and 20,480 dimensions respectively) in duration. In contrast to the CHB-MIT database, electrodes in the VUMC database are recorded in reference to a common average reference (CAR) and converted to differential signals digitally. Segments 4 hours before or after the start/end of a seizure are considered baseline and those within 1 hour of seizure onset are classified as pre-ictal as in [20]. A significant portion of the data is labeled as baseline (non-seizure) data, thus for each subject, the dominating class was randomly subsampled to match the number of pre-ictal segments.

CHB-MIT Scalp EEG Database This database is a collection of 22 pediatric patients (grouped into 23 cases) with intractable epilepsy [94]. The authors share the following relevant information: subjects were monitored for several days at the Children’s Hospital Boston following the withdrawal of anti-seizure medication. Subject characteristics comprising the dataset are listed in Table A.1. Case 21 was obtained 1.5 years after case chb01 from the same subject. Occasionally there are gaps in the data, typically less than 10 seconds, but sometimes longer. 182 seizures were recorded in total over 916 hours. Sample counts averaged 3459 segments between subjects, ranging from 439 to 8,515. Subject sample counts and the class distribution for the utilized temporal split are shown in Table 3.2.

VUMC Database The portion of the VUMC database used in this study consists of 20 patients consisting of over 2093 hours of surface EEG recordings. A subset of patients were exposed to photic stimulation, a common trigger for seizures. Typically, these triggers are used to induce seizures in the epilepsy monitoring unit (EMU) to facilitate diagnosis and treatment. However, for this study, these triggers are not representative of the pre-ictal state in natural settings. To ensure that our analysis was focused on naturally induced interictal and pre-ictal EEG segments during nonseizure states, seizures that were induced by photic stimulation and showed no EEG correlation were labeled as nonseizure states. The resulting set of interictal segments is largely devoid of segments preceding and including artificially induced seizures. Across all patients, we collected 19 such photic stimulation sessions, each 4m 30s long. Therefore, of the roughly 1,959 hours of interictal baseline a maximum of 0.073% of the interictal, 1.188% of the pre-ictal, and 0.069% of the total nonseizure state is composed of photic

Chapter 3. Seizure prediction and non-stationary machine learning

Case	Proportional		Maximum		Temporal Split Train Class Distribution [Pre-ictal%, Interictal%]	Temporal Split Test Class Distribution [Pre-ictal%, Interictal%]
	4 Sec	5 Sec	4 Sec	5 Sec		
vumc01	2697-2697	2157-2157	2697-76310	2157-61048	[58.8, 41.2]	[14.9, 85.1]
vumc02	5327-5327	4261-4261	5327-24767	4261-19814	[62.5, 37.5]	[0.0, 100.0]
vumc03	2755-2755	2203-2203	2755-83848	2203-67079	[50.1, 49.8]	[49.4, 50.6]
vumc04	2697-2697	2157-2157	2697-55920	2157-44735	[55.3, 44.7]	[28.7, 71.3]
vumc06	9322-9322	7454-7454	9322-117208	7454-93767	[62.5, 37.5]	[0.0, 100.0]
vumc07	5394-5394	4314-4314	5394-91255	4314-73004	[46.6, 53.4]	[63.5, 36.5]
vumc08	5394-5394	4314-4314	5394-106915	4314-85532	[58.8, 41.2]	[14.8, 85.2]
vumc09	3596-3596	2876-2876	3596-76532	2876-61224	[48.0, 52.0]	[58.0, 42.0]
vumc10	2697-2697	2157-2157	2697-130934	2157-104747	[62.5, 37.5]	[0.0, 100.0]
vumc11	1798-1798	1438-1438	1798-122164	1438-97732	[62.5, 37.5]	[0.0, 100.0]
vumc12	7163-7163	5729-5729	7163-100439	5729-80351	[62.5, 37.5]	[0.0, 100.0]
vumc13	3596-3596	2876-2876	3596-151129	2876-120902	[62.5, 37.5]	[0.0, 100.0]
vumc14	6293-6293	5033-5033	6293-123456	5033-98764	[46.1, 53.9]	[65.5, 34.5]
vumc15	20504-20504	16396-16396	20504-91637	16396-73309	[50.5, 49.5]	[48.0, 52.0]
vumc16	3596-3596	2876-2876	3596-92941	2876-74353	[57.0, 43.0]	[22.9, 78.1]
vumc18	4426-4426	3540-3540	4426-59630	3540-47705	[49.8, 50.1]	[50.8, 49.2]
vumc19	3596-3596	2876-2876	3596-55372	2876-44296	[59.0, 41.0]	[14.1, 85.9]
vumc20	4475-4475	3579-3579	4475-80444	3579-64355	[56.3, 43.7]	[24.7, 75.3]
vumc21	8024-8024	6417-6417	8024-51357	6417-41086	[58.5, 41.5]	[16.2, 83.8]
vumc22	4457-4457	3565-3565	4457-70933	3565-56745	[50.1, 49.9]	[49.5, 50.5]

Table 3.1: Number of samples for each subject in the VUMC dataset. 4-second segments are used in Time Scale Networks, and 5-second segments are used in Convolutional Neural Networks. The pre-ictal and interictal class distribution for training and testing splits are shown for an 80%-20% temporal division. Subjects highlighted in bold contain at least both classes in the test split.

stimulations. Subject sample counts averaged 5133 segments between subjects, ranging from 1,798 to 20,504.

In total, we used 3,009 hours of EEG data from 43 patients across the two datasets, providing us with ample data to differentiate temporal distribution shifts. The average number of EEG segments per patient was 3459 and 5133 for patients in CHB-MIT and VUMC respectively. Subject sample counts and the class distribution for the utilized temporal split are shown in Table 3.1 and Table 3.2

Subject	Proportional		Maximum		Temporal Split Train Class Distribution [Pre-ictal%, Interictal%]	Temporal Split Test Class Distribution [Pre-ictal%, Interictal%]
	4 Sec	5 Sec	4 Sec	5 Sec		
chb01	5638-5638	4510-4510	5638-11128	4510-8900	[62.5, 37.5]	[0.0, 100]
chb02	2536-2536	2028-2028	2536-17129	2028-13699	[62.5, 37.5]	[0.0, 100]
chb03	4515-4515	3613-3613	4515-22830	3613-18259	[37.5, 62.5]	[100, 0.0]
chb04	2027-2027	1621-1621	2027-101878	1621-81497	[62.5, 37.5]	[0.0, 100]
chb05	4486-4486	3590-3590	4486-8386	3590-6706	[62.5, 37.5]	[0.0, 100]
chb06	8515-8515	6811-6811	8515-22309	6811-17847	[49.3, 50.7]	[52.8, 47.2]
chb07	2695-2695	2155-2155	2695-42547	2155-34057	[41.7, 58.3]	[83.4, 16.6]
chb08	0-0	0-0	4255-0	3402-0	N/A	N/A
chb09	3594-3594	2875-2875	3594-38116	2875-30491	[46.9, 53.1]	[62.5, 37.5]
chb10	5914-5914	4730-4730	5914-18248	4730-14596	[37.5, 62.5]	[100, 0.0]
chb11	1110-1110	887-887	1110-24277	887-1941	[37.5, 62.5]	[100, 0.0]
chb12	0-0	0-0	11391-0	9107-0	N/A	N/A
chb13	5075-5075	4057-4057	5075-7304	4057-5841	[37.5, 62.5]	[100, 0.0]
chb14	2444-2444	1955-1955	5068-2444	4053-1955	[62.5, 37.5]	[0.0, 100]
chb15	439-439	351-351	10310-439	8248-351	[37.5, 62.5]	[100, 0.0]
chb16	1481-1481	1185-1185	4703-1481	3760-1185	[37.5, 62.5]	[100, 0.0]
chb17	2250-2250	1800-1800	2250-10612	1800-8487	[62.5, 37.5]	[0.0, 100]
chb18	3073-3073	2460-2460	3073-19277	2460-15416	[41.6, 58.4]	[83.4, 16.6]
chb19	1603-1603	1282-1282	1603-19783	1282-15822	N/A	N/A
chb20	4861-4861	3888-3888	4861-13558	3888-10844	[58.0, 42.0]	[17.9, 82.1]
chb21	3191-3191	2552-2552	3191-14778	2552-11819	[60.4, 39.6]	[08.6, 91.4]
chb22	2109-2109	1687-1687	2109-10846	1687-8675	[53.2, 46.8]	[37.3, 62.7]
chb23	5099-5099	4077-4077	5099-9196	4077-7356	[62.5, 37.5]	[0.0, 100]

Table 3.2: Number of samples for each subject in the CHB-MIT dataset. 4-second segments are used in Time Scale Networks, 5-second segments are used in Convolutional Neural Networks. The pre-ictal and interictal class distribution for training and testing splits are shown for an 80%-20% temporal division. Subjects highlighted in bold contain at least both classes in the test split.

3.4 Methodology and Results

In this section, we empirically highlight the importance of employing isolated training, validation, and testing sets and investigate the effects of temporal variations in data distribution on baseline models for predicting seizures. We assess their efficacy across random, divided, and temporal splits.

Architecture and Evaluations We compare the proposed dataset splits using a simple selection of convolutional neural networks (CNNs), CNN plus long short-term memory networks (LSTMs) [20], and a lightweight edge-operable Time-Scale Network (TiSc) tailored for use in medical device [58] with ReLU activations. Due to the patient-specific nature of epilepsy, we train a new network for each patient using 10-fold cross-validation on 80% of the data and reserve 20% of the data for hyperparameter tuning. We then run each task a minimum of six times, also reserving 20% of the data for evaluation as described in Section 3.3 with different random seeds for each model, subject, and split. Further details on the architecture, hyperparameters, and optimization regimes are provided in Appendix A.1.2.

Metrics for Seizure Forecasting We note that to discern the deflationary impact on performance resulting from shifts in distribution, rather than challenges associated with fitting distributions P_{train} versus P_{test} , it is advantageous to maintain consistency in the distribution of evaluation data across Random, Divided, and Temporal splits as suggested in [65]. This is particularly pertinent in scenarios where the task becomes more demanding in P_{test} , such as encountering a higher frequency of interictal or pre-ictal segments alongside motion artifacts or fluctuating class proportions. In

such cases, a model trained on P_{train} and tested on P_{test} may overestimate the actual decrement in performance induced by the distributional shift [65]. Nevertheless, practical constraints impede the feasibility of this approach in this setting, requiring sufficient data within P_{test} to enable training and evaluation under a Random I.I.D split. Under Random splits, we therefore train on P_{train} while evaluating on a held-out subset of P_{train} as P_{test} . We acknowledge that this does not control the difficulty of the evaluation distribution with regard to Divided and Temporal splits, however, this limitation is less of an issue when we expect P_{train} and P_{test} to be of equal difficulty [65]. Temporal splits can introduce an imbalance in the class distribution of the training and testing sets, leading to mode collapse for some subjects. To mitigate some of the imbalances introduced, we exclusively report outcomes for subjects that contain both classes within P_{train} and P_{test} , reserving comprehensive results across all subjects for supplementary documentation in Appendix A.2. For all configurations, we report the ROC-AUC, PR-AUC, F1 score, and accuracy. Aggregated results for Random, Divided, and Temporal splits are reported as the arithmetic mean over the considered subjects.

3.4.1 Impact On Accuracy Deflation

In this section, we quantify the performance degradation between random, divided, and temporal splits. The prevalent approach in the literature, employing random splits, consistently yields near-perfect results across subjects (See Table 3.3). However, this approach fosters a misleading perception of model performance, as the predictive performance deteriorates sharply when faced with testing data from distinct temporal periods. Across all models, we observe a decline in test F1 score when transitioning

Chapter 3. Seizure prediction and non-stationary machine learning

Metric	Models	CHB-MIT			VUMC		
		TiSc Net	CNN	CNN-LSTM	TiSc Net	CNN	CNN-LSTM
Accuracy	Random	90.41±0.71	91.44±0.65	90.80±0.66	77.70±0.68	83.92±0.70	84.47±0.78
	Divided	84.35±1.02	83.19±1.26	83.01±1.48	72.48±1.35	74.79±0.75	75.70±1.06
	Temporal	76.29±4.12	75.37±4.91	71.53±16.40	62.65±4.57	69.58±1.61	70.14±3.96
F1	Random	90.07±0.87	91.29±1.06	90.03±1.72	77.26±1.30	84.02±1.22	84.15±2.03
	Divided	83.44±2.00	81.52±4.26	80.80±5.17	72.26±4.87	69.44±2.61	73.83±4.37
	Temporal	55.09±21.38	53.62±25.27	48.94±27.41	57.63±7.04	62.89±1.96	62.78±7.99
ROC-AUC	Random	94.64±0.57	95.73±0.48	94.87±0.56	83.17±1.05	90.48±0.56	90.63±0.92
	Divided	89.39±1.15	87.34±2.08	86.26±3.11	75.76±2.32	76.70±2.39	79.89±2.82
	Temporal	64.43±0.97	59.26±3.28	59.57±3.63	73.12±2.63	73.51±2.18	74.51±4.64
PR-AUC	Random	93.82±0.68	95.10±0.64	93.98±0.88	81.05±1.14	89.12±0.75	89.30±1.08
	Divided	87.91±1.21	85.69±2.56	84.11±3.81	72.36±2.59	73.54±2.89	77.60±3.30
	Temporal	52.57±0.89	49.39±2.37	49.01±3.44	54.71±2.83	55.38±2.57	60.33±4.25

Table 3.3: Average Model Performance across models and splits for CHB-MIT and VUMC.

from random to divided or temporal splits. This indicates that the predictive models trained for seizure prediction fail to adapt to changes in data distribution across different temporal intervals during testing. On average, CNN and CNN-LSTM networks experience a deflation of 29.4% and 31.2% when transitioning from a random split to a temporal split, respectively, and similarly a deflation of 12.2% and 9.8% in a divided setting. The TiSc Net architecture exhibits a comparatively lower drop in F1 score of 5.8% for divided and 27.3% for temporal splits, suggesting a potential robustness advantage and considerations for production use. Furthermore, despite similar temporal epochs across repetitions, we note that temporal and divided splits experience significantly larger variance in performance across seeds (Figure 3.2), highlighting the brittleness of models to out-of-distribution data compared to their randomly i.i.d. trained counterparts. The performance metrics for all subjects, some

of which exhibit significant class imbalances and mode collapse during inference, are further illustrated in Appendix A.2.

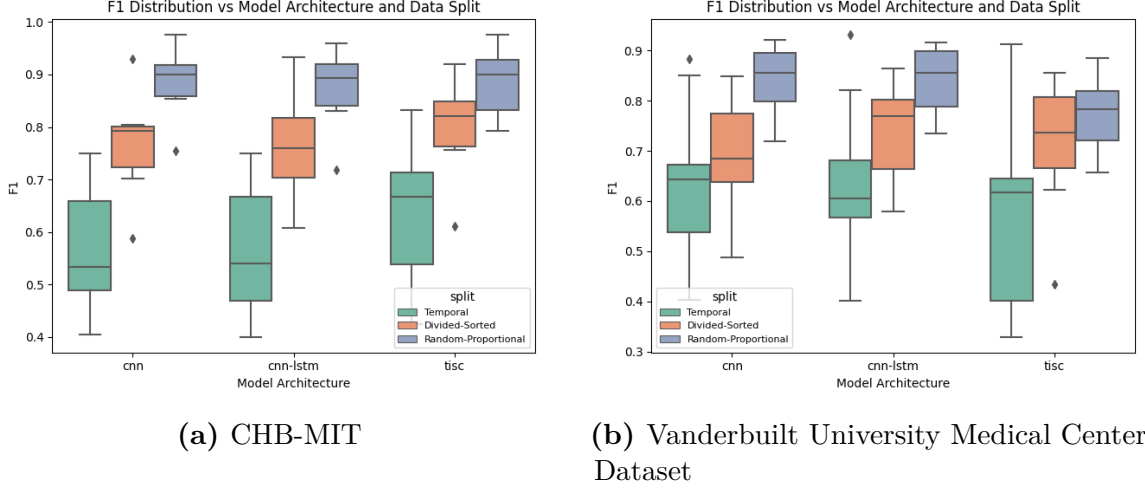


Figure 3.2: The in-distribution (Random i.i.d.) vs out-of-distribution (Divided, Temporal) F1 score of models trained on individual subjects containing both interictal and pre-ictal segments in their respective training and test sets as described in Section 3.3. For each dataset and model, higher numbers are better. The reported outcomes are aggregated over a minimum of six replicates across subjects and split strategy. Both datasets show a decrease in performance for all models due to distribution shifts and significantly better Random I.I.D performance than out-of-distribution performance.

On average (across all subjects), CNN and CNN-LSTM networks experience a deflation of 27.8% and 34.6% when transitioning from a random split to a temporal split, respectively, and similarly a deflation of 11.7% and 9.5% in a divided setting. The TiSc Net architecture exhibits a comparatively lower drop in F1 score of 5.5% for a divided split and a similar decrease of 31.7% for a temporal split. In Table 3.4 and Table 3.5 we present the full set of performance metrics for each utilized subject in the dataset for the VUMC and CHB-MIT datasets respectively.

Chapter 3. Seizure prediction and non-stationary machine learning

Model	Subject	Accuracy			F1			ROC-AUC			PR-AUC		
		Random	Divided	Temporal	Random	Divided	Temporal	Random	Divided	Temporal	Random	Divided	Temporal
TiSc Net	vumc01	81.94±0.85	69.96±3.26	37.70±3.07	81.50±0.92	67.44±4.97	32.83±4.12	89.42±0.56	71.25±2.72	84.10±1.81	87.97±0.65	63.13±2.68	34.16±2.22
	vumc02	89.34±0.96	95.51±0.50	99.64±0.30	89.06±1.19	95.05±1.00	49.19±53.89	95.09±0.50	99.17±0.43	50.00±0.00	94.55±0.63	98.94±0.93	0.00±0.00
	vumc03	84.28±0.68	82.02±0.65	95.99±1.27	83.29±0.78	79.55±0.95	91.30±5.49	91.66±0.71	86.53±1.83	96.59±2.78	92.99±0.46	90.39±1.10	94.61±5.10
	vumc04	65.77±1.22	51.75±2.38	44.38±14.09	65.73±1.61	43.33±26.17	36.41±7.27	69.53±1.51	51.82±1.25	42.19±5.61	64.91±1.11	51.31±1.57	23.31±1.99
	vumc06	90.60±0.98	84.99±3.12	69.55±9.18	90.56±1.02	86.14±2.57	31.70±34.85	94.38±0.52	92.92±1.27	50.00±0.00	91.84±0.71	89.63±1.25	0.00±0.00
	vumc07	73.27±0.69	53.85±1.50	62.68±0.91	70.85±1.89	62.19±5.40	62.54±9.75	79.23±0.80	62.19±1.16	63.33±3.58	79.59±0.54	61.47±0.72	70.90±2.67
	vumc08	85.36±0.22	73.37±0.58	59.28±1.55	85.60±0.50	76.01±2.75	49.85±9.70	89.09±0.57	79.02±0.81	86.12±0.70	85.09±0.97	74.97±1.11	40.57±1.16
	vumc09	69.59±1.35	64.58±1.16	61.63±0.79	69.23±2.84	66.24±2.86	61.56±2.48	71.06±2.84	61.97±2.04	67.54±1.45	66.94±2.42	58.51±1.72	71.05±0.90
	vumc10	81.23±1.18	60.51±1.12	62.22±41.57	81.18±1.54	65.17±5.28	3.51±3.97	87.95±1.04	68.98±2.30	50.00±0.00	85.17±1.28	62.35±1.83	0.00±0.00
	vumc11	90.24±0.94	92.25±2.38	64.72±10.11	89.94±0.86	89.71±5.50	31.00±34.58	95.48±0.62	93.00±6.83	50.00±0.00	95.27±0.64	87.54±11.82	0.00±0.00
	vumc12	76.33±0.92	76.94±0.87	60.37±1.56	76.90±1.86	76.24±2.74	27.40±30.06	83.27±1.13	80.44±2.68	50.00±0.00	80.40±1.51	78.85±1.45	0.00±0.00
	vumc13	86.22±0.87	90.54±0.55	81.03±2.00	86.10±1.18	90.22±0.86	38.52±42.25	92.35±0.73	95.07±0.59	50.00±0.00	90.84±1.03	93.05±1.39	0.00±0.00
	vumc14	74.13±0.41	69.75±1.17	62.05±1.98	72.79±0.81	68.40±2.12	64.92±7.74	78.97±1.05	74.58±2.52	61.35±2.10	78.73±0.95	74.12±2.43	73.46±1.51
	vumc15	81.93±0.45	81.92±1.14	71.33±1.92	82.09±0.50	81.04±1.38	71.37±3.11	89.95±0.33	88.05±1.34	73.21±3.18	89.13±0.50	83.06±2.43	64.92±3.79
	vumc16	72.77±0.63	70.32±3.42	48.84±26.97	71.97±1.58	71.23±6.33	34.11±14.54	78.41±1.19	73.47±6.25	82.12±0.69	75.71±0.94	66.86±2.32	42.70±0.95
	vumc18	79.98±0.65	82.94±0.61	80.42±1.11	79.32±0.76	80.94±2.53	78.81±4.38	87.31±0.64	87.47±2.00	85.08±3.01	85.62±0.77	86.72±2.72	81.17±3.08
	vumc19	88.10±0.25	85.15±1.00	81.12±5.14	88.54±0.72	85.46±2.11	61.85±10.17	92.17±0.43	85.96±3.95	84.97±2.00	87.60±1.13	77.67±7.27	36.31±5.05
	vumc20	76.99±0.96	86.70±0.60	33.40±0.67	77.26±1.59	85.60±3.29	36.81±6.02	82.48±1.24	89.73±2.14	47.51±2.75	80.77±1.70	85.89±2.80	22.68±1.27
	vumc21	80.19±0.54	81.12±0.40	76.19±2.74	80.83±0.98	80.13±2.61	62.89±10.09	88.56±0.70	84.80±2.38	88.81±1.31	87.29±1.24	78.44±4.50	49.25±5.17
	vumc22	73.56±0.66	61.27±1.00	62.10±1.75	72.69±2.71	64.05±4.68	61.50±3.72	76.60±2.14	63.78±2.16	60.75±5.88	72.35±2.53	60.56±2.92	60.90±4.78
	Average	80.09±0.77	75.77±1.37	65.73±6.43	79.77±1.29	75.71±4.31	49.40±14.91	85.65±0.96	79.51±2.33	66.18±1.84	83.64±1.09	76.17±2.75	38.30±1.98
	CNN	vumc01	88.40±0.55	67.59±1.04	78.25±2.23	87.85±1.46	61.23±2.70	64.93±0.07	94.33±0.41	68.00±1.80	91.74±1.24	93.75±0.47	61.85±2.59
vumc02		93.79±0.22	96.50±0.22	99.77±0.06	93.76±0.25	94.29±0.59	87.33±7.11	97.79±0.21	98.58±0.30	50.00±0.00	97.18±0.55	98.54±0.23	0.00±0.00
vumc03		89.73±0.50	86.05±0.49	96.11±1.19	89.75±0.98	84.28±0.80	88.28±0.24	96.36±0.39	92.95±1.01	93.93±0.60	96.38±0.79	92.85±1.32	89.27±2.36
vumc04		70.44±0.88	50.48±0.35	56.12±2.53	72.00±2.18	48.75±0.64	47.55±3.42	76.90±1.25	48.91±0.60	46.90±1.37	76.00±1.12	50.55±1.37	23.40±0.57
vumc06		86.63±3.22	85.25±0.21	66.93±5.42	87.09±3.80	69.10±9.82	37.23±11.41	96.39±0.36	95.75±0.89	50.00±0.00	95.44±0.68	93.45±1.50	0.00±0.00
vumc07		81.01±0.73	70.66±0.80	66.86±1.21	80.26±0.94	68.15±0.53	59.15±4.04	88.50±0.60	71.58±1.27	65.43±4.30	88.61±0.73	69.95±1.08	74.74±2.85
vumc08		88.99±0.28	79.27±0.09	68.62±0.72	89.44±0.48	78.15±1.16	63.87±2.51	94.27±0.37	83.57±1.28	85.76±1.05	92.26±1.25	78.37±2.23	40.51±2.07
vumc09		78.54±1.33	66.32±0.35	61.69±2.17	77.53±3.90	64.50±0.58	52.24±3.63	86.50±1.02	72.39±1.13	63.44±2.04	85.80±0.92	76.10±0.37	64.88±1.65
vumc10		92.38±1.11	78.46±1.23	68.37±2.26	91.41±2.91	69.29±2.27	56.89±2.91	97.10±0.75	84.18±1.87	50.00±0.00	96.48±1.23	80.86±2.66	0.00±0.00
vumc11		88.30±0.81	76.27±13.06	91.26±9.18	87.39±2.73	70.60±14.80	70.37±11.40	94.86±0.67	63.07±25.89	50.00±0.00	95.08±0.65	58.11±24.14	0.00±0.00
vumc12		79.72±2.00	82.38±0.09	89.12±4.88	78.26±7.14	76.91±3.85	76.53±1.33	88.91±1.16	87.91±1.46	50.00±0.00	86.48±1.60	86.67±1.97	0.00±0.00
vumc13		92.36±0.61	93.14±1.61	94.09±1.99	91.69±1.75	87.38±7.47	84.19±5.05	97.39±0.49	96.28±1.45	50.00±0.00	96.73±1.28	94.93±1.63	0.00±0.00
vumc14		80.15±0.75	76.70±0.40	71.35±1.92	79.88±1.15	75.02±1.62	66.15±3.32	87.36±0.71	82.29±0.42	67.53±1.68	85.89±0.99	80.82±0.45	71.96±1.15
vumc15		89.75±0.54	85.34±1.11	70.17±0.75	89.51±0.98	79.29±2.56	66.53±1.41	95.91±0.43	88.31±0.96	74.31±1.84	95.80±0.51	85.93±1.01	68.92±1.72
vumc16		75.63±1.03	67.65±2.41	74.28±2.14	76.89±0.56	63.60±1.37	67.56±0.96	83.49±0.58	72.54±2.34	78.05±1.76	79.91±0.85	67.59±2.96	38.03±2.50
vumc18		88.08±0.49	77.94±0.49	58.29±1.92	87.90±0.52	71.39±4.15	51.37±0.78	94.06±0.54	80.64±4.22	66.94±7.81	92.75±0.91	74.47±0.77	62.90±5.12
vumc19		89.62±0.41	85.46±0.35	73.53±1.29	89.99±0.68	84.88±0.43	69.16±0.53	95.09±0.21	88.35±1.24	92.80±1.79	93.23±0.37	80.51±2.77	52.22±6.03
vumc20		83.21±0.54	89.17±0.28	48.39±2.51	83.16±0.74	68.90±1.78	40.29±2.94	90.79±0.38	77.93±2.58	53.05±2.73	89.47±0.38	76.21±1.83	28.09±1.63
vumc21		92.34±0.54	77.53±2.20	88.74±1.33	92.19±0.61	68.07±14.07	85.04±2.32	97.69±0.21	74.76±12.53	90.39±0.73	97.11±0.26	67.60±12.84	50.78±2.36
vumc22		79.05±1.25	66.90±0.07	61.66±0.60	79.99±1.89	55.91±4.18	58.39±1.23	85.50±0.80	71.50±2.07	58.89±1.53	80.73±0.92	66.76±2.62	53.61±1.13
Average		85.41±0.89	77.95±1.34	74.18±2.31	85.30±1.78	71.98±3.77	64.65±3.33	91.96±0.58	79.97±3.27	66.46±1.52	90.75±0.82	77.11±3.63	38.77±1.80
CNN-LSTM		vumc01	87.89±0.86	69.94±1.75	73.42±4.83	86.50±2.92	66.02±6.36	59.02±11.06	93.98±0.67	71.84±5.76	91.01±1.08	92.55±0.82	67.58±6.86
	vumc02	93.75±0.63	95.76±0.29	98.65±0.92	93.71±0.37	95.05±1.26	49.91±46.31	98.26±0.16	99.13±0.38	50.00±0.00	97.92±0.32	98.95±0.59	0.00±0.00
	vumc03	90.05±0.89	82.62±0.28	97.19±0.77	89.80±1.07	79.85±1.39	93.20±3.96	96.84±0.51	92.72±0.32	98.49±1.22	97.08±0.43	93.13±0.23	98.58±1.17
	vumc04	72.43±0.75	51.68±1.38	51.87±15.49	73.48±0.92	57.91±9.04	40.09±7.36	78.38±0.94	49.87±1.60	42.70±17.81	77.72±1.43	49.18±3.77	23.38±7.06
	vumc06	86.79±5.48	87.04±2.80	69.50±24.55	87.94±3.76	84.27±7.69	25.78±27.36	95.75±1.03	95.13±1.85	50.00±0.00	94.56±1.43	92.45±2.21	0.00±0.00
	vumc07	81.33±0.81	70.91±1.64	67.42±1.97	80.35±1.75	69.23±2.22	65.08±8.72	87.99±0.65	73.75±4.26	70.59±3.73	87.78±0.59	72.49±3.37	77.84±2.54
	vumc08	89.80±0.43	80.17±0.78	71.83±9.26	89.94±0.85	80.42±2.57	56.26±9.55	93.79±0.23	84.25±1.02	86.13±0.94	90.96±0.59	79.44±1.95	46.28±3.67
	vumc09	79.58±1.29	66.52±0.64	64.07±3.27	78.06±3.93	65.99±0.77	60.14±6.42	88.20±1.30	73.24±0.75	71.37±4.38	87.65±1.09	77.47±0.58	69.83±3.58
	vumc10	92.64±0.70	78.64±1.56	65.59±45.09	92.70±0.88	75.17±4.92	13.30±18.54	97.53±0.34	85.46±2.94	50.00±0.00	97.26±0.53	84.29±1.87	0.00±0.00
	vumc11	87.88±0.86	77.39±10.57	78.82±19.23	86.98±1.85	71.02±9.31	37.22±34.08	94.05±0.59	69.45±16.89	50.00±0.00	94.04±0.53	61.94±16.49	0.00±0.00
	vumc12	82.37±1.50	81.79±0.62	92.30±3.72	78.87±8.85	79.30±3.93	45.03±41.32	88.97±2.67	88.54±2.32	50.00±0.00	87.01±0.92	87.92±2.14	0.00±0.00
	vumc13	92.05±0.41	94.75±0.53	93.78±1.89	91.85±0.64	93.43±2.51	52.37±47.82	97.29±0.22	98.33±0.54	50.00±0.00	97.01±0.53	98.25±0.57	0.00±0.00
	vumc14	80.68±0.35	77.64±0.62	72.20±1.28	80.42±0.98	77.22±1.48	73.80±6.43	87.56±0.56	83.86±0.96	70.45±2.79	86.04±0.94	83.26±0.86	75.56±2.76
	vumc15	91.18±0.60	86.49±1.51	70.34±1.69	91.19±0.69	81.44±6.89	69.13±4.35	96.81±0.35	90.58±4.57	75.36±2.45	96.79±0.43	88.86±5.31	69.70±2.04
	vumc16	74.72±1.27	71.16±2.13	76.25±4.09	76.54±3.11	67.76±8.41	58.03±11.09	81.42±0.95	75.86±2.43	76.59±3.07	77.49±1.02	71.95±3.35	39.90±5.12
	vumc18	87.32±0											

Chapter 3. Seizure prediction and non-stationary machine learning

Model	Subject	Accuracy			F1			ROC-AUC			PR-AUC		
		Random	Divided	Temporal	Random	Divided	Temporal	Random	Divided	Temporal	Random	Divided	Temporal
TiSc Net	chb01	1.00±0.00	0.98±0.00	0.99±0.00	1.00±0.00	0.97±0.00	0.37±0.51	1.00±0.00	0.99±0.00	0.50±0.00	1.00±0.00	0.99±0.00	0.00±0.00
	chb02	0.91±0.01	0.89±0.01	0.88±0.01	0.90±0.02	0.87±0.03	0.32±0.45	0.97±0.01	0.96±0.02	0.50±0.00	0.97±0.01	0.96±0.02	0.00±0.00
	chb03	0.89±0.01	0.82±0.01	0.66±0.04	0.89±0.01	0.82±0.02	0.72±0.12	0.94±0.01	0.86±0.01	0.50±0.00	0.94±0.00	0.81±0.02	1.00±0.00
	chb04	0.81±0.02	0.68±0.01	0.57±0.06	0.80±0.01	0.68±0.02	0.20±0.28	0.87±0.01	0.72±0.01	0.50±0.00	0.83±0.02	0.69±0.02	0.00±0.00
	chb05	0.90±0.01	0.76±0.01	0.89±0.02	0.89±0.01	0.75±0.02	0.30±0.41	0.96±0.01	0.83±0.02	0.50±0.00	0.96±0.01	0.84±0.02	0.00±0.00
	chb06	0.90±0.01	0.86±0.00	0.83±0.01	0.90±0.00	0.86±0.01	0.83±0.01	0.95±0.00	0.92±0.00	0.88±0.01	0.94±0.00	0.91±0.01	0.90±0.01
	chb07	0.93±0.01	0.92±0.00	0.46±0.02	0.93±0.01	0.92±0.01	0.48±0.07	0.97±0.00	0.96±0.00	0.85±0.01	0.96±0.00	0.93±0.01	0.96±0.01
	chb09	0.93±0.00	0.85±0.01	0.63±0.02	0.93±0.01	0.82±0.02	0.60±0.05	0.97±0.00	0.92±0.00	0.77±0.02	0.97±0.00	0.89±0.00	0.84±0.02
	chb10	0.95±0.00	0.80±0.00	0.85±0.00	0.95±0.00	0.79±0.01	0.89±0.04	0.99±0.00	0.83±0.03	0.50±0.00	0.99±0.00	0.88±0.01	1.00±0.00
	chb11	0.91±0.02	0.91±0.02	0.86±0.02	0.91±0.02	0.90±0.01	0.89±0.07	0.97±0.01	0.96±0.01	0.50±0.00	0.97±0.01	0.97±0.01	1.00±0.00
	chb13	0.97±0.00	0.97±0.00	0.97±0.00	0.97±0.01	0.97±0.00	0.98±0.01	0.99±0.00	0.99±0.00	0.50±0.00	0.99±0.00	0.98±0.00	1.00±0.00
	chb14	0.95±0.00	0.91±0.01	0.93±0.02	0.94±0.00	0.90±0.01	0.30±0.41	0.98±0.00	0.95±0.01	0.50±0.00	0.99±0.00	0.97±0.01	0.00±0.00
	chb15	0.93±0.01	0.89±0.02	0.70±0.34	0.91±0.02	0.87±0.03	0.61±0.31	0.95±0.01	0.92±0.02	0.50±0.00	0.96±0.01	0.95±0.02	1.00±0.00
	chb16	0.91±0.00	0.95±0.01	0.88±0.01	0.91±0.01	0.94±0.01	0.90±0.05	0.95±0.01	0.99±0.00	0.50±0.00	0.93±0.02	0.99±0.00	1.00±0.00
	chb17	0.93±0.01	0.90±0.02	0.98±0.01	0.92±0.01	0.89±0.02	0.36±0.49	0.97±0.00	0.96±0.01	0.50±0.00	0.96±0.01	0.96±0.01	0.00±0.00
	chb18	0.86±0.01	0.79±0.02	0.66±0.04	0.86±0.01	0.77±0.01	0.72±0.06	0.92±0.01	0.87±0.01	0.69±0.04	0.92±0.01	0.86±0.01	0.93±0.01
	chb20	0.98±0.00	0.86±0.01	0.83±0.01	0.98±0.00	0.84±0.03	0.67±0.12	1.00±0.00	0.96±0.01	0.88±0.02	1.00±0.00	0.94±0.02	0.46±0.06
	chb21	0.81±0.01	0.75±0.01	0.65±0.01	0.81±0.01	0.76±0.02	0.42±0.17	0.89±0.01	0.82±0.01	0.78±0.01	0.86±0.01	0.80±0.02	0.17±0.01
	chb22	0.79±0.01	0.60±0.01	0.76±0.02	0.79±0.01	0.61±0.04	0.71±0.02	0.86±0.01	0.59±0.03	0.81±0.03	0.83±0.01	0.53±0.02	0.61±0.02
	chb23	0.98±0.00	0.93±0.01	0.96±0.01	0.98±0.00	0.90±0.03	0.36±0.50	1.00±0.00	0.98±0.01	0.50±0.00	1.00±0.00	0.98±0.01	0.00±0.00
	Average	0.91±0.01	0.85±0.01	0.80±0.03	0.91±0.01	0.84±0.02	0.58±0.21	0.95±0.01	0.90±0.01	0.61±0.01	0.95±0.01	0.89±0.01	0.54±0.01
CNN	chb01	1.00±0.00	0.96±0.01	0.99±0.00	1.00±0.00	0.95±0.01	0.48±0.53	1.00±0.00	0.98±0.01	0.50±0.00	1.00±0.00	0.97±0.02	0.00±0.00
	chb02	0.96±0.01	0.97±0.01	0.96±0.01	0.96±0.01	0.97±0.01	0.46±0.50	0.99±0.00	0.99±0.00	0.50±0.00	0.99±0.00	0.99±0.00	0.00±0.00
	chb03	0.94±0.01	0.77±0.02	0.19±0.12	0.94±0.01	0.76±0.05	0.20±0.18	0.98±0.00	0.83±0.02	0.50±0.00	0.98±0.00	0.80±0.03	1.00±0.00
	chb04	0.89±0.00	0.65±0.03	0.73±0.03	0.89±0.01	0.66±0.06	0.29±0.33	0.95±0.00	0.70±0.03	0.50±0.00	0.93±0.01	0.67±0.03	0.00±0.00
	chb05	0.96±0.00	0.84±0.01	0.96±0.01	0.95±0.02	0.79±0.07	0.41±0.45	0.99±0.00	0.91±0.02	0.50±0.00	0.99±0.00	0.91±0.02	0.00±0.00
	chb06	0.74±0.03	0.71±0.01	0.62±0.05	0.75±0.01	0.70±0.04	0.53±0.07	0.81±0.03	0.76±0.04	0.52±0.09	0.80±0.03	0.76±0.04	0.59±0.08
	chb07	0.93±0.00	0.94±0.00	0.60±0.10	0.93±0.01	0.93±0.02	0.53±0.21	0.98±0.00	0.96±0.01	0.93±0.02	0.98±0.00	0.93±0.03	0.98±0.00
	chb09	0.90±0.01	0.82±0.01	0.54±0.05	0.90±0.01	0.80±0.03	0.45±0.18	0.96±0.00	0.90±0.01	0.40±0.16	0.95±0.01	0.87±0.02	0.60±0.12
	chb10	0.91±0.02	0.80±0.04	0.91±0.03	0.91±0.02	0.80±0.03	0.82±0.18	0.97±0.01	0.88±0.04	0.50±0.00	0.97±0.02	0.89±0.04	1.00±0.00
	chb11	0.96±0.00	0.89±0.02	0.96±0.01	0.96±0.00	0.87±0.03	0.86±0.13	0.99±0.00	0.96±0.02	0.50±0.00	0.99±0.00	0.95±0.02	1.00±0.00
	chb13	0.97±0.00	0.95±0.01	1.00±0.00	0.97±0.00	0.95±0.01	0.98±0.02	0.99±0.00	0.98±0.00	0.50±0.00	0.99±0.00	0.98±0.01	1.00±0.00
	chb14	0.93±0.02	0.83±0.09	0.88±0.04	0.93±0.02	0.83±0.07	0.28±0.36	0.98±0.01	0.89±0.08	0.50±0.00	0.98±0.01	0.89±0.08	0.00±0.00
	chb15	0.97±0.00	0.89±0.01	0.97±0.02	0.97±0.00	0.80±0.08	0.92±0.08	0.99±0.00	0.92±0.02	0.50±0.00	0.99±0.00	0.95±0.01	1.00±0.00
	chb16	0.85±0.01	0.98±0.01	0.87±0.02	0.84±0.01	0.93±0.06	0.78±0.16	0.91±0.01	0.99±0.01	0.50±0.00	0.90±0.01	0.99±0.01	1.00±0.00
	chb17	0.95±0.01	0.85±0.02	0.93±0.01	0.95±0.01	0.85±0.02	0.45±0.49	0.98±0.00	0.90±0.03	0.50±0.00	0.98±0.00	0.87±0.07	0.00±0.00
	chb18	0.92±0.01	0.81±0.02	0.56±0.18	0.91±0.04	0.80±0.03	0.58±0.18	0.97±0.01	0.87±0.03	0.52±0.18	0.97±0.00	0.85±0.03	0.85±0.08
	chb20	0.98±0.00	0.85±0.02	0.86±0.02	0.98±0.01	0.79±0.07	0.74±0.05	1.00±0.00	0.92±0.03	0.94±0.01	1.00±0.00	0.87±0.04	0.69±0.03
	chb21	0.85±0.00	0.74±0.01	0.67±0.10	0.85±0.01	0.74±0.03	0.41±0.26	0.92±0.00	0.81±0.01	0.76±0.01	0.91±0.01	0.79±0.01	0.09±0.00
	chb22	0.86±0.01	0.58±0.01	0.82±0.01	0.87±0.02	0.59±0.10	0.75±0.05	0.92±0.01	0.56±0.04	0.82±0.03	0.89±0.02	0.55±0.03	0.61±0.04
	chb23	0.96±0.00	0.96±0.00	0.99±0.00	0.96±0.00	0.95±0.02	0.47±0.52	0.99±0.00	0.99±0.00	0.50±0.00	0.99±0.00	0.99±0.00	0.00±0.00
	Average	0.92±0.01	0.84±0.02	0.80±0.04	0.92±0.01	0.82±0.04	0.57±0.25	0.96±0.01	0.89±0.02	0.57±0.02	0.96±0.01	0.87±0.03	0.52±0.02
CNN-LSTM	chb01	1.00±0.00	0.94±0.03	0.99±0.01	1.00±0.00	0.89±0.10	0.45±0.50	1.00±0.00	0.98±0.02	0.50±0.00	1.00±0.00	0.97±0.02	0.00±0.00
	chb02	0.97±0.00	0.97±0.00	0.98±0.02	0.96±0.01	0.95±0.02	0.47±0.51	0.99±0.00	0.99±0.01	0.50±0.00	0.99±0.01	0.98±0.01	0.00±0.00
	chb03	0.94±0.00	0.75±0.03	0.38±0.47	0.94±0.01	0.73±0.08	0.25±0.38	0.98±0.00	0.79±0.07	0.50±0.00	0.98±0.01	0.77±0.06	1.00±0.00
	chb04	0.90±0.01	0.67±0.03	0.70±0.27	0.90±0.02	0.67±0.07	0.22±0.24	0.95±0.00	0.71±0.05	0.50±0.00	0.94±0.01	0.69±0.04	0.00±0.00
	chb05	0.95±0.00	0.83±0.03	0.98±0.02	0.94±0.01	0.75±0.08	0.45±0.49	0.98±0.00	0.84±0.06	0.50±0.00	0.98±0.00	0.87±0.04	0.00±0.00
	chb06	0.72±0.01	0.74±0.01	0.58±0.04	0.72±0.01	0.68±0.05	0.54±0.08	0.77±0.01	0.74±0.06	0.64±0.13	0.76±0.01	0.70±0.05	0.68±0.10
	chb07	0.93±0.01	0.94±0.01	0.69±0.20	0.93±0.01	0.93±0.02	0.61±0.24	0.97±0.00	0.97±0.01	0.96±0.02	0.97±0.01	0.95±0.02	0.99±0.01
	chb09	0.90±0.01	0.83±0.01	0.45±0.12	0.89±0.02	0.76±0.09	0.47±0.23	0.95±0.01	0.87±0.03	0.31±0.13	0.94±0.01	0.85±0.03	0.54±0.10
	chb10	0.85±0.01	0.70±0.03	0.72±0.23	0.84±0.01	0.70±0.04	0.73±0.25	0.92±0.01	0.76±0.03	0.50±0.00	0.91±0.01	0.73±0.03	1.00±0.00
	chb11	0.96±0.01	0.91±0.02	0.77±0.34	0.95±0.01	0.89±0.02	0.70±0.34	0.99±0.00	0.96±0.01	0.50±0.00	0.99±0.00	0.95±0.01	1.00±0.00
	chb13	0.97±0.00	0.94±0.01	1.00±0.00	0.97±0.01	0.93±0.02	0.97±0.05	0.99±0.00	0.98±0.00	0.50±0.00	0.99±0.00	0.97±0.01	1.00±0.00
	chb14	0.83±0.01	0.69±0.07	0.54±0.51	0.83±0.01	0.70±0.04	0.02±0.04	0.90±0.01	0.73±0.07	0.50±0.00	0.90±0.01	0.72±0.05	0.00±0.00
	chb15	0.95±0.01	0.85±0.02	0.92±0.06	0.94±0.01	0.81±0.05	0.86±0.13	0.98±0.01	0.91±0.02	0.50±0.00	0.98±0.00	0.94±0.01	1.00±0.00
	chb16	0.82±0.00	0.97±0.01	0.51±0.31	0.80±0.04	0.96±0.02	0.36±0.11	0.88±0.01	1.00±0.00	0.50±0.00	0.87±0.01	1.00±0.00	1.00±0.00
	chb17	0.93±0.01	0.84±0.01	0.65±0.45	0.92±0.01	0.83±0.04	0.31±0.46	0.98±0.00	0.89±0.03	0.50±0.00	0.97±0.01	0.86±0.09	0.00±0.00
	chb18	0.92±0.01	0.82±0.01	0.50±0.35	0.91±0.02	0.81±0.03	0.47±0.39	0.97±0.01	0.84±0.05	0.51±0.14	0.97±0.01	0.77±0.09	0.85±0.05
	chb20	0.98±0.00	0.86±0.02	0.89±0.02	0.96±0.03	0.83±0.08	0.73±0.03	1.00±0.00	0.89±0.06	0.90±0.06	1.00±0.00	0.85±0.08	0.57±0.17
	chb21	0.84±0.01	0.73±0.01	0.68±0.09	0.83±0.03	0.73±0.03	0.40±0.23	0.91±0.01	0.80±0.02	0.80±0.01	0.88±0.02	0.77±0.02	0.12±0.02
	chb22	0.86±0.01	0.59±0.02	0.80±0.05	0.85±0								

3.4.2 Temporal Non-Stationary

In the following sections, we explore various methods for quantifying, tracking, and visualizing temporal distributional shifts.

t-SNE Visualization of dataset shifts

In Figure 3.3, we present a comparative analysis of training and test data obtained through Random and Temporal splits for CHB-MIT. The figure shows the unsupervised t-distributed Stochastic Neighbor Embedding (t-SNE) [95] projection of latent activations from the final feature extraction layer for training and test data across three subjects along with the reported test accuracy for each of the classifiers. For ease of visualization, the data has been projected onto two dimensions.

As examples of subjects with large accuracy deflations, in subjects CHB09 and CHB18 we observe notable disparities in the distributions of test data compared to training data under a temporal split versus a random i.i.d. setting. This supports our intuition that there exists a temporal shift that can lead to significant declines in classifier performance. However, such divergence is not universal. Subject CHB06, for instance, exhibits only marginal differences in data distributions between training and test sets, resulting in minimal degradation of test accuracy, even within a temporal split context.

Moreover, we observe that certain classifiers exacerbate the disjoint support between training and test data more than others. Specifically, CNN-based architectures, commonly utilized as feature extractors in seizure prediction tasks, display heightened

sensitivity to the proposed splits - isolating pre-ictal segments in the training and test datasets to a greater extent compared to the TiSc network. While they perform adequately under i.i.d. sampling within the random split, this tenancy alongside their inherent specificity (i.e. high parameter count) suggests a leaning towards memorization which may impede generalization in the presence of covariate shift. Somewhat surprisingly however, for the presented subjects, TiSc consistently yields a discernible boundary that separates a substantial portion of baseline and pre-ictal segments while maintaining support from both the training (P_{train}) and test (P_{test}) data.

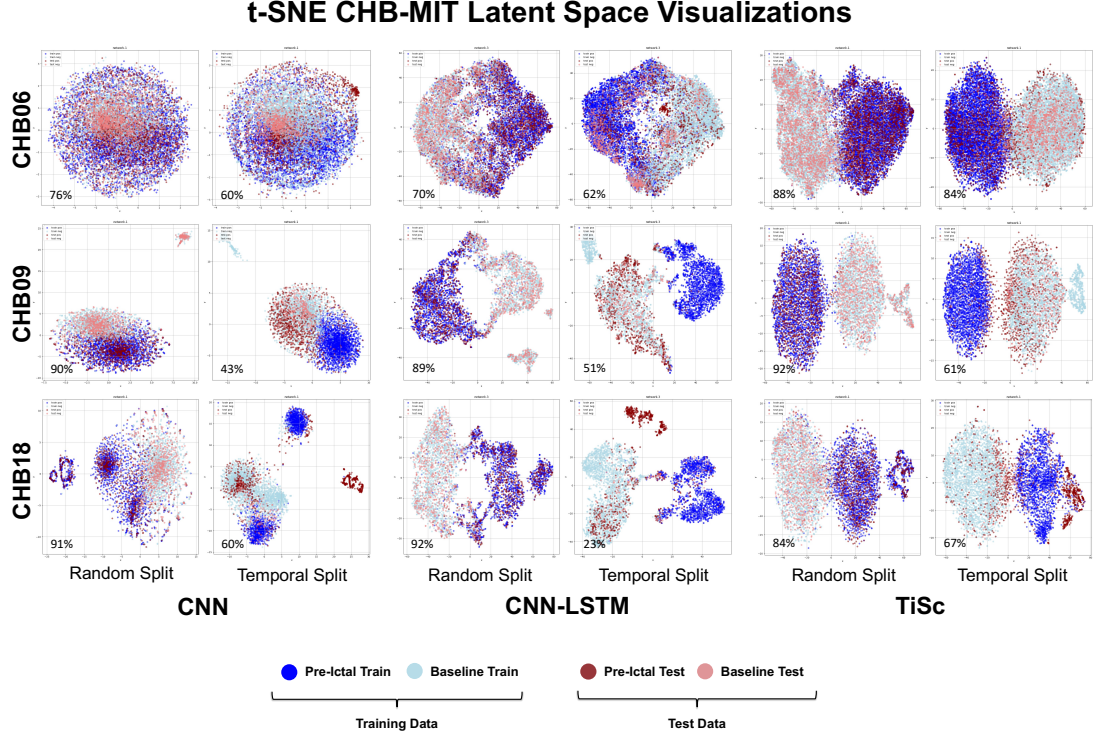


Figure 3.3: CHBMIT: t-SNE dimensionality reduction of activations from the last feature extraction layer. For good performance on Random I.I.D splits, we expect dark and light (Pre-Ictal vs Baseline) to be well separated and blue and red (P_{train} vs P_{test}) to be well mixed representing similar embeddings for train/test segments. Subjects CHB06, CHB09, and CHB18 are shown as examples of subjects with accuracy deflations that are small, large, and model-dependent, respectively. Testing accuracy is shown in the bottom left corner.

3.4.2.1 Multivariate Linear Shifts and PCA Reconstruction Error

Reconstruction error analysis serves as a useful approach employed in detecting covariate shifts within the multivariate setting, wherein the marginal distributions may remain the same while alterations occur in the joint distribution [96, 97]. Principal Component Analysis (PCA) emerges as a prominent technique for dimensionality reduction, achieved by learning an optimal orthogonal transformation from the d -dimensional input space to a projection across the orthogonal principal components that span the k -dimensional subspace (where $k \ll d$) [98]. This transformation is learned in such a way that the first principal component accounts for as much of the variability in the dataset as possible, and that each succeeding principal component captures as much of the remaining variance as possible subject to the constraint that it be orthogonal to the preceding components [99]. The reconstruction error can be conceptualized as the average squared distance between original data points and their respective projections onto the principal subspace. Hence, minimizing the reconstruction error equates to maximizing the variance of the projected data [97, 99]. By learning a compression over a subset of data, detection of covariate shift becomes feasible by comparing the reconstruction error of shifted data to that of the expected reconstruction error over the training data [96, 100].

Table 3.6 presents a comparison of the average reconstruction error among the last feature extraction layers of each neural network across all subjects. Each model underwent training a minimum of six times using the same set of hyperparameters as outlined in Appendix A.1.2. A ratio of 1 indicates that the learned PCA embedding captures a similar amount of error across both the test and training sets. Conversely,

a higher ratio signals the presence of covariate shift, indicating that the learned PCA transformations no longer adequately represent the test data. Across all evaluated models, test sets formed via a Random split exhibit the lowest reconstruction error, indicating minimal distributional shift, while those formed via a temporal split yield the highest error (an additional 13% higher on average), suggestive of significant distributional shift. Moreover, with the exception of the CNN-LSTM model evaluated on CHB-MIT, there is a degradation in reconstruction quality between the divided and temporal splits.

The presented results are averaged over all subjects for each dataset. While we did explore initial ideas on identifying if there was a statistically significant difference in relative test reconstruction error between "class balanced" subjects and the remaining subset, conducting a student's t-test yields unremarkable results. Furthermore, while the magnitude of relative reconstruction error is more noticeable in the CNN-based architectures, we note that comparing the reconstruction errors between models is not necessarily meaningful as the dimensionality varies between their layers. Moreover, we acknowledge that a notable limitation of using PCA for dimensionality reduction lies in its inability to discern nonlinear deviations that maintain means, standard deviations, and inter-feature correlations [101]. PCA dimensionality reduction techniques are surjective, with many input mappings from dimension d to the same output in k [102]. Given the nonlinear dynamics inherent in seizure activity and the potential for nonlinear covariate shifts within data distributions [103] [104], we present an alternative method built on Optimal Transport for detecting multivariate distributional shifts in Section 3.4.2.3.

Dataset	Models	Average Relative Test Reconstruction Error (%)		
		Random	Divided	Temporal
CHBMIT	TiSc Net	1.01 (0.01)	1.02 (0.02)	1.08 (0.04)
	CNN	1.22 (0.01)	1.22 (0.02)	1.31 (0.04)
	CNN-LSTM	1.00 (0.01)	1.17 (0.08)	1.15 (0.09)
VUMC	TiSc Net	1.02 (0.01)	1.07 (0.03)	1.23 (0.11)
	CNN	1.14 (0.03)	1.21 (0.05)	1.33 (0.06)
	CNN-LSTM	1.01 (0.01)	1.05 (0.03)	1.08 (0.04)

Table 3.6: Relative PCA reconstruction error between test and training splits. A ratio of 1 suggests that the learned PCA embedding captures a similar amount of error over the test set as it does the training set. A higher ratio is indicative of covariate shift, as the learned PCA transformations are no longer representative of the test data. For any particular model, test sets constructed with a Random split produce the lowest reconstruction error (low distributional shift) and those with a Temporal split tend to produce the highest (Significant distributional shift). Results are aggregated over a minimum of six replicates with the standard deviation reported in parentheses.

3.4.2.2 *Relative Test PCA Reconstruction Error*

In Table 3.7 and Table 3.8 we present the full evaluation of relative test PCA reconstruction error for the VUMC and CHB-MIT datasets respectively.

Subject	Relative Test Reconstruction Error (%) (Std)								
	TiSc			CNN			CNN-LSTM		
	Random (I.I.D)	Divided	Temporal	Random (I.I.D)	Divided	Temporal	Random (I.I.D)	Divided	Temporal
vumc01	1.10 (0.02)	1.04 (0.04)	1.06 (0.06)	1.17 (0.04)	1.12 (0.05)	1.25 (0.06)	1.00 (0.01)	1.07 (0.03)	1.07 (0.04)
vumc02	0.98 (0.01)	1.66 (0.03)	2.23 (0.75)	1.08 (0.01)	1.57 (0.07)	2.00 (0.14)	1.00 (0.02)	1.19 (0.04)	1.23 (0.10)
vumc03	1.02 (0.02)	0.94 (0.01)	1.09 (0.03)	1.13 (0.01)	1.10 (0.02)	1.25 (0.01)	1.01 (0.01)	1.05 (0.02)	1.04 (0.01)
vumc04	1.05 (0.01)	0.93 (0.01)	0.89 (0.03)	1.19 (0.03)	0.98 (0.02)	1.01 (0.02)	1.00 (0.01)	0.97 (0.02)	0.95 (0.03)
vumc06	0.94 (0.01)	0.77 (0.01)	0.98 (0.04)	1.06 (0.03)	0.97 (0.02)	1.12 (0.02)	1.01 (0.01)	0.96 (0.01)	1.15 (0.10)
vumc07	1.01 (0.00)	1.18 (0.04)	1.26 (0.07)	1.11 (0.03)	1.25 (0.03)	1.25 (0.02)	1.01 (0.01)	1.06 (0.01)	1.09 (0.04)
vumc08	0.99 (0.00)	0.96 (0.03)	1.11 (0.01)	1.11 (0.06)	1.22 (0.01)	1.26 (0.03)	1.00 (0.02)	1.01 (0.03)	1.14 (0.04)
vumc09	1.05 (0.02)	1.05 (0.02)	1.12 (0.02)	1.20 (0.03)	1.21 (0.01)	1.33 (0.05)	1.00 (0.02)	1.03 (0.03)	1.09 (0.01)
vumc10	1.05 (0.03)	0.86 (0.05)	0.79 (0.04)	1.11 (0.02)	1.04 (0.03)	1.21 (0.04)	1.03 (0.02)	1.11 (0.06)	1.12 (0.06)
vumc11	0.93 (0.02)	1.43 (0.15)	1.38 (0.20)	1.10 (0.04)	1.82 (0.35)	1.72 (0.26)	1.02 (0.02)	1.21 (0.05)	1.22 (0.06)
vumc12	0.99 (0.01)	1.16 (0.03)	1.14 (0.04)	1.19 (0.06)	1.39 (0.14)	1.29 (0.10)	1.01 (0.02)	1.05 (0.01)	0.99 (0.03)
vumc13	0.99 (0.01)	1.13 (0.03)	1.64 (0.17)	1.16 (0.03)	1.16 (0.05)	1.38 (0.03)	1.01 (0.01)	1.04 (0.04)	1.10 (0.09)
vumc14	1.05 (0.02)	1.00 (0.01)	1.36 (0.05)	1.13 (0.02)	1.14 (0.02)	1.29 (0.07)	1.00 (0.01)	1.01 (0.01)	1.07 (0.04)
vumc15	1.01 (0.00)	1.29 (0.03)	1.36 (0.10)	1.15 (0.02)	1.35 (0.02)	1.41 (0.05)	1.00 (0.01)	1.10 (0.01)	1.11 (0.03)
vumc16	1.00 (0.01)	0.97 (0.02)	0.87 (0.04)	1.14 (0.03)	1.14 (0.03)	0.90 (0.03)	1.01 (0.01)	1.03 (0.02)	0.97 (0.02)
vumc18	1.00 (0.00)	0.94 (0.02)	0.96 (0.02)	1.11 (0.01)	1.09 (0.03)	1.11 (0.02)	1.00 (0.01)	0.99 (0.00)	0.99 (0.03)
vumc19	1.04 (0.01)	0.93 (0.01)	1.37 (0.01)	1.13 (0.02)	1.10 (0.01)	1.29 (0.03)	0.99 (0.01)	1.00 (0.05)	0.95 (0.02)
vumc20	1.01 (0.01)	1.00 (0.02)	1.27 (0.23)	1.09 (0.03)	1.13 (0.06)	1.20 (0.04)	1.01 (0.01)	1.04 (0.01)	1.12 (0.03)
vumc21	1.02 (0.01)	1.21 (0.06)	1.50 (0.20)	1.12 (0.01)	1.20 (0.06)	1.81 (0.15)	1.01 (0.01)	1.07 (0.04)	1.15 (0.05)
vumc22	1.01 (0.01)	1.03 (0.01)	1.32 (0.04)	1.24 (0.04)	1.23 (0.04)	1.45 (0.07)	1.01 (0.01)	1.06 (0.02)	1.09 (0.02)
Average	1.02 (0.01)	1.07 (0.03)	1.23 (0.11)	1.14 (0.03)	1.21 (0.05)	1.33 (0.06)	1.01 (0.01)	1.05 (0.03)	1.08 (0.04)

Table 3.7: Relative PCA reconstruction error between test and training splits for subjects in VUMC. A ratio of 1 suggests that the learned PCA embedding captures a similar amount of error over the test set as it does the training set. A higher ratio is indicative of covariate shift, as the learned PCA transformations are no longer representative of the test data. For any particular model, test sets constructed with a Random split produce the lowest reconstruction error (low distributional shift) and those with a Temporal split produce the highest (Significant distributional shift). Results are aggregated over a minimum of six replicates with the standard deviation reported in parentheses.

Subject	Relative Test Reconstruction Error (%)								
	TiSc			CNN			CNN-LSTM		
	Random (I.I.D)	Divided	Temporal	Random (I.I.D)	Divided	Temporal	Random (I.I.D)	Divided	Temporal
chb01	1.00 (0.00)	1.14 (0.08)	1.29 (0.07)	1.10 (0.02)	1.26 (0.02)	1.29 (0.05)	1.01 (0.03)	2.52 (0.65)	1.84 (0.53)
chb02	1.01 (0.00)	0.86 (0.03)	0.87 (0.06)	1.18 (0.01)	1.06 (0.01)	1.13 (0.02)	1.00 (0.02)	1.07 (0.05)	1.01 (0.05)
chb03	1.04 (0.01)	1.03 (0.01)	1.15 (0.03)	1.18 (0.01)	1.17 (0.03)	1.23 (0.03)	1.01 (0.02)	1.02 (0.03)	1.08 (0.04)
chb04	1.03 (0.01)	0.83 (0.02)	1.43 (0.07)	1.15 (0.01)	1.02 (0.02)	1.60 (0.02)	1.02 (0.02)	1.13 (0.04)	1.22 (0.03)
chb05	1.01 (0.00)	1.11 (0.01)	0.96 (0.02)	1.20 (0.01)	1.29 (0.02)	1.22 (0.02)	1.00 (0.01)	1.15 (0.06)	1.14 (0.07)
chb06	1.00 (0.00)	1.00 (0.00)	0.98 (0.02)	1.27 (0.01)	1.26 (0.01)	1.28 (0.02)	1.00 (0.00)	1.01 (0.01)	1.04 (0.02)
chb07	1.03 (0.00)	0.99 (0.01)	0.95 (0.03)	1.22 (0.01)	1.22 (0.02)	1.22 (0.02)	0.99 (0.01)	1.04 (0.03)	1.09 (0.09)
chb09	1.01 (0.00)	1.03 (0.01)	1.09 (0.02)	1.23 (0.01)	1.24 (0.01)	1.35 (0.02)	1.01 (0.01)	1.07 (0.03)	1.07 (0.04)
chb10	1.01 (0.00)	1.01 (0.01)	1.12 (0.02)	1.25 (0.01)	1.27 (0.01)	1.40 (0.01)	1.00 (0.01)	1.04 (0.03)	1.09 (0.04)
chb11	1.04 (0.02)	1.03 (0.03)	1.10 (0.11)	1.23 (0.00)	1.23 (0.02)	1.45 (0.05)	0.98 (0.02)	1.14 (0.07)	1.31 (0.13)
chb13	1.01 (0.00)	1.01 (0.02)	0.85 (0.04)	1.17 (0.02)	1.14 (0.01)	1.02 (0.02)	1.00 (0.02)	1.01 (0.03)	1.00 (0.06)
chb14	1.02 (0.00)	0.93 (0.02)	0.91 (0.02)	1.26 (0.01)	1.22 (0.01)	1.18 (0.01)	0.99 (0.01)	1.03 (0.04)	1.00 (0.03)
chb15	1.11 (0.02)	1.00 (0.04)	1.39 (0.06)	1.32 (0.02)	1.16 (0.03)	1.44 (0.05)	1.03 (0.02)	1.21 (0.06)	1.37 (0.12)
chb16	1.03 (0.00)	1.09 (0.02)	1.12 (0.04)	1.26 (0.01)	1.35 (0.02)	1.58 (0.04)	1.00 (0.01)	1.03 (0.02)	1.14 (0.04)
chb17	1.01 (0.00)	1.12 (0.03)	0.95 (0.05)	1.21 (0.02)	1.34 (0.02)	1.10 (0.02)	1.00 (0.01)	1.20 (0.07)	1.10 (0.12)
chb18	1.03 (0.01)	1.13 (0.00)	1.36 (0.13)	1.23 (0.02)	1.30 (0.03)	1.78 (0.24)	0.99 (0.01)	1.20 (0.05)	1.30 (0.12)
chb20	1.01 (0.01)	0.98 (0.01)	1.06 (0.01)	1.20 (0.01)	1.10 (0.01)	1.17 (0.02)	1.01 (0.02)	1.23 (0.13)	1.22 (0.08)
chb21	1.01 (0.00)	0.98 (0.02)	1.08 (0.04)	1.24 (0.02)	1.26 (0.01)	1.31 (0.02)	0.98 (0.00)	1.11 (0.04)	1.03 (0.02)
chb22	1.02 (0.01)	1.10 (0.02)	1.05 (0.03)	1.24 (0.01)	1.31 (0.01)	1.18 (0.02)	0.98 (0.01)	1.22 (0.05)	1.04 (0.03)
chb23	1.01 (0.01)	0.99 (0.02)	0.97 (0.03)	1.23 (0.01)	1.19 (0.01)	1.23 (0.01)	1.01 (0.01)	1.04 (0.02)	1.01 (0.07)
Average	1.01 (0.01)	1.02 (0.02)	1.08 (0.04)	1.22 (0.01)	1.22 (0.02)	1.31 (0.04)	1.00 (0.01)	1.17 (0.08)	1.15 (0.09)

Table 3.8: Relative PCA reconstruction error between test and training splits for subjects in CHB-MIT. A ratio of 1 suggests that the learned PCA embedding captures a similar amount of error over the test set as it does the training set. A higher ratio is indicative of covariate shift, as the learned PCA transformations are no longer representative of the test data. For any particular model, test sets constructed with a Random split produce the lowest reconstruction error (low distributional shift) and those with a Temporal split tend to produce the highest (Significant distributional shift), with the exception of CNN-LSTM. Results are aggregated over a minimum of six replicates with the standard deviation reported in parentheses.

3.4.2.3 *Online Out-Of-Distribution Detection with Optimal Transport Dataset Distance*

A driving motivation for proposing alternative data-splitting strategies stems from the need to establish a pre-deployment criterion for assessing the generalization capabilities of seizure prediction algorithms. In our experimental setup, we adopt a formulation of dataset distances proposed by [87] leveraging optimal transport theory. The Optimal Transport Dataset Distance (OTDD) enables quantification of the temporal discrepancy between training and testing sets across all features and optionally the labels of the dataset. We compute the OTDD ratio as the ratio between the distances observed between training and test splits, as calculated on both temporal and divided splits, relative to the random split.

The OTDD distances between training and testing sets are independent of the model architecture and computed for each subject and split based on the raw segment features. Figure 3.4 illustrates the OTDD ratio compared to accuracy deflation in instances where an 80%-20% temporal division encompasses interictal and pre-ictal segments across both training and testing sets. In these subjects, we observe a significant correlation between the OTDD ratio and degradation in model performance across datasets. Furthermore, temporal splits for a given subject typically exhibit a higher OTDD ratio compared to their corresponding divided splits, reflecting the inherent temporality within the splits. This alignment underscores OTDD as a reliable indicator of model generalization, robustness, and temporal data characteristics. As in [87], we test the robustness of OTDD to training and test sample sizes for subjects in the CHB-MIT and VUMC datasets by computing the distance repeatedly. As expected, variance decreases as the sample size approaches the number of samples in

the respective splits (Appendix A; Figure A.7).

In Figure 3.5, we present a conceptual framework outlining the application of Optimal Transport distances for discerning distributional deviations in raw EEG features within an operational system at inference in which test batches are compared to the training data. We further juxtapose the extent of OTDD observed between two distinct data period partitions: a random split (Figure 3.4a) and a temporal split (Figure 3.4b). By establishing thresholds based on the Relative Optimal Transport Distance, such as three standard deviations from the mean of a reference epoch (P_{train}), one can effectively generate alerts for possibly out-of-distribution data and low competence predictions during inference. Further examples are provided in Appendix A.

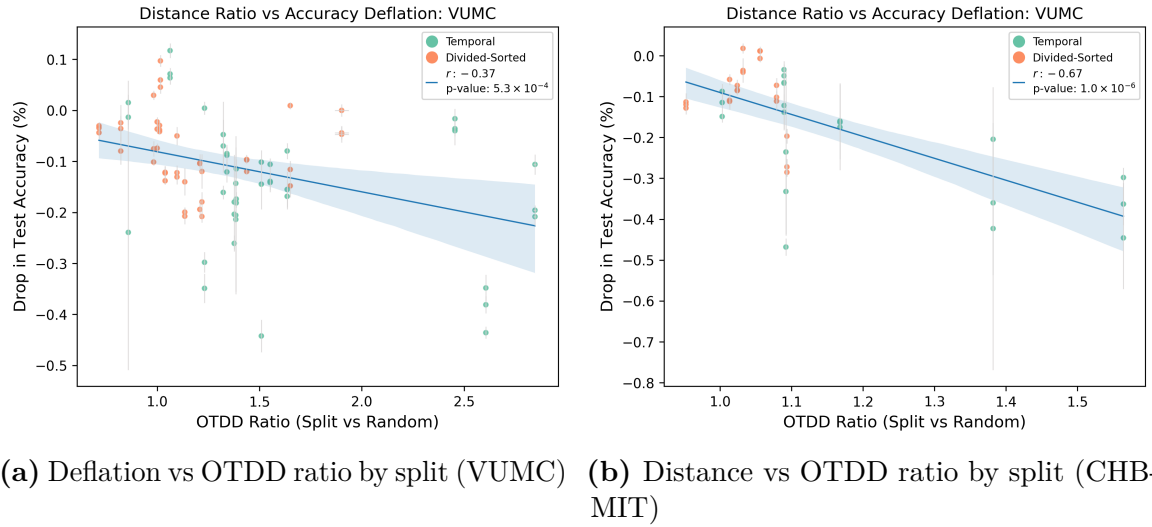


Figure 3.4: Dataset distance ratio vs data split for VUMC and CHBMIT. The error bars correspond to ± 1 s.d. over six repetitions. A subset of subjects is selected on the criterion that both the training and testing sets contain both interictal and pre-ictal segments. Dataset distances are computed on the raw EEG segments using the OTDD metric. Ratios are computed as the ratio of the distances between temporal/divided and Random Proportional splits.

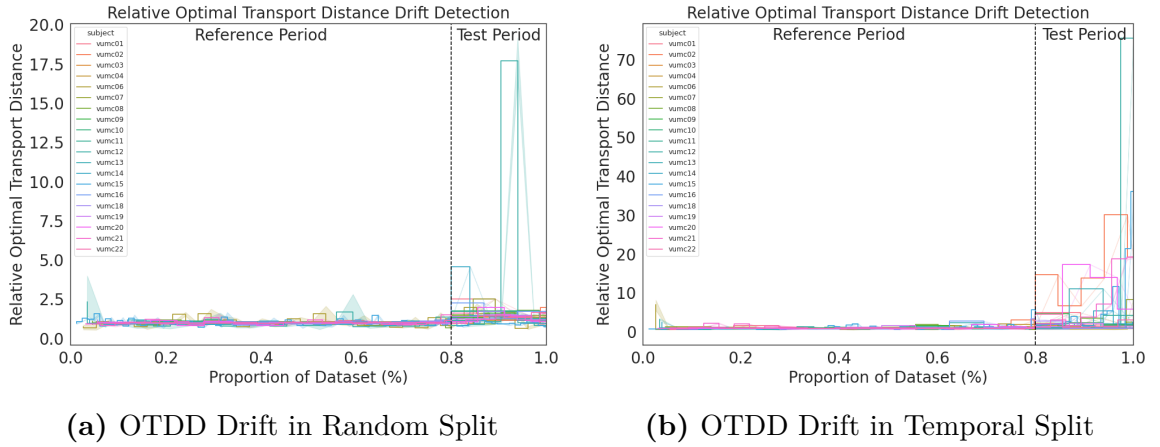


Figure 3.5: Relative Optimal Transport Distance between reference distribution (P_{train}) and chunks from the test distribution (P_{test}) A) Random i.i.d. splits and B) Temporal splits. The distance of 500-sample chunks is compared against a maximum of 10,000 samples from the reference distribution. To compute the distance ratio, the distance of a chunk is compared to the average distance of a chunk within the reference period. Temporal splits introduce a notable delineation between in-distribution data and out-of-distribution data and therefore typically experience significantly higher OTDD ratios than their random split counterpart. Each experiment is repeated 3 times.

3.5 Discussion

The findings of this study reveal significant insights into the performance disparities observed among different data-splitting techniques in machine learning models. Our results demonstrate that both divided splitting and temporal splitting consistently exhibit lower accuracies compared to random splitting. This suggests that the prevalent use of random splitting may contribute to the discrepancy between promising offline results and subpar performance in real-time scenarios.

Considering the performance of divided or temporal split methodologies could serve as a valuable tool for researchers in identifying algorithms that generalize well to unseen data, thereby enhancing efficacy in real-world applications. Moreover, our analysis sheds light on the impact of model complexity on performance. While high-parameter convolutional neural network (CNN) and CNN-long short-term memory (CNN-LSTM) models are often favored for their potential to improve performance, our findings illustrate that they may be more prone to mischaracterizing unseen EEG data. Conversely, implicitly regularized models with fewer parameters, such as TiSc Net, appear to offer a more robust solution in deployment, albeit with a moderate decrease in performance.

Additionally, we propose latent space visualizations, PCA-multivariate-reconstruction errors, and optimal transport distances as model-agnostic techniques for assessing algorithm robustness. These measures provide valuable insights into the generalizability and stability of machine learning models to varying input data. Further research into the application of these metrics across various domains is warranted.

3.6 Conclusion

We systematically study the impact of possible temporal covariate shifts in time-series EEG data for the seizure prediction task in two different datasets: CHB-MIT and VUMC. Evidence of distribution shift within subjects is explored visually with t-SNE and quantified via PCA reconstruction error and optimal transport dataset distance. Further, we propose two time-aware strategies to organize training and testing segments for prediction tasks. Based on over 10,000 trained models, we highlight the impact of out-of-distribution data on the degradation of performance.

Chapter 4

Conclusion

Machine learning has the potential to significantly enhance treatment, diagnosis, and prognosis for patients with epilepsy. When integrated within an embedded device, this endeavor furthers the ability to bridge two pivotal clinical phases: sensing and intervention. Efficient integration of closed-loop sensing, data-driven decision-making, and targeted intervention is a major problem that this thesis aims to explore. We focus on two major limiting factors of personalized treatment: relevant research tools and the shortcomings of modern ML solutions.

In **Chapter 2** we presented Toast, a framework designed to address the intricate demands of developing, training, and rigorously evaluating neural networks over extensive patient-specific datasets. Toast’s focus on individualized treatments aligns with the personalized nature of epilepsy management, facilitating the exploration and refinement of patient-centric predictive models. We showcase Toast’s practical application through the development of the Time Scale Network (TiSc Net), demonstrating its effectiveness in research and algorithm design. In [58], we evaluate our system in a subject-specific setting and lay the groundwork for future directions in lightweight interpretable models capable of operating on the edge. TiSc Net has the potential to enable continual learning directly on implanted medical devices all while ensuring patient privacy and mobility, as it prevents the need for data transfer outside the device.

In **Chapter 3**, we dove into some of the challenges posed by evolving data distributions, particularly in the context of surface EEG data from epilepsy patients. One pressing concern emerges during model deployment: the potential disparity between the data distribution at inference time and during training, which leads to unwarranted confidence in predictions and results in erroneous outcomes. To study this, we proposed alternative methods for organizing training, validation, and test data to mimic real-world scenarios better and address the over-reliance on random sampling for high performance in research settings. We explored model-agnostic techniques to monitor distributional shifts in input features and assess the impact of out-of-distribution data on model performance. Finally, we developed a system capable of alerting machine learning practitioners when a model in production may underperform based on an unsupervised optimal transport distance metric. To facilitate this work and future longitudinal studies, we developed the VUMC dataset in collaboration with William P. Nobis [93], providing a rich resource for further exploration.

The work presented in this thesis represents a significant advancement toward enhancing treatment options for patients with epilepsy. The presented Python package aims to move the needle for what’s possible in regard to model development and patient-specific algorithm research. Moreover, by addressing key challenges such as evolving data distributions, model deployment robustness, and privacy concerns, we strive to lay the groundwork for more effective and scalable solutions in epilepsy diagnosis and treatment. Looking forward, the integration of innovative approaches like TiSc Net holds the promise of revolutionizing healthcare delivery, equipping clinicians with powerful tools for personalized, data-driven interventions while ensuring patient

privacy and confidentiality. To support future work in the field, we have made all of our resources available (See Section 1.3).

Bibliographic references

1. World Health Organization. *Epilepsy* 2024.
2. Bosl, W. J., Leviton, A. & Loddenkemper, T. Prediction of Seizure Recurrence. A Note of Caution. *Frontiers in neurology* **12**, 675728. doi:[10.3389/fneur.2021.675728](https://doi.org/10.3389/fneur.2021.675728) (2021).
3. Stafstrom, C. E. & Carmant, L. Seizures and Epilepsy: An Overview for Neuroscientists. *Cold spring harbor perspectives in medicine* **5**, a022426–a022426. doi:[10.1101/cshperspect.a022426](https://doi.org/10.1101/cshperspect.a022426) (2015).
4. Besag, F. & Vasey, M. *Social cognition and psychopathology in childhood and adolescence* 2019. doi:[10.1016/j.yebeh.2019.03.015](https://doi.org/10.1016/j.yebeh.2019.03.015).
5. Kloster, R. & Engelskjon, T. Sudden unexpected death in epilepsy (SUDEP): a clinical perspective and a search for risk factors. *Journal of neurology, neurosurgery & psychiatry* **67**, 439–444. doi:[10.1136/jnnp.67.4.439](https://doi.org/10.1136/jnnp.67.4.439) (1999).
6. Devinsky, O. & Sisodiya, S. M. SUDEP: Advances and Challenges. *Epilepsy currents* **20**, 29S–31S. doi:[10.1177/1535759720947438](https://doi.org/10.1177/1535759720947438) (2020).
7. Tellez-Zenteno, J. & Nguyen, R. Injuries in epilepsy: a review of its prevalence, risk factors, type of injuries and prevention. *Neurology international* **1**, 20. doi:[10.4081/ni.2009.e20](https://doi.org/10.4081/ni.2009.e20) (2009).
8. Kanner, A. M. Depression and Epilepsy: A New Perspective on Two Closely Related Disorders. *Epilepsy currents* **6**, 141–146. doi:[10.1111/j.1535-7511.2006.00125.x](https://doi.org/10.1111/j.1535-7511.2006.00125.x) (2006).
9. Kanner, A. M. & Balabanov, A. Depression and epilepsy: How closely related are they? *Neurology* **58**. doi:[10.1212/WNL.58.8_suppl_5.S27](https://doi.org/10.1212/WNL.58.8_suppl_5.S27) (2002).
10. Jackson, M. J. Depression and anxiety in epilepsy. *Journal of neurology, neurosurgery & psychiatry* **76**, i45–i47. doi:[10.1136/jnnp.2004.060467](https://doi.org/10.1136/jnnp.2004.060467) (2005).

Bibliographic references

11. Feigin, V. L. *et al.* Global, regional, and national burden of neurological disorders, 1990–2016: a systematic analysis for the Global Burden of Disease Study 2016. *The lancet neurology* **18**, 459–480. doi:[10.1016/S1474-4422\(18\)30499-X](https://doi.org/10.1016/S1474-4422(18)30499-X) (2019).
12. Beghi, E. The Epidemiology of Epilepsy. *Neuroepidemiology* **54**, 185–191. doi:[10.1159/000503831](https://doi.org/10.1159/000503831) (2020).
13. Dumanis, S., French, J., Bernard, C., Worrell, G. & Fureman, B. *Seizure forecasting from idea to reality* 2017. doi:[10.1523/ENEURO.0349-17.2017](https://doi.org/10.1523/ENEURO.0349-17.2017).
14. Ghosh, S., Sinha, J. K., Khan, T., Devaraju, K. S., Singh, P., Vaibhav, K. & Gaur, P. Pharmacological and Therapeutic Approaches in the Treatment of Epilepsy. *Biomedicines* **9**, 470. doi:[10.3390/biomedicines9050470](https://doi.org/10.3390/biomedicines9050470) (2021).
15. Miller, J. W. & Hakimian, S. Surgical Treatment of Epilepsy: *Continuum: lifelong learning in neurology* **19**, 730–742. doi:[10.1212/01.CON.0000431398.69594.97](https://doi.org/10.1212/01.CON.0000431398.69594.97) (2013).
16. Skarpaas, T. L., Jarosiewicz, B. & Morrell, M. J. Brain-responsive neurostimulation for epilepsy (RNS® System). *Epilepsy research* **153**, 68–70. doi:[10.1016/j.eplepsyres.2019.02.003](https://doi.org/10.1016/j.eplepsyres.2019.02.003) (2019).
17. Bergey, G. K., Morrell, M. J., Mizrahi, E. M., Goldman, A., King-Stephens, D., Nair, D., Srinivasan, S., Jobst, B., Gross, R. E., Shields, D. C., Barkley, G., Salanova, V., Olejniczak, P., Cole, A., Cash, S. S., Noe, K., Wharen, R., Worrell, G., Murro, A. M., Edwards, J., Duchowny, M., Spencer, D., Smith, M., Geller, E., Gwinn, R., Skidmore, C., Eisenschenk, S., Berg, M., Heck, C., Van Ness, P., Fountain, N., Rutecki, P., Massey, A., O'Donovan, C., Labar, D., Duckrow, R. B., Hirsch, L. J., Courtney, T., Sun, F. T. & Seale, C. G. Long-term treatment with responsive brain stimulation in adults with refractory partial seizures. *Neurology* **84**, 810–817. doi:[10.1212/WNL.0000000000001280](https://doi.org/10.1212/WNL.0000000000001280) (2015).
18. Fan, J.-J., Shan, W., Wu, J.-P. & Wang, Q. Research progress of vagus nerve stimulation in the treatment of epilepsy. *Cns neuroscience & therapeutics* **25**, 1222–1228. doi:[10.1111/cns.13209](https://doi.org/10.1111/cns.13209) (2019).
19. Matias, C., Sharan, A. & Wu, C. Responsive neurostimulation for the treatment of epilepsy. *Neurosurgery clinics of north america* **30**, 231–42. doi:[10.1016/j.nec.2018.12.006](https://doi.org/10.1016/j.nec.2018.12.006) (2019).

Bibliographic references

20. Daoud, H. & Bayoumi, M. A. Efficient Epileptic Seizure Prediction Based on Deep Learning. *Ieee transactions on biomedical circuits and systems* **13**, 804–813. doi:[10.1109/TBCAS.2019.2929053](https://doi.org/10.1109/TBCAS.2019.2929053) (2019).
21. Gramacki, A. & Gramacki, J. A deep learning framework for epileptic seizure detection based on neonatal EEG signals. *Scientific reports* **12**, 13010. doi:[10.1038/s41598-022-15830-2](https://doi.org/10.1038/s41598-022-15830-2) (2022).
22. Farooq, M. S., Zulfiqar, A. & Riaz, S. Epileptic Seizure Detection Using Machine Learning: Taxonomy, Opportunities, and Challenges. *Diagnostics* **13**, 1058. doi:[10.3390/diagnostics13061058](https://doi.org/10.3390/diagnostics13061058) (2023).
23. Siddiqui, M. K., Morales-Menendez, R., Huang, X. & Hussain, N. A review of epileptic seizure detection using machine learning classifiers. *Brain informatics* **7**, 5. doi:[10.1186/s40708-020-00105-1](https://doi.org/10.1186/s40708-020-00105-1) (2020).
24. Abdelhameed, A. & Bayoumi, M. A Deep Learning Approach for Automatic Seizure Detection in Children With Epilepsy. *Frontiers in computational neuroscience* **15**, 650050. doi:[10.3389/fncom.2021.650050](https://doi.org/10.3389/fncom.2021.650050) (2021).
25. Almustafa, K. M. Classification of epileptic seizure dataset using different machine learning algorithms. *Informatics in medicine unlocked* **21**, 100444. doi:[10.1016/j.imu.2020.100444](https://doi.org/10.1016/j.imu.2020.100444) (2020).
26. Tsiouris, K., Tzallas, A., Markoula, S., Koutsouris, D., Konitsiotis, S. & Fotiadis, D. A review of automated methodologies for the detection of epileptic episodes using long-term eeg signals. *Healthcare pol reform concepts methodol tools appl* **3201**, 8 1464–96. doi:[10.4018/978-1-5225-6915-2.ch066](https://doi.org/10.4018/978-1-5225-6915-2.ch066).
27. Billeci, L., Marino, D., Insana, L., Vatti, G. & Varanini, M. Patient-specific seizure prediction based on heart rate variability and recurrence quantification analysis. *Plos one* **13** (ed Biagini, G.) e0204339. doi:[10.1371/journal.pone.0204339](https://doi.org/10.1371/journal.pone.0204339) (2018).
28. Zhang, A., Xing, L., Zou, J. & Wu, J. C. Shifting machine learning for healthcare from development to deployment and from models to data. *Nature biomedical engineering* **6**, 1330–1345. doi:[10.1038/s41551-022-00898-y](https://doi.org/10.1038/s41551-022-00898-y) (2022).

Bibliographic references

29. Kim, H.-S., Kim, D.-J. & Yoon, K.-H. Medical Big Data Is Not Yet Available: Why We Need Realism Rather than Exaggeration. *Endocrinology and metabolism* **34**, 349. doi:[10.3803/EnM.2019.34.4.349](https://doi.org/10.3803/EnM.2019.34.4.349) (2019).
30. Detti, P., De Lara, G. Z. M., Bruni, R., Pranzo, M., Sarnari, F. & Vatti, G. A Patient-Specific Approach for Short-Term Epileptic Seizures Prediction Through the Analysis of EEG Synchronization. *Ieee transactions on biomedical engineering* **66**, 1494–1504. doi:[10.1109/TBME.2018.2874716](https://doi.org/10.1109/TBME.2018.2874716) (2019).
31. Amengual-Gual, M., Sánchez Fernández, I. & Loddenkemper, T. Patterns of epileptic seizure occurrence. *Brain research* **1703**, 3–12. doi:[10.1016/j.brainres.2018.02.032](https://doi.org/10.1016/j.brainres.2018.02.032) (2019).
32. Kuhlmann, L., Lehnertz, K., Richardson, M., Schelter, B. & Zaveri, H. Seizure prediction - ready for a new era. *Nature reviews. neurology* **14**, 618–30. doi:[10.1038/s41582-018-0055-2](https://doi.org/10.1038/s41582-018-0055-2) (2018).
33. Sridevi, V., Ramasubba Reddy, M., Srinivasan, K., Radhakrishnan, K., Rathore, C. & Nayak, D. Improved patient-independent system for detection of electrical onset of seizures. *Journal of clinical neurophysiology : official publication of the american electroencephalographic society* **36**, 14–24. doi:[10.1097/WNP.0000000000000533](https://doi.org/10.1097/WNP.0000000000000533) (2019).
34. Leguia, M. G., Andrzejak, R. G., Rummel, C., Fan, J. M., Mirro, E. A., Tcheng, T. K., Rao, V. R. & Baud, M. O. Seizure Cycles in Focal Epilepsy. *Jama neurology* **78**, 454. doi:[10.1001/jamaneurol.2020.5370](https://doi.org/10.1001/jamaneurol.2020.5370) (2021).
35. Schroeder, G. M., Diehl, B., Chowdhury, F. A., Duncan, J. S., De Tisi, J., Trevelyan, A. J., Forsyth, R., Jackson, A., Taylor, P. N. & Wang, Y. Seizure pathways change on circadian and slower timescales in individual patients with focal epilepsy. *Proceedings of the national academy of sciences* **117**, 11048–11058. doi:[10.1073/pnas.1922084117](https://doi.org/10.1073/pnas.1922084117) (2020).
36. Sarmast, S. T., Abdullahi, A. M. & Jahan, N. Current Classification of Seizures and Epilepsies: Scope, Limitations and Recommendations for Future Action. *Cureus*. doi:[10.7759/cureus.10549](https://doi.org/10.7759/cureus.10549) (2020).
37. McWilliam, M. & Al Khalili, Y. in *StatPearls* (StatPearls Publishing, Treasure Island (FL), 2024).

Bibliographic references

38. Eseosa T. Ighodaro, Kushagra Maini, Kapil Arya & Sandeep Sharma. *Focal Onset Seizure* (StatPearls Publishing LLC, 2023).
39. Fisher, R. S., Cross, J. H., French, J. A., Higurashi, N., Hirsch, E., Jansen, F. E., Lagae, L., Moshé, S. L., Peltola, J., Roulet Perez, E., Scheffer, I. E. & Zuberi, S. M. Operational classification of seizure types by the International League Against Epilepsy: Position Paper of the ILAE Commission for Classification and Terminology. *Epilepsia* **58**, 522–530. doi:[10.1111/epi.13670](https://doi.org/10.1111/epi.13670) (2017).
40. Jang, H.-J., Kwon, M. J. & Cho, K.-O. Central Regulation of Micturition and Its Association With Epilepsy. *International neurourology journal* **22**, 2–8. doi:[10.5213/inj.1836040.020](https://doi.org/10.5213/inj.1836040.020) (2018).
41. Oke, O., Johnson, B. O., Chida, S., Kekere, V. & Azim, S. Psychosis as a Manifestation of Focal Impaired Awareness Seizure in a Geriatric Patient. *Cureus*. doi:[10.7759/cureus.41838](https://doi.org/10.7759/cureus.41838) (2023).
42. Kumar, A. & Sharma, S. *Focal Impaired Awareness Seizure* (StatPearls Publishing, Treasure Island (FL), 2023).
43. De Curtis, M. & Avoli, M. Initiation, Propagation, and Termination of Partial (Focal) Seizures. *Cold spring harbor perspectives in medicine* **5**, a022368. doi:[10.1101/cshperspect.a022368](https://doi.org/10.1101/cshperspect.a022368) (2015).
44. MacDonald, D. B., Simon, M. V. & Nuwer, M. R. in *Handbook of Clinical Neurology* 103–121 (Elsevier, 2022). doi:[10.1016/B978-0-12-819826-1.00017-X](https://doi.org/10.1016/B978-0-12-819826-1.00017-X).
45. Burns, S. P., Santaniello, S., Yaffe, R. B., Jouny, C. C., Crone, N. E., Bergey, G. K., Anderson, W. S. & Sarma, S. V. Network dynamics of the brain and influence of the epileptic seizure onset zone. *Proceedings of the national academy of sciences* **111**. doi:[10.1073/pnas.1401752111](https://doi.org/10.1073/pnas.1401752111) (2014).
46. Wendling, F., Chauvel, P., Biraben, A. & Bartolomei, F. From Intracerebral EEG Signals to Brain Connectivity: Identification of Epileptogenic Networks in Partial Epilepsy. *Frontiers in systems neuroscience* **4**. doi:[10.3389/fnsys.2010.00154](https://doi.org/10.3389/fnsys.2010.00154) (2010).
47. Kramer, M. A. & Cash, S. S. Epilepsy as a Disorder of Cortical Network Organization. *The neuroscientist* **18**, 360–372. doi:[10.1177/1073858411422754](https://doi.org/10.1177/1073858411422754) (2012).

Bibliographic references

48. Bromfield EB, Cavazos JE & Sirven JI. *An Introduction to Epilepsy* (American Epilepsy Society, West Hartford (CT), 2006).
49. Jiruska, P., Curtis, M., Jefferys, J., Schevon, C., Schiff, S. & Schindler, K. Synchronization and desynchronization in epilepsy: controversies and hypotheses. *The journal of physiology* **591**, 787–97. doi:[10.1113/jphysiol.2012.239590](https://doi.org/10.1113/jphysiol.2012.239590) (2013).
50. Nayak CS & Anilkumar AC. *EEG Normal Waveforms* (StatPearls Publishing, Treasure Island (FL), 2024).
51. Wachsmuth, L., Hebbelmann, L., Prade, J., Kohnert, L. C., Lambers, H., Lüttjohann, A., Budde, T., Hess, A. & Faber, C. Epilepsy-related functional brain network alterations are already present at an early age in the GAERS rat model of genetic absence epilepsy. *Frontiers in neurology* **15**, 1355862. doi:[10.3389/fneur.2024.1355862](https://doi.org/10.3389/fneur.2024.1355862) (2024).
52. Gonzalez-Escamilla, G., Muthuraman, M., Chirumamilla, V. C., Vogt, J. & Groppa, S. Brain Networks Reorganization During Maturation and Healthy Aging-Emphases for Resilience. *Frontiers in psychiatry* **9**, 601. doi:[10.3389/fpsyt.2018.00601](https://doi.org/10.3389/fpsyt.2018.00601) (2018).
53. Wu, Z. & Sabel, B. A. Spacetime in the brain: rapid brain network reorganization in visual processing and recovery. *Scientific reports* **11**, 17940. doi:[10.1038/s41598-021-96971-8](https://doi.org/10.1038/s41598-021-96971-8) (2021).
54. Schaper, F. L. W. V. J., Nordberg, J., Cohen, A. L., Lin, C., Hsu, J., Horn, A., Ferguson, M. A., Siddiqi, S. H., Drew, W., Soussand, L., Winkler, A. M., Simó, M., Bruna, J., Rheims, S., Guenot, M., Bucci, M., Nummenmaa, L., Staals, J., Colon, A. J., Ackermans, L., Bubrick, E. J., Peters, J. M., Wu, O., Rost, N. S., Grafman, J., Blumenfeld, H., Temel, Y., Rouhl, R. P. W., Joutsa, J. & Fox, M. D. Mapping Lesion-Related Epilepsy to a Human Brain Network. *Jama neurology* **80**, 891. doi:[10.1001/jamaneurol.2023.1988](https://doi.org/10.1001/jamaneurol.2023.1988) (2023).
55. Laiou, P., Biondi, A., Bruno, E., Viana, P., Winston, J., Rashid, Z., Ranjan, Y., Conde, P., Stewart, C., Sun, S., Zhang, Y., Folarin, A., Dobson, R., Schulze-Bonhage, A., Dümpelmann, M., Richardson, M. & RADAR-CNS Consortium. Temporal Evolution of Multiday, Epileptic Functional Networks Prior to Seizure Occurrence. *Biomedicines* **10**, 2662. doi:[10.3390/biomedicines10102662](https://doi.org/10.3390/biomedicines10102662) (2022).

Bibliographic references

56. Karoly, P. J., Freestone, D. R., Eden, D., Stirling, R. E., Li, L., Vianna, P. F., Maturana, M. I., D’Souza, W. J., Cook, M. J., Richardson, M. P., Brinkmann, B. H. & Nurse, E. S. Epileptic Seizure Cycles: Six Common Clinical Misconceptions. *Frontiers in neurology* **12**, 720328. doi:[10.3389/fneur.2021.720328](https://doi.org/10.3389/fneur.2021.720328) (2021).
57. Karoly, P., Goldenholz, D., Freestone, D., Moss, R., Grayden, D. & Theodore, W. Circadian and circaseptan rhythms in human epilepsy: a retrospective cohort study. *The lancet. neurology* **17**, 977–85. doi:[10.1016/S1474-4422\(18\)30274-6](https://doi.org/10.1016/S1474-4422(18)30274-6) (2018).
58. Meyer, T., Shultz, C., Dehak, N., Moro-Velazquez, L. & Irazoqui, P. *Time Scale Network: A Shallow Neural Network For Time Series Data* 2023. arXiv: [2311.06170](https://arxiv.org/abs/2311.06170) [cs].
59. Meyer, T. & Shultz, C. *Toast* Center for Implantable Devices. 2022.
60. Moody, G. B. & Mark, R. G. *MIT-BIH Arrhythmia Database* 1992. doi:[10.13026/C2F305](https://doi.org/10.13026/C2F305).
61. Freestone, D. R., Karoly, P. J. & Cook, M. J. A forward-looking review of seizure prediction. *Current opinion in neurology* **30**, 167–173. doi:[10.1097/WCO.0000000000000429](https://doi.org/10.1097/WCO.0000000000000429) (2017).
62. Eck, B., Kabakci-Zorlu, D., Chen, Y., Savard, F. & Bao, X. *A monitoring framework for deployed machine learning models with supply chain examples* 2022. arXiv: [2211.06239](https://arxiv.org/abs/2211.06239) [cs, stat].
63. Zhang, H., Dullerud, N., Seyyed-Kalantari, L., Morris, Q., Joshi, S. & Ghassemi, M. *An empirical framework for domain generalization in clinical settings* in *Proceedings of the Conference on Health, Inference, and Learning* (ACM, Virtual Event USA, 2021), 279–290. doi:[10.1145/3450439.3451878](https://doi.org/10.1145/3450439.3451878).
64. Gulrajani, I. & Lopez-Paz, D. IN SEARCH OF LOST DOMAIN GENERALIZATION (2021).
65. Koh, P. W., Sagawa, S., Marklund, H., Xie, S. M., Zhang, M., Balsubramani, A., Hu, W., Yasunaga, M., Phillips, R. L., Gao, I., Lee, T., David, E., Stavness, I., Guo, W., Earnshaw, B. A., Haque, I. S., Beery, S., Leskovec, J., Kundaje, A., Pierson, E., Levine, S., Finn, C. & Liang, P. *WILDS: A Benchmark of*

Bibliographic references

- in-the-Wild Distribution Shifts* 2021. doi:[10.48550/arXiv.2012.07421](https://doi.org/10.48550/arXiv.2012.07421). arXiv: [2012.07421](https://arxiv.org/abs/2012.07421) [cs].
66. Yang, Y., Zhang, H., Katabi, D. & Ghassemi, M. *Change is Hard: A Closer Look at Subpopulation Shift* 2023. arXiv: [2302.12254](https://arxiv.org/abs/2302.12254) [cs].
67. Dragoi, M., Burceanu, E., Haller, E., Manolache, A. & Brad, F. AnoShift: A Distribution Shift Benchmark for Unsupervised Anomaly Detection.
68. Shoeb, A. H. *Application of machine learning to epileptic seizure onset detection and treatment* Thesis (Massachusetts Institute of Technology, 2009).
69. Lee, K., Jeong, H., Kim, S., Yang, D., Kang, H.-C. & Choi, E. *Real-Time Seizure Detection using EEG: A Comprehensive Comparison of Recent Approaches under a Realistic Setting* in *Proceedings of the Conference on Health, Inference, and Learning* (PMLR, 2022), 311–337.
70. Obeid, I. & Picone, J. The Temple University Hospital EEG Data Corpus. *Frontiers in neuroscience* **10**. doi:[10.3389/fnins.2016.00196](https://doi.org/10.3389/fnins.2016.00196) (2016).
71. Wong, E. *Distribution shift* 2022.
72. Zhao, S., Sinha, A., He, Y., Perreault, A., Song, J. & Ermon, S. COMPARING DISTRIBUTIONS BY MEASURING DIFFERENCES THAT AFFECT DECISION MAKING (2022).
73. Chen, I. Y., Pierson, E., Rose, S., Joshi, S., Ferryman, K. & Ghassemi, M. Ethical Machine Learning in Healthcare. *Annual review of biomedical data science* **4**, 123–144. doi:[10.1146/annurev-biodatasci-092820-114757](https://doi.org/10.1146/annurev-biodatasci-092820-114757) (2021).
74. Subbaswamy, A., Adams, R. & Saria, S. *Evaluating Model Robustness and Stability to Dataset Shift* in *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics* (PMLR, 2021), 2611–2619.
75. Futoma, J., Simons, M., Panch, T., Doshi-Velez, F. & Celi, L. A. The myth of generalisability in clinical research and machine learning in health care. *The lancet. digital health* **2**, e489–e492. doi:[10.1016/S2589-7500\(20\)30186-2](https://doi.org/10.1016/S2589-7500(20)30186-2) (2020).
76. Wong, H. Y. F., Lam, H. Y. S., Fong, A. H.-T., Leung, S. T., Chin, T. W.-Y., Lo, C. S. Y., Lui, M. M.-S., Lee, J. C. Y., Chiu, K. W.-H., Chung, T. W.-H.,

Bibliographic references

- Lee, E. Y. P., Wan, E. Y. F., Hung, I. F. N., Lam, T. P. W., Kuo, M. D. & Ng, M.-Y. Frequency and Distribution of Chest Radiographic Findings in Patients Positive for COVID-19. *Radiology* **296**, E72–E78. doi:[10.1148/radiol.2020201160](https://doi.org/10.1148/radiol.2020201160) (2020).
77. Beyene, A. A., Welemariam, T., Persson, M. & Lavesson, N. Improved concept drift handling in surgery prediction and other applications. *Knowledge and information systems* **44**, 177–196. doi:[10.1007/s10115-014-0756-9](https://doi.org/10.1007/s10115-014-0756-9) (2015).
78. Gretton, A., Borgwardt, K. M., Rasch, M. J., Schölkopf, B. & Smola, A. A Kernel Two-Sample Test. *Journal of machine learning research* **13**, 723–773 (2012).
79. Kolouri, S., Park, S., Thorpe, M., Slepčev, D. & Rohde, G. K. Optimal Mass Transport: Signal processing and machine-learning applications. *Ieee signal processing magazine* **34**, 43–59. doi:[10.1109/MSP.2017.2695801](https://doi.org/10.1109/MSP.2017.2695801) (2017).
80. Wang, J., Chen, M., Zhao, T., Liao, W. & Xie, Y. A manifold two-sample test study: integral probability metric with neural networks. *Information and inference: a journal of the ima* **12**, 1867–1897. doi:[10.1093/imaiai/iaad018](https://doi.org/10.1093/imaiai/iaad018) (2023).
81. Birrell, J., Dupuis, P., Katsoulakis, M. A., Pantazis, Y. & Rey-Bellet, L. (f, gamma)-Divergences: interpolating between f-divergences and integral probability metrics. *The journal of machine learning research* **23**, 39:1816–39:1885 (2022).
82. Sugiyama, M., Suzuki, T. & Kanamori, T. *Density Ratio Estimation: A Comprehensive Review* in (2010).
83. Justel, A., Peña, D. & Zamar, R. A multivariate Kolmogorov-Smirnov test of goodness of fit. *Statistics & probability letters* **35**, 251–259. doi:[10.1016/S0167-7152\(97\)00020-5](https://doi.org/10.1016/S0167-7152(97)00020-5) (1997).
84. Villani, C. *Optimal transport: old and new Grundlehren Der Mathematischen Wissenschaften* **338** (Springer, Berlin, 2009).
85. Villani, C. *Topics in optimal transportation* Reprinted with corrections. *Graduate Studies in Mathematics* **58** (American Mathematical Society, Providence, Rhode Island, 2016).

Bibliographic references

86. Peyré, G. & Cuturi, M. *Computational Optimal Transport* 2020. arXiv: [1803.00567 \[stat\]](#).
87. Alvarez-Melis, D. & Fusi, N. *Geometric Dataset Distances via Optimal Transport* 2020. doi:[10.48550/arXiv.2002.02923](#). arXiv: [2002.02923 \[cs, stat\]](#).
88. Lu, Y., Qin, Y., Zhai, R., Shen, A., Chen, K., Wang, Z., Kolouri, S., Stepputtis, S., Campbell, J. & Sycara, K. *Characterizing Out-of-Distribution Error via Optimal Transport* 2023. arXiv: [2305.15640 \[cs\]](#).
89. del Barrio, E., Cuesta-Albertos, J. A., Matran, C. & Rodriguez-Rodriguez, J. M. Tests of Goodness of Fit Based on the L2-Wasserstein Distance. *The annals of statistics* **27**, 1230–1239. JSTOR: [120161](#) (1999).
90. Ramdas, A., Garcia, N. & Cuturi, M. *On Wasserstein Two Sample Testing and Related Families of Nonparametric Tests* 2015. doi:[10.48550/arXiv.1509.02237](#). arXiv: [1509.02237 \[math, stat\]](#).
91. Kantorovich, L. V. On the Translocation of Masses. *Journal of mathematical sciences* **133**, 1381–1382. doi:[10.1007/s10958-006-0049-2](#) (2006).
92. Rubner, Y., Tomasi, C. & Guibas, L. The Earth Mover’s Distance as a Metric for Image Retrieval. *International journal of computer vision* **40**, 99–121. doi:[10.1023/A:1026543900054](#) (2000).
93. William P. Nobis. *Vanderbilt University Medical Center EEG Database* 2023.
94. Shoeb, A. & Guttag, J. Application of Machine Learning To Epileptic Seizure Detection.
95. van der Maaten, L. & Hinton, G. Visualizing data using t-SNE. *Journal of machine learning research* **9**, 2579–2605 (2008).
96. Shyu, M.-L., Chen, S.-C., Sarinnapakorn, K. & Chang, L. *A Novel Anomaly Detection Scheme Based on Principal Component Classifier in Proceedings of International Conference on Data Mining* (2003).
97. Salekshahrezaee, Z., Leevy, J. L. & Khoshgoftaar, T. M. A reconstruction error-based framework for label noise detection. *Journal of big data* **8**, 57. doi:[10.1186/s40537-021-00447-5](#) (2021).

Bibliographic references

98. Wold, S., Esbensen, K. & Geladi, P. Principal component analysis. *Chemometrics and intelligent laboratory systems. Proceedings of the Multivariate Statistical Workshop for Geologists and Geochemists* **2**, 37–52. doi:[10.1016/0169-7439\(87\)80084-9](https://doi.org/10.1016/0169-7439(87)80084-9) (1987).
99. Wiskott, L. Principal Component Analysis.
100. Takeishi, N. *Shapley Values of Reconstruction Errors of PCA for Explaining Anomaly Detection* 2020. doi:[10.48550/arXiv.1909.03495](https://doi.org/10.48550/arXiv.1909.03495). arXiv: [1909.03495](https://arxiv.org/abs/1909.03495) [cs, stat].
101. Tr, T. Dimensionality Reduction: A Comparative Review (2009).
102. Rabanser, S., Günnemann, S. & Lipton, Z. C. *Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift* 2019. doi:[10.48550/arXiv.1810.11953](https://doi.org/10.48550/arXiv.1810.11953). arXiv: [1810.11953](https://arxiv.org/abs/1810.11953) [cs, stat].
103. Mormann, F., Kreuz, T., Rieke, C., Andrzejak, R. G., Kraskov, A., David, P., Elger, C. E. & Lehnertz, K. On the predictability of epileptic seizures. *Clinical neurophysiology* **116**, 569–587. doi:[10.1016/j.clinph.2004.08.025](https://doi.org/10.1016/j.clinph.2004.08.025) (2005).
104. Lehnertz, K. Epilepsy and Nonlinear Dynamics. *Journal of biological physics* **34**, 253–266. doi:[10.1007/s10867-008-9090-3](https://doi.org/10.1007/s10867-008-9090-3) (2008).
105. Shoeb, A., Edwards, H., Connolly, J., Bourgeois, B., Ted Treves, S. & Gutttag, J. Patient-specific seizure onset detection. *Epilepsy & behavior* **5**, 483–498. doi:[10.1016/j.yebeh.2004.05.005](https://doi.org/10.1016/j.yebeh.2004.05.005) (2004).

Appendix A

Supplementary Material

A.1 Datasets and Training

A.1.1 EEG Channels

From the multi-channel EEG data contained in the publicly available CHB-MIT dataset, we extracted the same 16 channels from each subject as this was the largest power-of-two value present in all subjects, allowing us to use the same architecture for everyone. We selected the following channels, as together they uniformly sample all areas of the skull:

1. FP1-F7
2. FP1-F3
3. FP2-F4
4. FP2-F8
5. F7-T7
6. F3-C3
7. F4-C4

8. F8-T8
9. T7-P7
10. C3-P3
11. C4-P4
12. T8-P8
13. P7-O1
14. P3-O1
15. P4-O2
16. P8-O2

A.1.2 Optimization and Training Details

For the deflation experiments, we use a series of simple CNNs, CNN and LSTM backbones, and Time-Scale Networks (TiSc) with ReLU activations. The CNN networks are trained for 300 epochs using ADAM with a subject-specific learning rate ($[1e^{-5}, 5e^{-5}, 5e^{-6}, 5e^{-7}]$) and weight decay of 0. Similarly, TiSc networks are trained for 200 epochs using rmsprop with a similar subject-specific learning rate and weight decay of 0.05. The best hyperparameters are selected for each subject and split pair using 10-Fold cross-validation. Reported errors are ± 1 s.d. over three repetitions.

Dataset distances are computed for each split and subject pair over a maximum randomly selected subset of 15,000 samples. Reported errors are ± 1 s.d. over three repetitions. The inner optimal transport distance, as described in [87], are computed with the default parameters as an exact Wasserstein problem using the debiased Sinkhorn algorithm with an entropy regularization parameter of $1e-1$.

A.2 Additional Results

A.2.1 Vanderbilt University Medical Center (Some)

In Figure A.1 we provide the distribution of accuracy, F1 Score, ROC-AUC, and PR-AUC for subjects in which there exists both classes in the temporal training and testing splits.

A.2.2 CHB-MIT (Some)

In Figure A.2 we provide the distribution of accuracy, F1 Score, ROC-AUC, and PR-AUC for subjects in which there exists both classes in the temporal training and testing splits.

A.2.3 Vanderbilt University Medical Center (VUMC) Dataset Figures (25 Per)

In Figure A.3 we provide the distribution of accuracy, F1 Score, ROC-AUC, and PR-AUC for subjects in which there classes in the temporal training and testing are composed at least of 25% of each class.

A.2.4 CHB-MIT (25 Per)

In Figure A.4 we provide the distribution of accuracy, F1 Score, ROC-AUC, and PR-AUC for subjects in which there classes in the temporal training and testing are composed at least of 25% of each class.

A.2.5 Vanderbilt University Medical Center (VUMC) Dataset Figures (All)

In Figure A.5 we provide the distribution of accuracy, F1 Score, ROC-AUC, and PR-AUC for all subjects in VUMC.

A.2.6 CHB-MIT (all)

In Figure A.6 we provide the distribution of accuracy, F1 Score, ROC-AUC, and PR-AUC for all subjects in CHB-MIT.

Case	Gender	Age (years)
chb01	F	11
chb02	M	11
chb03	F	14
chb04	M	22
chb05	F	7
chb06	F	1.5
chb07	F	14.5
chb08	M	3.5
chb09	F	10
chb10	M	3
chb11	F	12
chb12	F	2
chb13	F	3
chb14	F	9
chb15	M	16
chb16	F	7
chb17	F	12
chb18	F	18
chb19	F	19
chb20	F	6
chb21	F	13
chb22	F	9
chb23	F	6

Table A.1: CHBMIT Subject Info, Shoeb, Edwards, Connolly, Bourgeois, Ted Treves & Guttag [105]

Appendix A. Supplementary Material

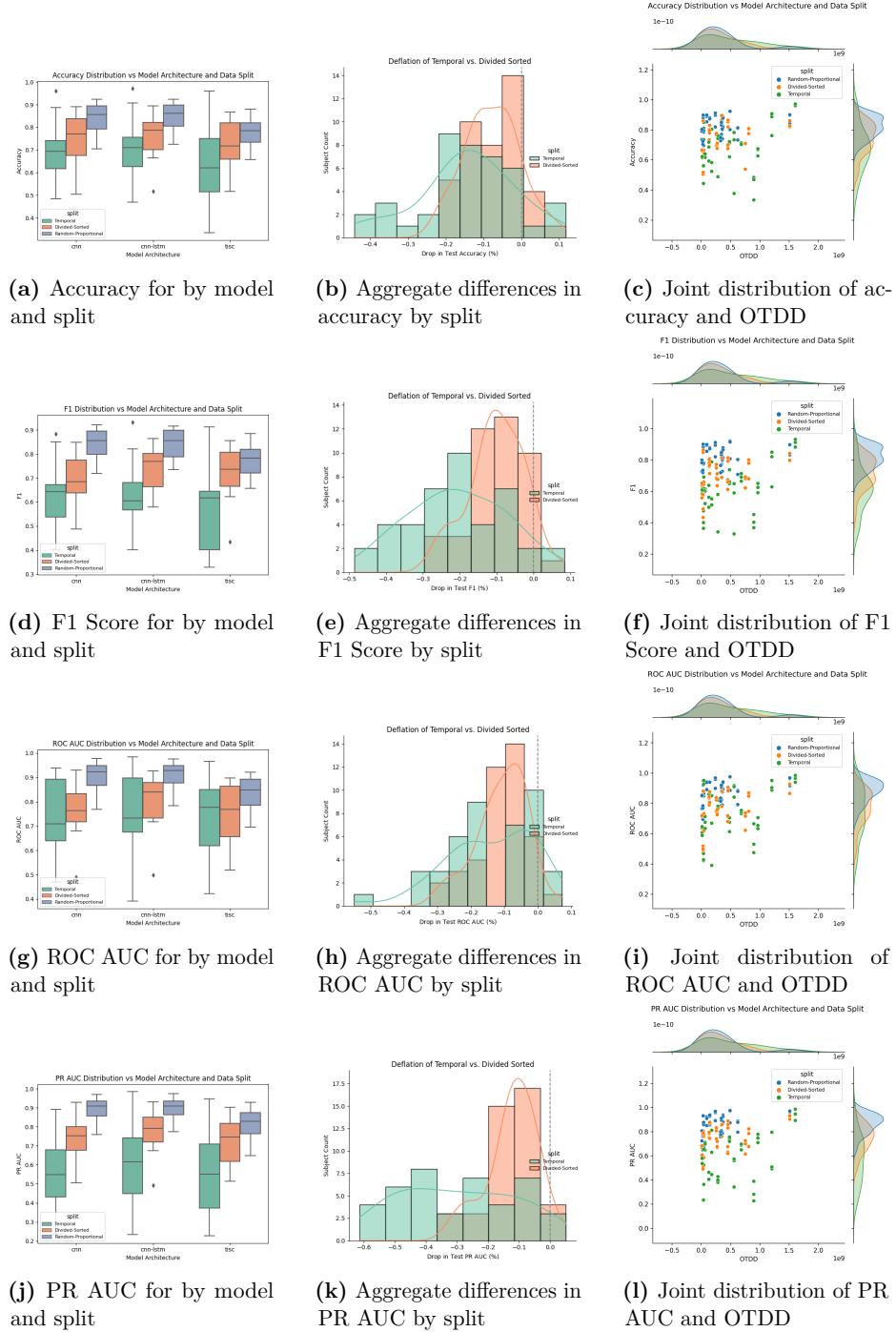


Figure A.1: Vanderbilt University Medical Center (VUMC) Dataset for subjects that contain both interictal and pre-ictal segments in both training and testing sets.

Appendix A. Supplementary Material

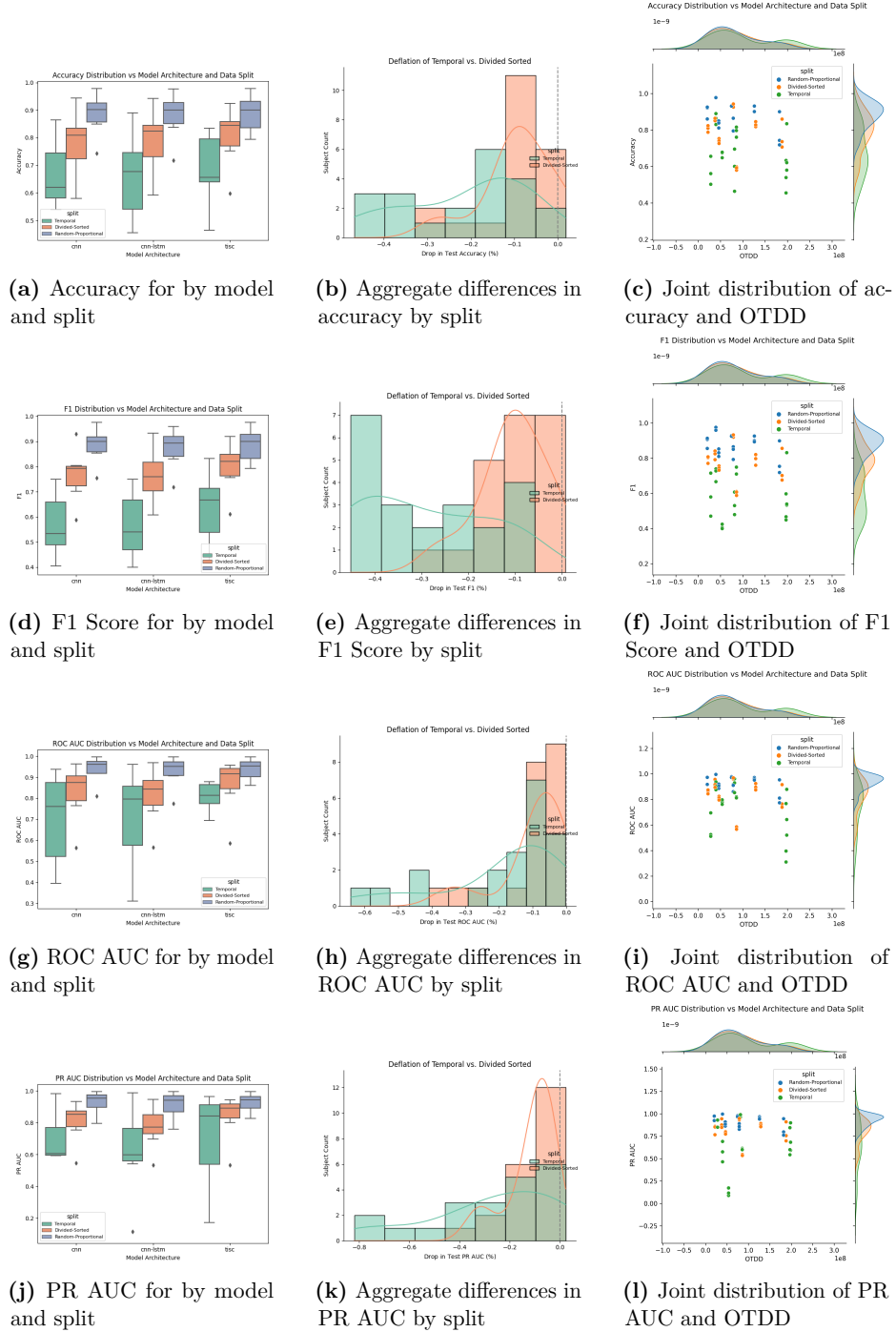


Figure A.2: CHB-MIT Dataset for subjects that contain both interictal and pre-ictal segments in both training and testing sets.

Appendix A. Supplementary Material

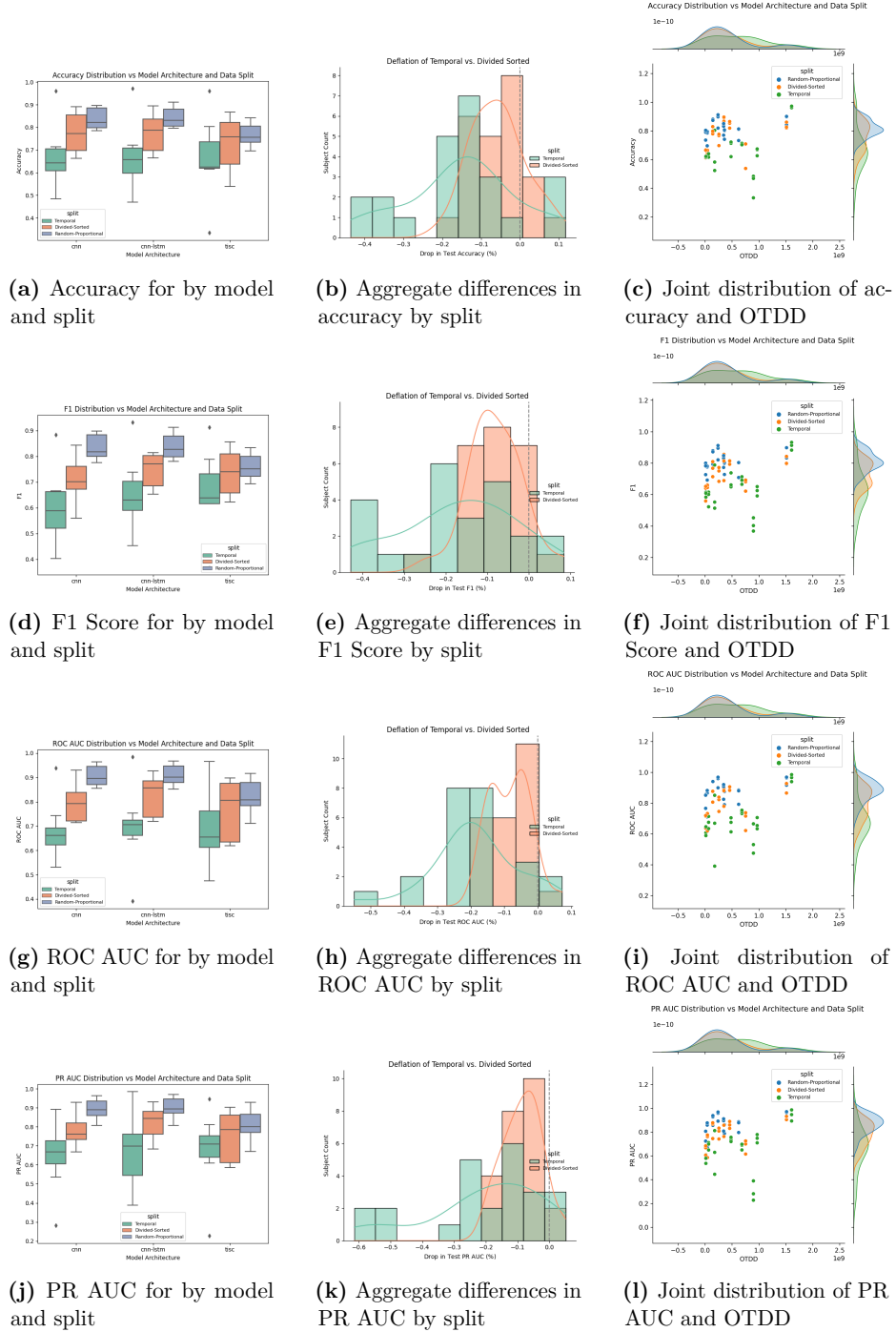


Figure A.3: Vanderbilt University Medical Center (VUMC) Dataset in which both the interictal and baseline data comprise at least 25%.

Appendix A. Supplementary Material

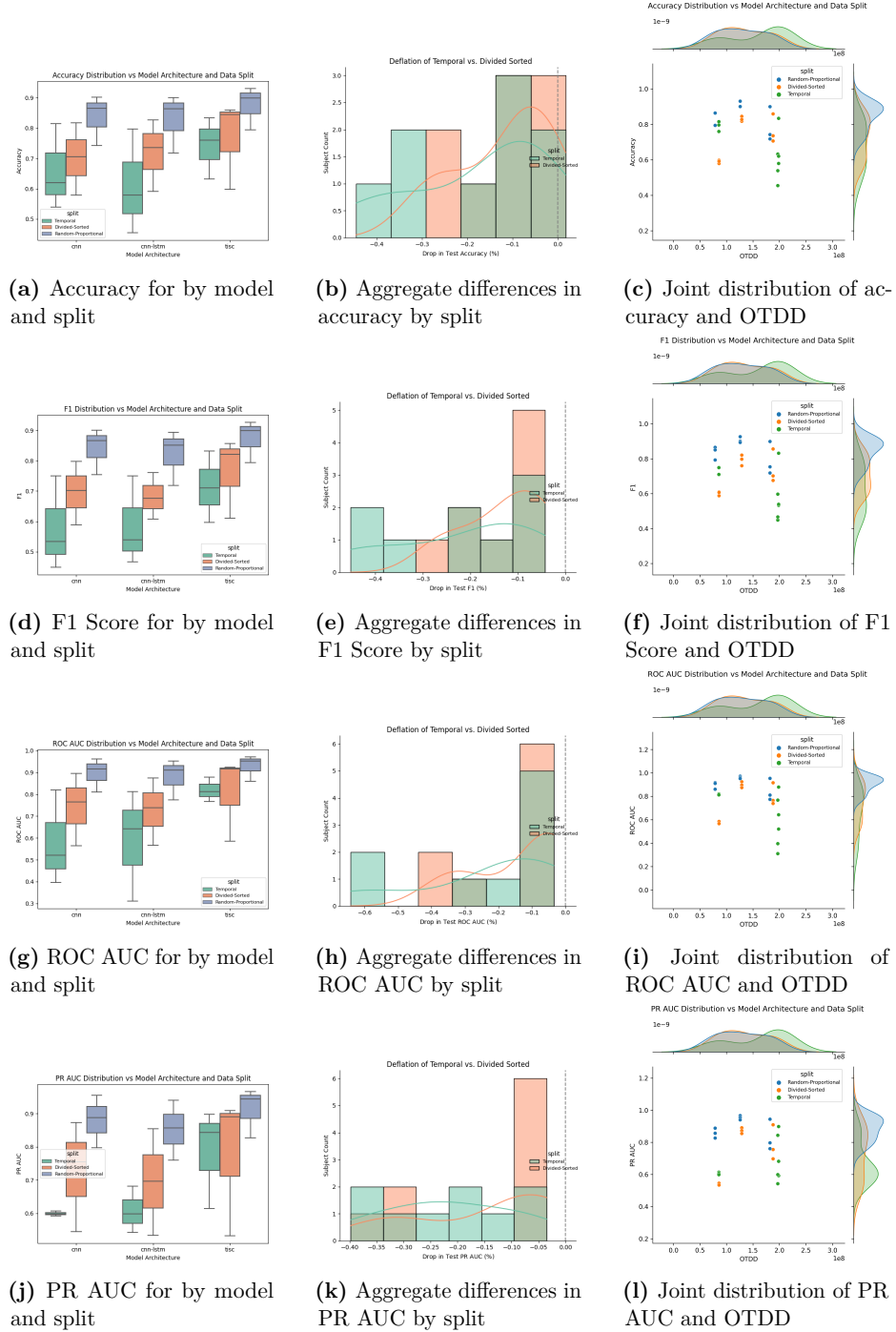


Figure A.4: CHB-MIT Dataset for subjects in which both the interictal and baseline data comprise at least 25%.

Appendix A. Supplementary Material

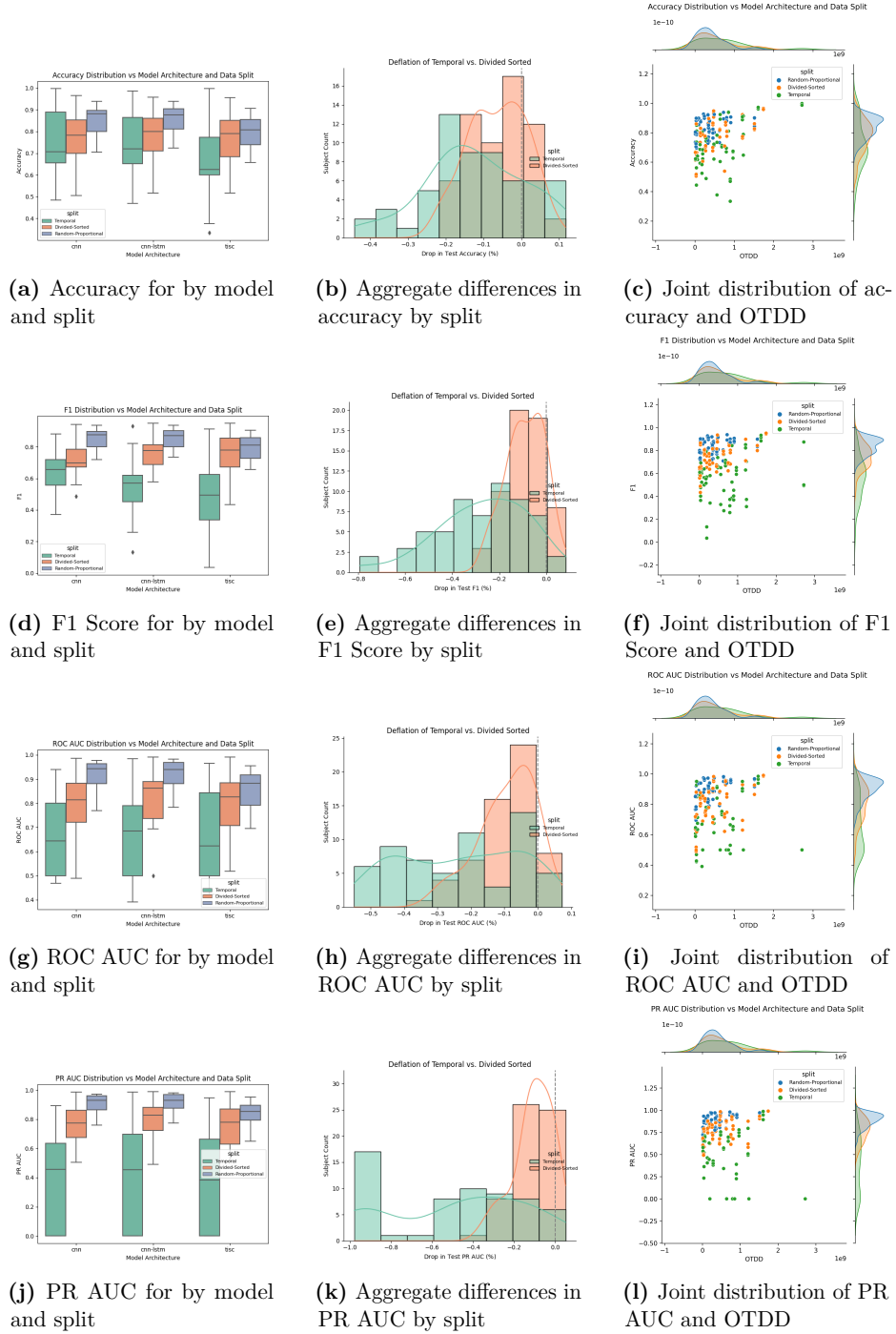


Figure A.5: Vanderbilt University Medical Center (VUMC) Dataset for all subjects.

Appendix A. Supplementary Material

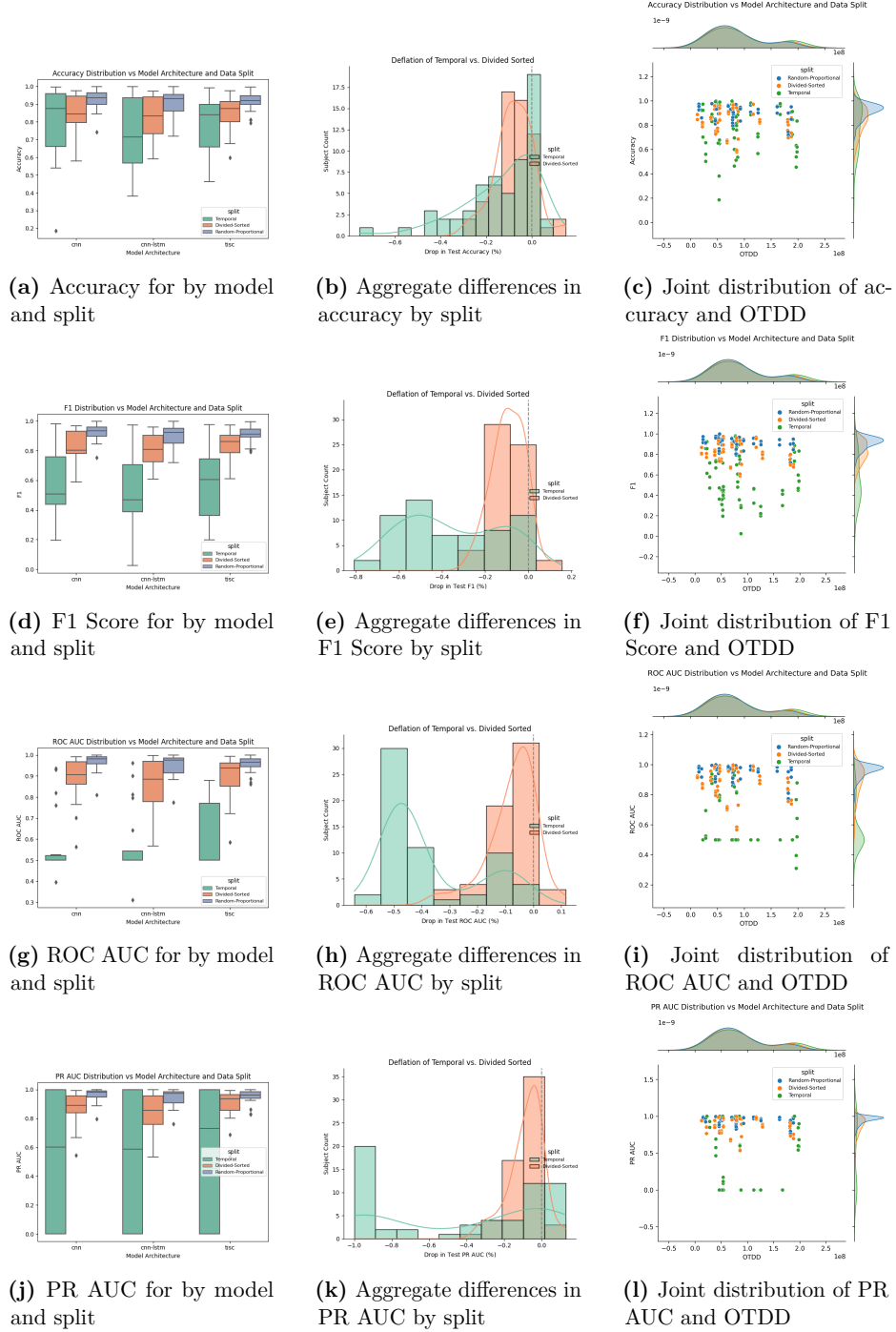


Figure A.6: CHB-MIT Dataset for all subjects.

A.3 Robustness of Optimal Transport Distance to Number of Samples

In Figure A.7 we compute the distance repeatedly for varying sized subsets of the training and test distributions, over 10 repetitions for the VUMC dataset as in [87]. As the number of samples increase (typically approaching the total number of samples in the test split), the values converge to a single value.

A.4 Optimal Transport Drift Detection Examples

In Figures A.8, A.9, A.10, and A.11 we provide additional drift detection examples using the `NannyML` package and a custom tracking module developed to use the OTDD distance.

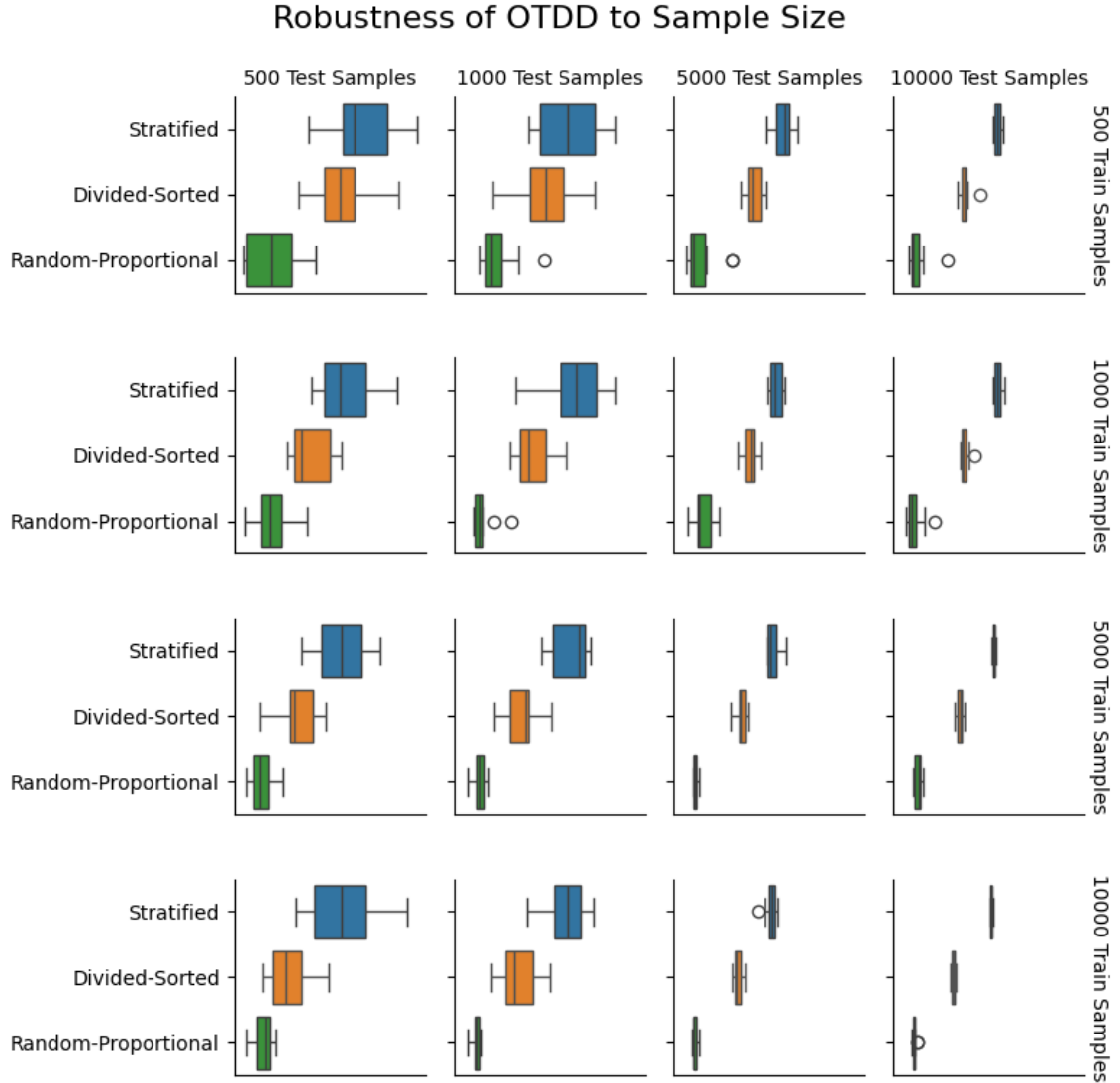


Figure A.7: Sample Robustness: Distances between the training and test distributions are computed for varying sized subsets of the compared distributions over 10 repetitions. Example for a randomly selected subject in the VUMC dataset. Patterns are consistent across subjects in CHB-MIT and VUMC.

Appendix A. Supplementary Material

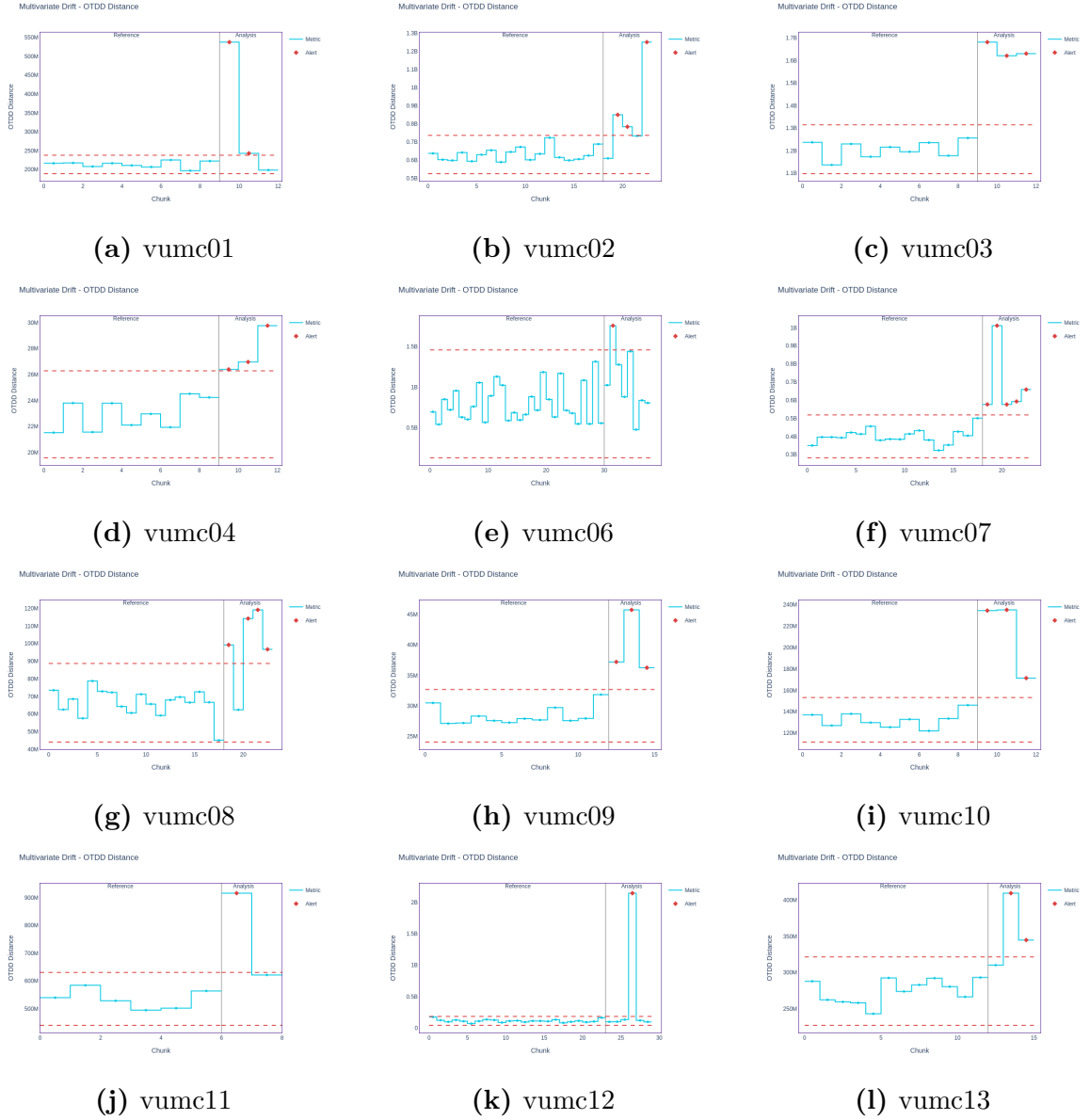


Figure A.8: Optimal Transport Drift Detection: Random Split for subjects in the VUMC dataset. 10,000 reference samples, 500-sampled test chunks.

Appendix A. Supplementary Material

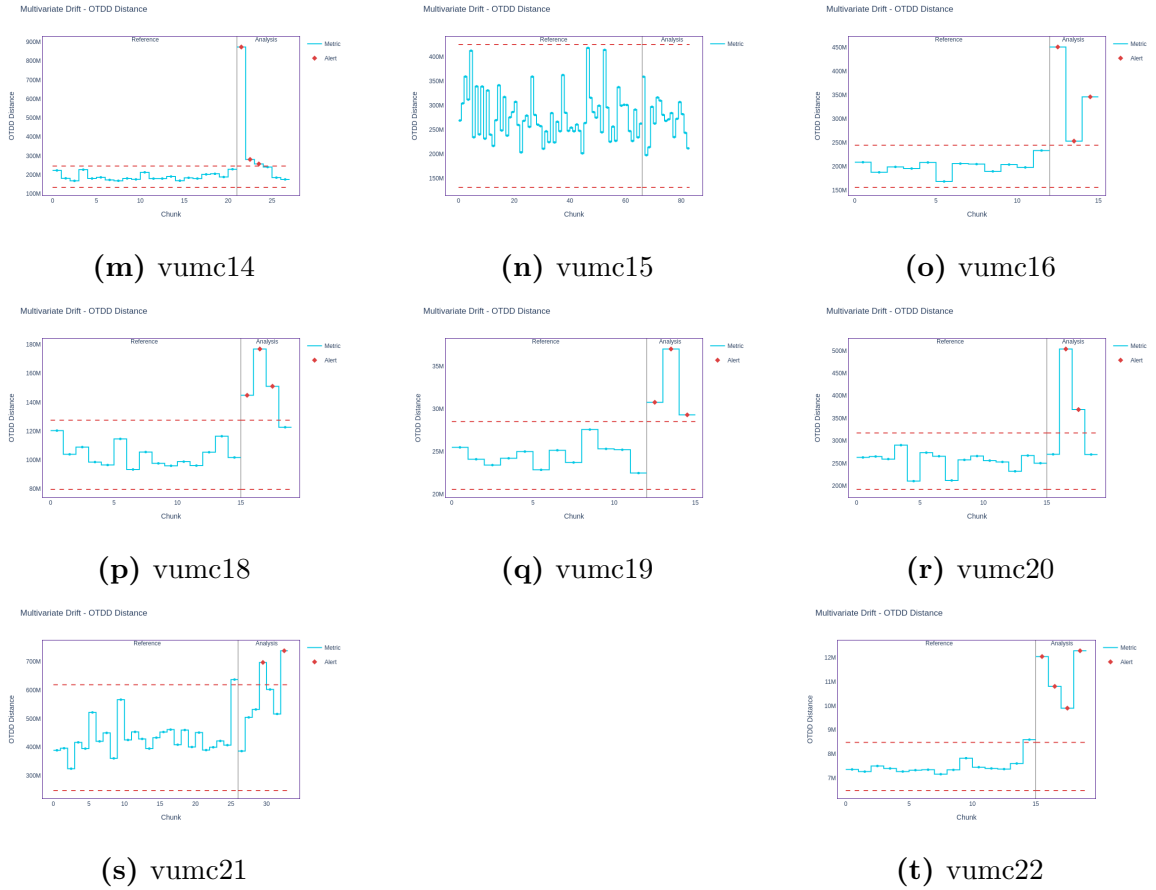


Figure A.8: Optimal Transport Drift Detection: Random Split for subjects in the VUMC dataset. 10,000 reference samples, 500-sampled test chunks.

Appendix A. Supplementary Material

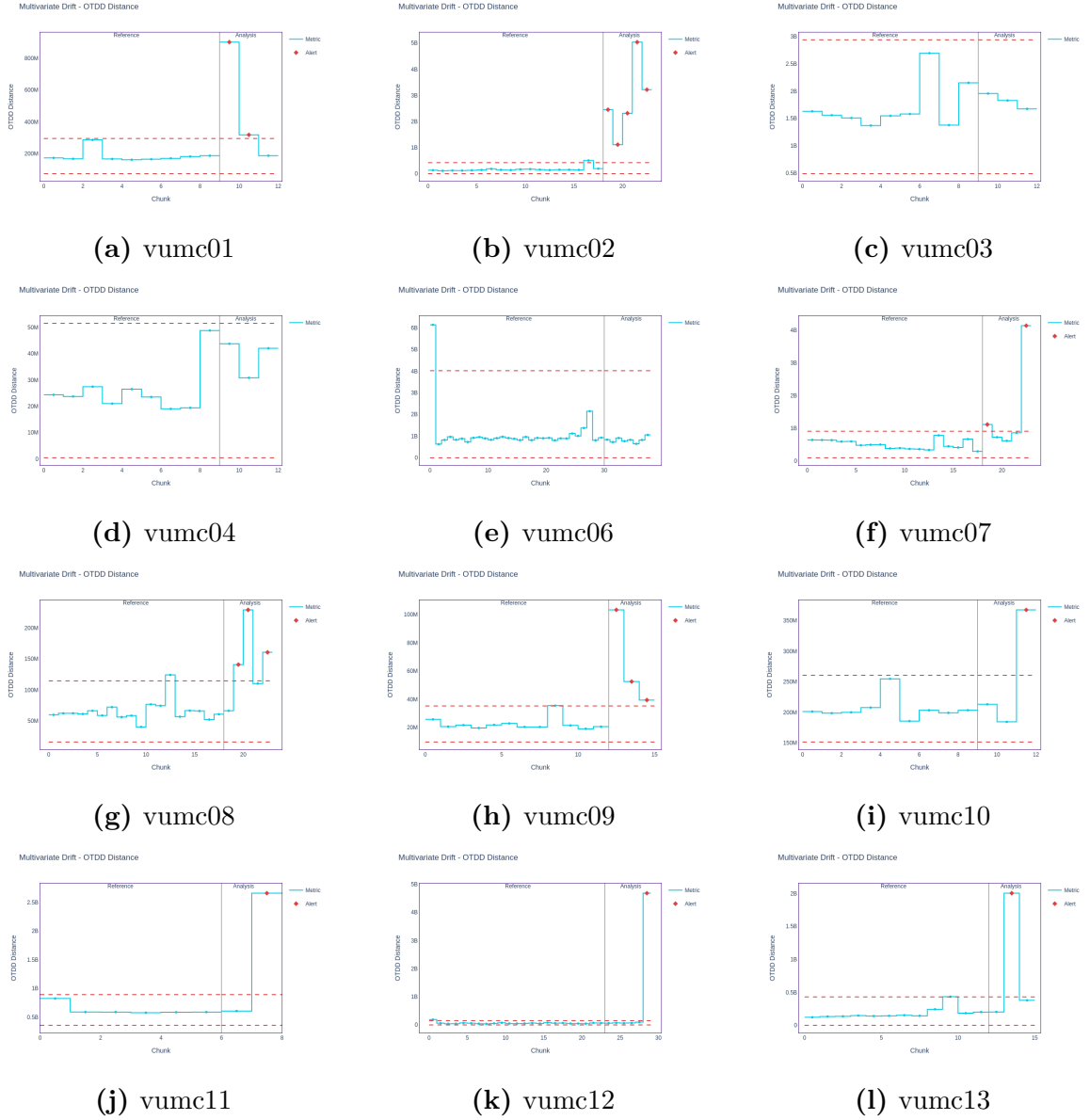


Figure A.9: Optimal Transport Drift Detection: Temporal Split for subjects in the VUMC dataset. 10,000 reference samples, 500-sampled test chunks.

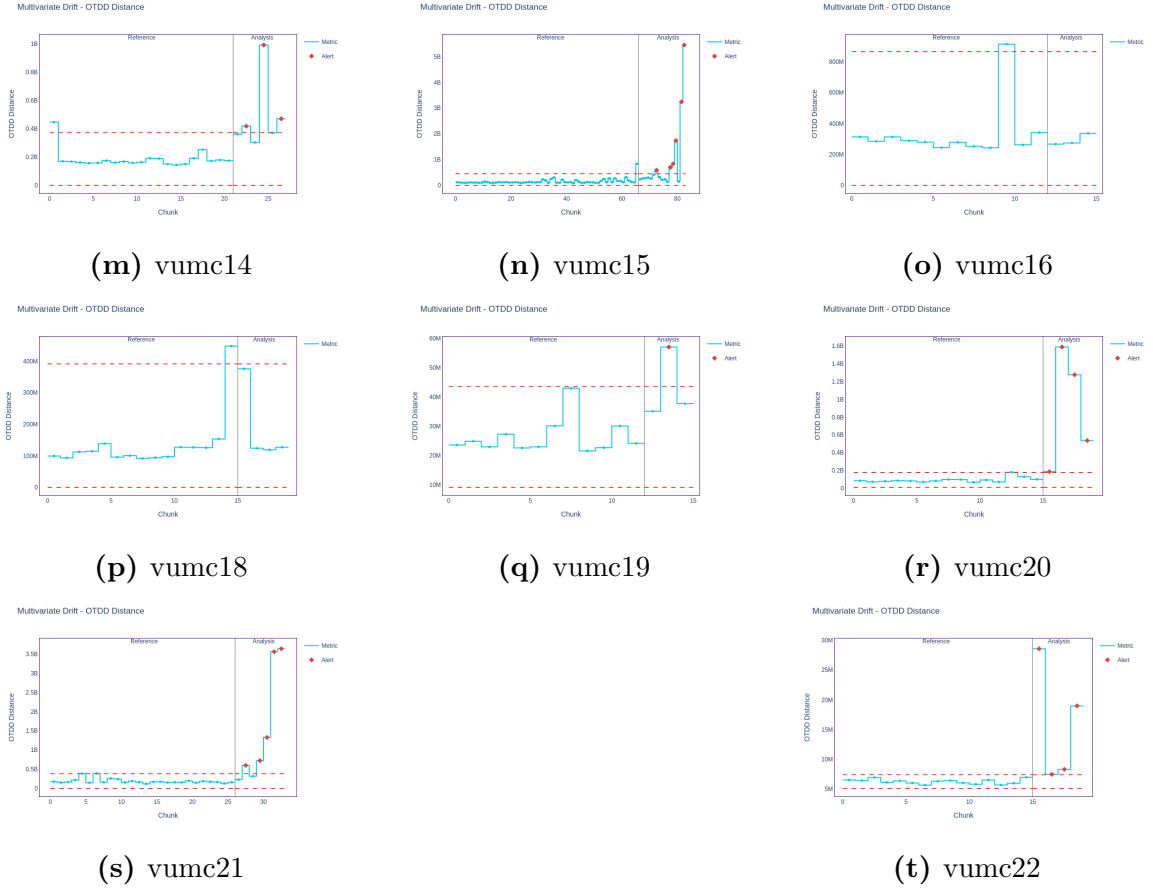


Figure A.9: Optimal Transport Drift Detection: Temporal Split for subjects in the VUMC dataset. 10,000 reference samples, 500-sampled test chunks.

Appendix A. Supplementary Material

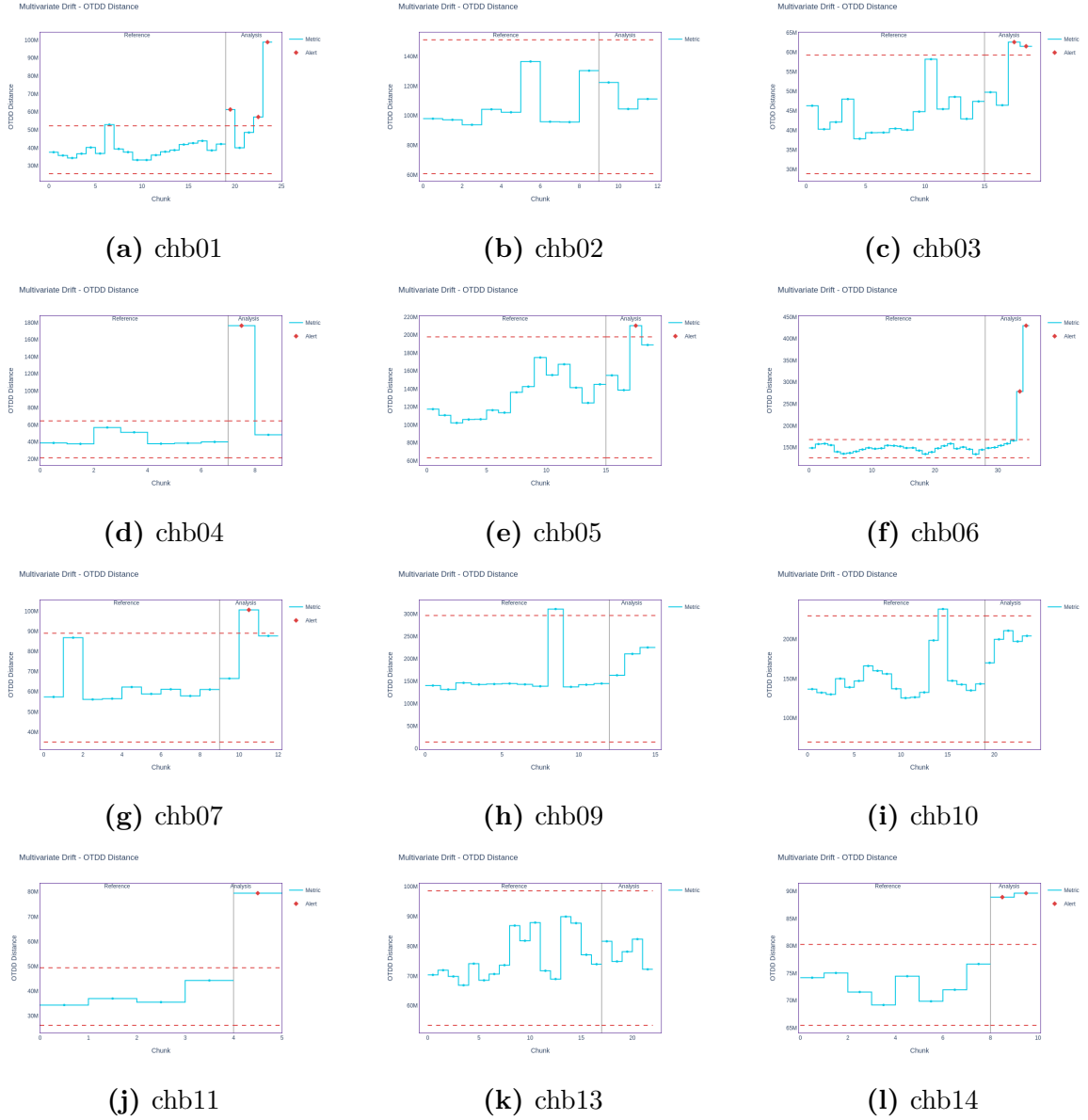


Figure A.10: Optimal Transport Drift Detection: Random Split for subjects in the CHBMIT dataset. 10,000 reference samples, 500-sampled test chunks.

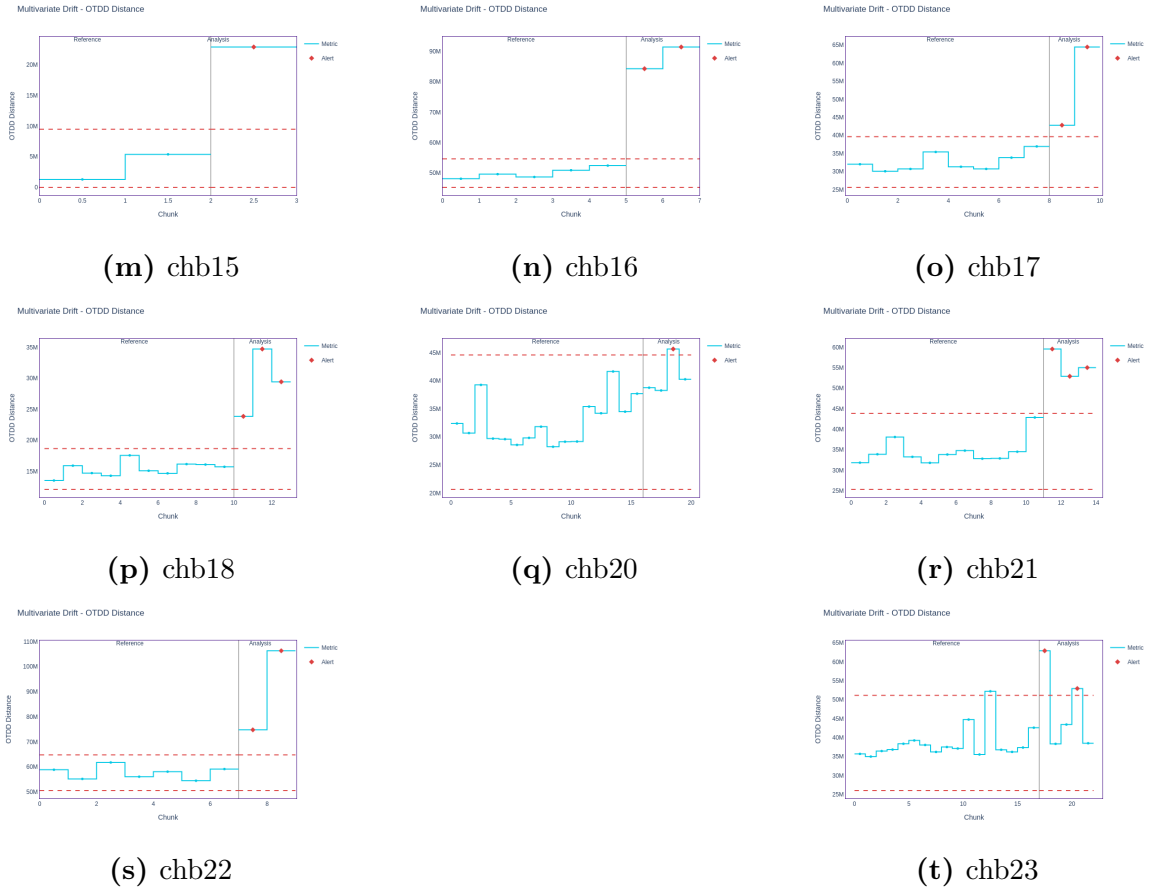


Figure A.10: Optimal Transport Drift Detection: Random Split for subjects in the CHBMIT dataset. 10,000 reference samples, 500-sampled test chunks.

Appendix A. Supplementary Material

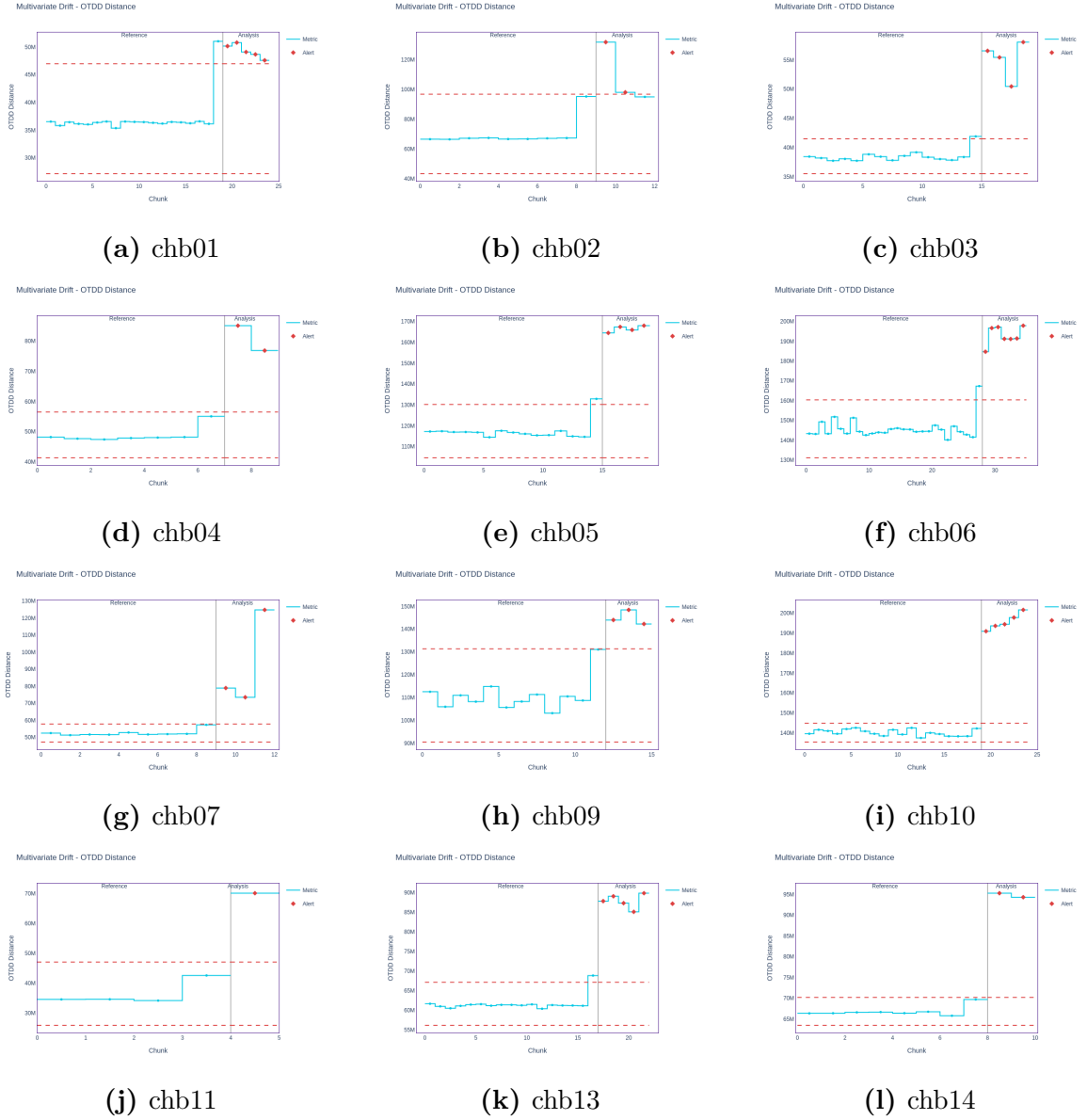


Figure A.11: Optimal Transport Drift Detection: Temporal Split for subjects in the CHBMIT dataset. 10,000 reference samples, 500-sampled test chunks.

Appendix A. Supplementary Material

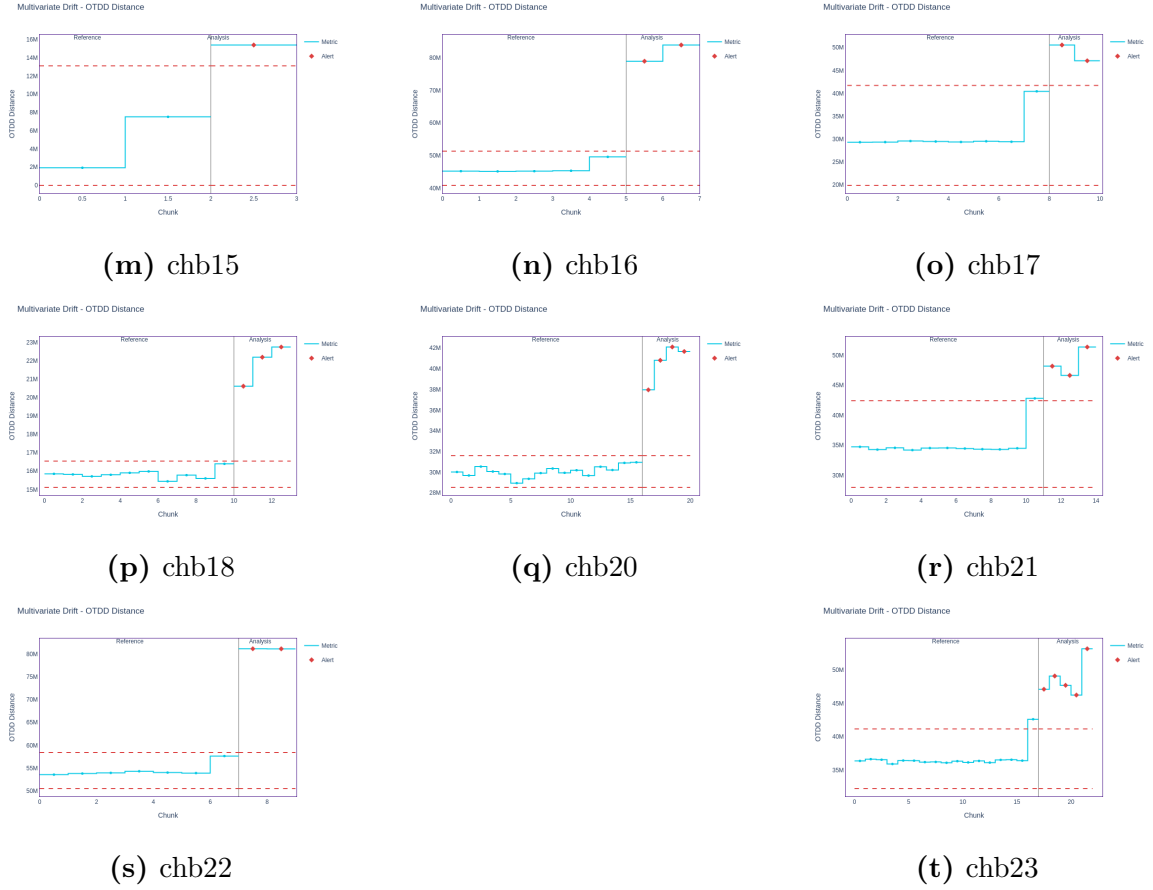


Figure A.11: Optimal Transport Drift Detection: Temporal Split for subjects in the CHBMIT dataset. 10,000 reference samples, 500-sampled test chunks.